

BGV/BFV Bootstrapping

梁朝阳

2026 年 4 月 18 日

目录

引言	20
0.1 为什么要讲 Bootstrapping	20
0.2 本讲义的目标	20
0.3 讲解路线	21
第一章 代数与数论基础：从分圆环到 Slot 几何	22
1.1 为什么必须先补这章	22
1.1.1 <i>Bootstrapping</i> 中真正困难的地方	22
1.1.2 本章将解决哪些符号问题	22
1.2 环、商环与多项式模	22
1.2.1 从整数到多项式环	22
1.2.2 商环的含义	23
1.2.3 <i>BGV/BFV</i> 的明文环	23
1.2.4 子环 R'_{p^e} 的预告	24
1.3 分圆多项式基础	24
1.3.1 单位根与本原单位根	24
1.3.2 分圆多项式的定义	24
1.3.3 <i>power-of-two</i> 情形	24
1.4 有限域、有限环与 Galois ring 视角	25
1.4.1 $\mathbb{F}_p$ 与 $\mathbb{Z}_{p^e}$ 的区别	25
1.4.2 为什么先研究模 p 再提升到模 p^e	25
1.4.3 <i>slot algebra</i> 的预告	26
1.5 群论基础： $\mathbb{Z}_m^\times$ 与元素的阶	26
1.5.1 模 m 乘法群	26
1.5.2 元素的阶	26
1.5.3 这里的关键量 $d = \text{ord}_m(p)$	26
1.6 Frobenius 自同构与轨道	27

1.6.1	<i>Frobenius</i> 映射.....	27
1.6.2	为什么会出现 ζ^{hp^j}	27
1.7	模 p^e 下分圆多项式的分解型.....	28
1.7.1	模 p 下的分解	28
1.7.2	从模 p 提升到模 p^e	28
1.7.3	因此得到的核心参数关系.....	29
1.7.4	<i>power-of-two</i> 情形下因子的稀疏形状	29
1.8	中国剩余定理与 <i>Slot</i> 分解	31
1.8.1	<i>CRT</i> 的一般形式	31
1.8.2	应用到 R_{p^e}	31
1.8.3	<i>Slot</i> 的定义	31
1.8.4	标准 <i>slot algebra</i> E	31
1.9	<i>Abel</i> 群直积分解与 <i>Slot</i> 编号集 S	32
1.9.1	为什么 <i>slot</i> 要编号	32
1.9.2	商群 $\mathbb{Z}_m^\times / \langle p \rangle$ 的来源	32
1.9.3	有限 <i>Abel</i> 群分解定理	32
1.9.4	将商群坐标化.....	32
1.9.5	为什么这套编号特别适合 <i>rotation</i>	33
1.10	<i>power-of-two</i> 情况下 <i>Automorphism</i> 群的具体结构.....	33
1.10.1	$\mathbb{Z}_{2^r}^\times$ 的分解	33
1.10.2	$p \equiv 1 \pmod{4}$ 与 $p \equiv 3 \pmod{4}$	33
1.11	<i>Slot</i> 几何与一维旋转	34
1.11.1	多维 <i>slot</i> 坐标	34
1.11.2	<i>one-dimensional rotation</i> 的定义.....	34
1.11.3	<i>good dimension</i> 与 <i>bad dimension</i> 的预告	34
1.12	自同构 $a(X) \mapsto a(X^j)$ 的代数意义.....	35
1.12.1	为什么 $j \in \mathbb{Z}_m^\times$ 对应一个自同构	35
1.12.2	系数视角下的含义	35
1.12.3	<i>slot</i> 视角下的含义.....	35
1.13	稀疏子环与换元 $Y = X^c$	36
1.13.1	为什么 <i>sparse packed</i> 会落到子环.....	36
1.13.2	为什么会出现 $q(Y)$	36
1.14	基底问题: 局部幂基、统一幂基与系数基.....	37
1.14.1	整个明文环的系数基.....	37
1.14.2	第 i 个 <i>slot</i> 的局部幂基.....	37

1.14.3	统一参考 <i>slot algebra</i> 的幂基	37
1.14.4	为什么三套基不能混	38
1.14.5	M 与 M^{-1} 的本质	38
1.15	从 CRT 到插值：为什么会出现 Vandermonde/FFT-like 矩阵	38
1.15.1	CRT 与“在若干点上求值”的等价	38
1.15.2	因此出现 Vandermonde 矩阵	39
1.15.3	为什么它又像 FFT	39
1.15.4	<i>sparse</i> 与 <i>fully packed</i> 的区别	39
1.16	参数关系总表与推导链	39
1.16.1	核心参数	39
1.16.2	最重要的推导链	40
1.17	本章小结：后续 Bootstrapping 需要哪些代数工具	40

第二章 BGV/BFV 的基础代数框架 42

2.1	本章目标	42
2.2	明文环与密文环	42
2.2.1	分圆环作为底层工作空间	42
2.2.2	明文空间的两种视角	43
2.2.3	子环 R'_{p^e} 的位置	43
2.3	BGV/BFV 的密文结构	44
2.3.1	RLWE 视角	44
2.3.2	秘密钥与公钥的角色	44
2.3.3	为什么噪声是不可避免的	44
2.4	BGV 的解密关系	45
2.4.1	BGV 的基本形状	45
2.4.2	为什么这条公式足以说明 BGV 的核心任务	45
2.4.3	BGV 中噪声为什么是“ p^e 的倍数”特别有利	46
2.5	BFV 的解密关系	46
2.5.1	BFV 的基本形状	46
2.5.2	解密步骤的性质	46
2.5.3	与 BGV 的核心区别	46
2.6	BGV 与 BFV：为什么都属于“同态解密”	47
2.6.1	同态解密的抽象模板	47
2.6.2	为什么不是“先解密再加密”	47
2.6.3	同态解密的两部分	47

2.7	噪声增长与 Leveled FHE	48
2.7.1	为什么 <i>Leveled FHE</i> 不够	48
2.7.2	噪声增长的来源	48
2.7.3	为什么“刷新噪声”不是免费操作	48
2.8	Bootstrapping 前的表示问题	48
2.8.1	同一个明文有两套常用表示	48
2.8.2	为什么 <i>bootstrapping</i> 不会改变密文本体	49
2.9	BGV/BFV 与 CKKS 的 bootstrapping 差别	49
2.9.1	三者都包含 <i>CoeffToSlot</i> / <i>SlotToCoeff</i>	49
2.9.2	但核心非线性不同	49
2.9.3	为什么这会影响后续优化	49
2.10	本章小结	50

第三章 明文空间、CRT 与 SIMD 打包 51

3.1	什么是 batching / SIMD	51
3.1.1	为什么一个密文里想装多个消息	51
3.1.2	<i>SIMD</i> 的直观含义	51
3.1.3	为什么 <i>BGV/BFV</i> 天然适合 <i>batching</i>	52
3.1.4	<i>batching</i> 与 <i>bootstrapping</i> 的关系	52
3.2	CRT 分解的基本思想	52
3.2.1	明文环如何分裂成多个 <i>slot</i>	52
3.2.2	为什么这就叫“分裂成多个 <i>slot</i> ”	53
3.2.3	用根来理解 <i>slot</i> 分裂	53
3.2.4	<i>slot</i> 数为什么等于 $\ell = N/d$	54
3.2.5	<i>slot algebra</i> 的含义	54
3.2.6	“ <i>slot</i> 不是系数”的正确理解	55
3.3	<i>slot</i> 表示与 coefficient 表示	56
3.3.1	<i>coefficient view</i>	56
3.3.2	<i>slot view</i>	57
3.3.3	两种表示为何都重要	58
3.4	<i>sparse packed</i> 与 <i>fully packed</i>	58
3.4.1	<i>sparse packed</i> 的定义	58
3.4.2	<i>fully packed</i> 的定义	59
3.4.3	两者的本质差别	59
3.4.4	什么时候两者会退化成一回事	60
3.5	子环、稀疏幂次与 packing 结构	61

3.5.1	子环的意义	61
3.5.2	为什么 <i>sparse packed</i> 会落在子环	61
3.5.3	X^c 变量替换与“外层多项式”	62
3.6	本章小结	63

第四章 Automorphism、rotation 与线性变换 65

4.1	自同构群的基本概念	65
4.1.1	为什么这一章重要	65
4.1.2	抽象定义	65
4.1.3	我们关心哪个环的自同构	66
4.1.4	为什么分圆环的自同构很特殊	66
4.1.5	分圆环自同构群的结构	66
4.1.6	<i>power-of-two</i> 情形下的群结构回顾	67
4.2	$a(X) \mapsto a(X^j)$ 的真正含义	67
4.2.1	从 $\zeta_m \mapsto \zeta_m^j$ 到 $a(X) \mapsto a(X^j)$	67
4.2.2	为什么 j 必须属于 $\mathbb{Z}_m^\times$	68
4.2.3	在多项式系数视角下会发生什么	68
4.2.4	一个简单例子	69
4.2.5	为什么这一步对密文实现有意义	69
4.3	在 slot 视角下的作用	70
4.3.1	<i>slot permutation</i>	70
4.3.2	为什么是乘法而不是加法	70
4.3.3	在商群上的良定义性	70
4.3.4	slot 内部 <i>Frobenius</i>	71
4.4	one-dimensional rotation 的定义	72
4.4.1	为什么需要“坐标化”的 slot 几何	72
4.4.2	slot 索引群的直积表示	72
4.4.3	定义 <i>one-dimensional rotation</i>	72
4.4.4	为什么这看起来像“循环移位”	72
4.4.5	与 <i>automorphism</i> 的关系	73
4.5	为什么 rotation 不一定等于单个 automorphism	73
4.5.1	表面上的困惑	73
4.5.2	关键区别：商群里的阶 vs 原群里的值	74
4.5.3	为什么这会影响 rotation 的实现	74
4.5.4	<i>good dimension</i> 与 <i>bad dimension</i> 的预告	74
4.6	掩码、两支 automorphism 与 bad/good dimension	75

4.6.1	<i>good dimension</i> 的定义	75
4.6.2	<i>bad dimension</i> 的定义	75
4.6.3	<i>bad dimension</i> 中发生了什么	75
4.6.4	为什么会出现“掩码 + 两支 <i>automorphism</i> ”	75
4.6.5	为什么 <i>good dimension</i> 可以省掉一半麻烦	76
4.7	一般线性变换如何写成 rotations 的加权和	77
4.7.1	问题背景	77
4.7.2	先从最简单的循环情形说起	77
4.7.3	把这个思想搬到 <i>slot</i> 几何里	78
4.7.4	为什么这件事对 <i>bootstrapping</i> 至关重要	78
4.7.5	<i>bad dimension</i> 下的细化	79
4.7.6	从 <i>weighted rotations</i> 到 <i>BSGS</i> 的桥梁	79
4.8	本章小结	79

第五章 BGV/BFV 中的 CoeffToSlot 与 SlotToCoeff 81

5.1	这两个变换到底是什么	81
5.1.1	从表示方向理解	81
5.1.2	从 <i>bootstrapping</i> 流程角色理解	82
5.1.3	容易混淆的命名问题	83
5.2	sparse packed 下的 CoeffToSlot / SlotToCoeff	84
5.2.1	为什么只剩一个 U	84
5.2.2	为什么这时 U 可以理解成 <i>CRT/Vandermonde</i> 矩阵	85
5.3	fully packed 下的 CoeffToSlot / SlotToCoeff	87
5.3.1	局部幂基与统一幂基	87
5.3.2	M 、 U 、 M^{-1} 的分工	88
5.3.3	为什么不能直接乘 U	89
5.4	unpack / repack 的必要性	90
5.4.1	为什么 <i>fully packed</i> 时要 <i>unpack</i>	90
5.4.2	<i>unpack</i> 之后发生了什么	91
5.4.3	<i>repack</i> 在哪里出现	92
5.5	本章小结	93

第六章 BGV Bootstrapping 的完整流程 95

6.1	总体框图	95
6.2	模数提升 (modulus raising / lifting)	96
6.2.1	为什么 <i>bootstrapping</i> 之前必须先“变大模数”	96

6.2.2	模数提升的抽象形式	96
6.2.3	为什么模数提升在 <i>BGV</i> 中是自然的	96
6.2.4	模数提升不等于噪声刷新	97
6.3	同态线性解密	97
6.3.1	从密文得到加密的 <i>noisy plaintext</i>	97
6.3.2	为什么这一步还没有刷新噪声	98
6.4	CoeffToSlot	99
6.4.1	为什么需要先开到 <i>slots</i>	99
6.4.2	如果不做会发生什么	100
6.5	unpack (如适用)	100
6.5.1	什么时候需要 <i>unpack</i>	100
6.5.2	<i>unpack</i> 的目标	101
6.5.3	<i>unpack</i> 的递归结构	101
6.5.4	为什么 <i>bootstrapping</i> 中 <i>unpack</i> 很昂贵	101
6.6	digit extraction / modular reduction	102
6.6.1	这一步要解决什么问题	102
6.6.2	为什么可以逐槽做	102
6.6.3	与 <i>CRT/slot</i> 的关系	103
6.6.4	<i>digit extraction</i> 的基本思想	103
6.6.5	为什么它是非线性核心	104
6.7	repack (如适用)	104
6.7.1	为什么 <i>digit extraction</i> 之后还不能直接结束	104
6.7.2	<i>repack</i> 的作用	104
6.7.3	为什么 <i>repack</i> 常与后续线性变换融合	104
6.8	SlotToCoeff	105
6.8.1	它在流程中的位置	105
6.8.2	<i>sparse packed</i> 与 <i>fully packed</i> 的形式差别	105
6.8.3	为什么 <i>bootstrapping</i> 必须回到 <i>coefficient view</i>	105
6.9	输出新密文与后处理	105
6.9.1	<i>key switching</i> / <i>relinearization</i>	105
6.9.2	回到正常计算模数层	106
6.10	full bootstrap 与 thin bootstrap 的区别	106
6.10.1	<i>full bootstrap</i>	106
6.10.2	<i>thin bootstrap</i>	107
6.10.3	两者的选择逻辑	107

6.11 本章小结	107
第七章 BGV 例子：从头到尾走一遍	109
7.1 参数设定与问题目标	109
7.1.1 一个教学型参数组	109
7.1.2 这个例子会展示什么	110
7.1.3 本章使用的两种层次	110
7.2 解密表达式的写法	111
7.2.1 旧密文的抽象形式	111
7.2.2 BGV 的核心关系	111
7.2.3 为什么这条公式是 <i>bootstrapping</i> 的起点	111
7.2.4 提升到 <i>bootstrapping</i> 模数后的写法	112
7.2.5 本章的记号约定	112
7.3 noisy plaintext 的结构	112
7.3.1 <i>coefficient view</i> 下的结构	112
7.3.2 <i>slot view</i> 下的结构	113
7.3.3 在本例中为什么每个 <i>slot</i> 不是标量	113
7.3.4 为什么这会迫使我们 <i>unpack</i>	113
7.4 CoeffToSlot 的结果长什么样	113
7.4.1 先写出变换前后的形式	113
7.4.2 用统一幂基写法表达	114
7.4.3 这一步在流程中真正得到什么	114
7.4.4 和 <i>sparse packed</i> 的对比	114
7.5 fully packed 时的 <i>unpack</i>	115
7.5.1 目标：把四维 <i>slot</i> 拆成标量流	115
7.5.2 第一层：偶奇分裂	115
7.5.3 第二层：继续拆成标量	116
7.5.4 作用到两个 <i>slots</i> 上的结果	116
7.5.5 为什么这样就可以 <i>digit extract</i> 了	116
7.6 digit extraction 的逐步操作	117
7.6.1 核心目标	117
7.6.2 <i>base-p</i> 视角	117
7.6.3 <i>base-p^k</i> 视角	118
7.6.4 低位保留与高位丢弃	119
7.7 repack 与 SlotToCoeff	119
7.7.1 <i>digit extraction</i> 之后我们拥有什么	119

7.7.2	<i>repack</i> : 把四条标量流重新装回两个 <i>fully packed slots</i>	120
7.7.3	<i>SlotToCoeff</i> : 把 <i>slot</i> 向量重新装回环元素	120
7.7.4	最终输出的对象是什么	120
7.8	为什么最终得到的是同一明文的新密文	120
7.8.1	逻辑链条	120
7.8.2	关键事实: 除了 <i>digit extraction</i> , 其他步骤都不改变消息语义	121
7.9	这个例子里每一步的代价来源.....	122
7.9.1	模数提升的代价	122
7.9.2	同态线性解密的代价	122
7.9.3	<i>CoeffToSlot</i> 的代价	122
7.9.4	<i>unpack</i> 的代价	123
7.9.5	<i>digit extraction</i> 的代价	123
7.9.6	<i>repack</i> 与 <i>SlotToCoeff</i> 的代价	123
7.9.7	总成本的结构性来源	124
7.10	本章小结	124

第八章 *unpack* / *repack* 专题 126

8.1	<i>fully packed</i> 为什么一定会遇到它们	126
8.1.1	问题的根源: <i>slot</i> 里装的不是标量	126
8.1.2	为什么 <i>sparse packed</i> 没有这个问题	126
8.1.3	为什么不能直接对 E_i -元素做 <i>digit extraction</i>	127
8.1.4	从 <i>bootstrapping</i> 流程看 <i>unpack</i> 的位置	127
8.1.5	从维数角度看为什么一定要拆	127
8.2	<i>even/odd splitting</i> 的思想	128
8.2.1	拆分问题的抽象形式	128
8.2.2	一个最基本的代数恒等式	128
8.2.3	为什么在这里 T 对应 ζ_m	128
8.2.4	<i>even/odd splitting</i> 的基本构造	129
8.2.5	为什么这里要除以 2 和 ζ_m	129
8.3	递归分裂的结构	130
8.3.1	从 d -维到 $d/2$ -维	130
8.3.2	继续递归: 从 $d/2$ -维到 $d/4$ -维	130
8.3.3	递归到标量为止	131
8.3.4	递归树视角	131
8.3.5	系数索引如何被递归编码	132
8.4	从 E -slot 到标量 slot 的过程	132

8.4.1	起点：一个 <i>E-slot</i>	132
8.4.2	目标：标量 <i>slot</i>	132
8.4.3	对所有 <i>slots</i> 同时递归分裂	133
8.4.4	形式化描述	133
8.4.5	为什么这正是 <i>digit extraction</i> 需要的格式	133
8.5	repack 为什么是逆向重组	134
8.5.1	为什么 <i>unpack</i> 之后不能停下	134
8.5.2	repack 的目标	134
8.5.3	为什么说它是 <i>unpack</i> 的逆过程	134
8.5.4	为什么 <i>repack</i> 不是简单拼接	135
8.5.5	repack 与 <i>SlotToCoeff</i> 的关系	135
8.6	unpack / repack 的复杂度与瓶颈	135
8.6.1	复杂度的三个主要来源	135
8.6.2	递归层数带来的复杂度	136
8.6.3	为什么 <i>fully packed</i> 显著更贵	136
8.6.4	为什么它们是 <i>bootstrapping</i> 的主要瓶颈之一	136
8.6.5	复杂度的结构性总结	137
8.7	本章小结	137

第九章 digit extraction 专题 139

9.1	digit extraction 的数学目标	139
9.1.1	从 <i>BGV</i> 的 <i>noisy plaintext</i> 出发	139
9.1.2	最核心的一句话	139
9.1.3	为什么叫 <i>digit extraction</i>	140
9.1.4	一个最基本的数值直觉	140
9.1.5	抽象的代数表达	140
9.2	“模 p^e ”到底是在什么对象上做	141
9.2.1	最容易混淆的问题	141
9.2.2	第一层：在大环里的意思	141
9.2.3	第二层：在 <i>CRT</i> 分解后的意思	141
9.2.4	第三层：在 <i>E-slot</i> 里的问题	142
9.2.5	为什么 <i>bootstrapping</i> 实现里还要进一步拆成标量槽	142
9.2.6	结论：三层对象不要混	143
9.3	为什么 <i>CRT</i> 后可以逐槽处理	143
9.3.1	核心命题	143
9.3.2	这一定理到底说明了什么	144

9.3.3	为什么这正是 <i>packed bootstrapping</i> 的基础	144
9.3.4	为什么这仍不意味着“slot 就是系数”	145
9.3.5	为什么实现上还要注意 slot 的类型	145
9.4	标量槽与 <i>E</i> -槽的区别	145
9.4.1	标量槽是什么	145
9.4.2	<i>E</i> -槽是什么	146
9.4.3	二者的本质差别	146
9.4.4	为什么 <i>fully packed</i> 的核心困难就在这里	146
9.4.5	一个统一记忆方式	147
9.5	常见 digit extraction 思路	147
9.5.1	<i>base-p</i> 逐位提取	147
9.5.2	<i>base-p^k</i> 分块提取	148
9.5.3	<i>carry / correction</i> 视角	149
9.6	digit extraction 的电路深度来源	150
9.6.1	为什么 digit extraction 是 <i>bootstrapping</i> 的非线性瓶颈	150
9.6.2	深度来自哪里	150
9.6.3	<i>base-p</i> 与 <i>base-p^k</i> 对深度的影响	151
9.6.4	为什么这和线性变换优化不同	151
9.6.5	为什么 <i>fully packed</i> 还会额外放大 digit extraction 的代价	151
9.7	digit extraction 与 BFV / CKKS 中对应步骤的对比	152
9.7.1	与 BFV 的对比	152
9.7.2	与 CKKS 的对比	152
9.8	本章小结	154

第十章 线性变换实现与 BSGS 155

10.1	为什么线性变换是 bootstrapping 的大头	155
10.1.1	<i>bootstrapping</i> 中不止有一个“大矩阵”	155
10.1.2	为什么 <i>CoeffToSlot / SlotToCoeff</i> 特别贵	156
10.1.3	为什么 <i>unpack / repack</i> 同样属于线性变换问题	156
10.1.4	为什么要单独研究 BSGS	157
10.2	weighted rotations 形式	157
10.2.1	从普通矩阵乘法到 <i>rotation</i> 分解	157
10.2.2	循环置换基分解回顾	157
10.2.3	weighted rotations 的 FHE 解释	158
10.2.4	多维 slot 几何中的写法	158
10.2.5	为什么这是 bootstrapping 的统一语言	159

10.3	BSGS 的基本思想	159
10.3.1	为什么直接实现 <i>weighted rotations</i> 很贵	159
10.3.2	BSGS 的核心想法	160
10.3.3	直观图像	160
10.3.4	为什么要叫 <i>baby-step / giant-step</i>	160
10.3.5	一个简单例子	161
10.3.6	为什么能减少 <i>automorphism</i> 数量	161
10.4	one-dimensional BSGS	162
10.4.1	问题设定	162
10.4.2	BSGS 分块	163
10.4.3	实现上的组织方式	163
10.4.4	为什么一维 BSGS 特别适合 <i>rotation</i> 群	164
10.4.5	复杂度平衡	164
10.5	multidimensional BSGS	164
10.5.1	为什么需要多维版本	164
10.5.2	最基本的多维思路	165
10.5.3	为什么这叫 <i>multidimensional</i> 而不是 <i>many one-dimensional</i>	165
10.5.4	多维分块的一个典型情景	165
10.5.5	为什么多维结构对 <i>bootstrapping</i> 特别重要	166
10.6	good dimension / bad dimension 对成本的影响	166
10.6.1	回顾 <i>good / bad dimension</i>	166
10.6.2	在 <i>good dimension</i> 中的成本	166
10.6.3	在 <i>bad dimension</i> 中的成本	167
10.6.4	为什么这会影响最优分块	167
10.6.5	一个实践层面的结论	167
10.7	hoisting / double-hoisting	168
10.7.1	为什么只有减少 <i>rotation</i> 数量还不够	168
10.7.2	<i>hoisting</i> 的思想	168
10.7.3	一个抽象描述	168
10.7.4	<i>double-hoisting</i> 的思想	169
10.7.5	为什么 <i>hoisting</i> 与 BSGS 是天然搭档	169
10.8	BSGS 在 <i>CoeffToSlot / SlotToCoeff</i> 中的作用	169
10.8.1	<i>CoeffToSlot / SlotToCoeff</i> 为什么天然适合 BSGS	169
10.8.2	在 <i>sparse packed</i> 中的作用	170
10.8.3	在 <i>fully packed</i> 中的作用	170
10.8.4	为什么 <i>FFT-like</i> 分解与 BSGS 可以同时存在	170

10.8.5 实际意义: 为什么它们决定了 <i>bootstrapping</i> 是否可行.....	171
10.9 本章小结	171

第十一章 稀疏矩阵分解与 FFT-like 结构 173

11.1 为什么要分解 U 矩阵	173
11.1.1 从“一个大矩阵”到“很多小层”	173
11.1.2 为什么“直接实现一个大矩阵”很贵.....	173
11.1.3 <i>FFT</i> 的经验: 大密集矩阵往往隐藏着递归结构	174
11.1.4 为什么 <i>BGV/BFV</i> 中的 U 值得期待类似结构	174
11.1.5 分解 U 的真正目标	174
11.2 列置换版本的意义	175
11.2.1 为什么要先做列置换.....	175
11.2.2 列置换在矩阵论中的一般含义	175
11.2.3 为什么列顺序会影响递归分块	175
11.2.4 <i>BGV/BFV</i> 里为什么特别需要列置换	176
11.2.5 列置换不会改变 <i>bootstrapping</i> 的数学目标.....	176
11.2.6 一个最简单的例子	176
11.3 蝶形分解思想	177
11.3.1 为什么叫“蝶形”	177
11.3.2 从偶奇分裂得到蝶形	177
11.3.3 矩阵形式.....	178
11.3.4 为什么这正是 U 分解的核心.....	178
11.3.5 与 <i>unpack</i> 的 <i>even/odd splitting</i> 的相似性	178
11.4 稀疏矩阵乘积表示	179
11.4.1 从大矩阵到递归乘积	179
11.4.2 为什么说“稀疏”	179
11.4.3 一个抽象递归定理	180
11.4.4 为什么这一定理足够适用于 <i>bootstrapping</i> 语境	180
11.5 为什么这种分解有利于同态实现	181
11.5.1 从一次“大求和”变成很多次“小求和”	181
11.5.2 为什么少量模式特别重要.....	181
11.5.3 层数增加为什么不一定是坏事	182
11.5.4 和 <i>FFT</i> 的复杂度直觉一致.....	182
11.5.5 对 <i>bootstrapping</i> 的直接好处	182
11.6 与 CKKS decoding matrix 分解的关系	183
11.6.1 为什么人们总说“CKKS-like”	183

11.6.2	CKKS decoding matrix 的经典结构	183
11.6.3	BGV/BFV 与 CKKS 在这一步上的共同点	183
11.6.4	但它们并不完全相同	184
11.6.5	为什么这种“像但不等于”恰好最有价值	184
11.7	BGV/BFV 中“像 FFT 但不完全一样”的地方	184
11.7.1	相似之处：都在做分治的 roots-of-unity 求值	184
11.7.2	不同之处一：roots of unity 的来源不同	185
11.7.3	不同之处二：fully packed 时还多了一层内部基结构	185
11.7.4	不同之处三：rotation 实现受 good/bad dimension 影响	185
11.7.5	不同之处四：FHE 中层数与稀疏度的权衡不同	186
11.7.6	一个统一理解	186
11.8	本章小结	186

第十二章 fully packed 与 sparse packed 的优化路线 188

12.1	sparse packed 为什么更简单	188
12.1.1	从对象类型上看：slot 是否是标量	188
12.1.2	从线性变换结构上看：是否需要 M 与 M^{-1}	188
12.1.3	从子环视角看：问题维数更小	189
12.1.4	从 bootstrapping 流程上看：中间环节更少	189
12.1.5	从实现层面看：更容易吃满现有优化	190
12.1.6	一个总结性判断	190
12.2	fully packed 为什么更贵	190
12.2.1	最根本的原因：吞吐量不是免费的	190
12.2.2	第一层代价：slot 内部基结构	191
12.2.3	第二层代价：digit extraction 不能直接作用	191
12.2.4	第三层代价：automorphism / key switching 爆炸	191
12.2.5	第四层代价：层数与中间密文数量	192
12.2.6	第五层代价：优化空间更碎片化	192
12.2.7	总结：fully packed 的成本本质	193
12.3	何时值得做 full bootstrap	193
12.3.1	问题不是“能不能做”，而是“值不值”	193
12.3.2	当后续计算强依赖高吞吐 SIMD 时	193
12.3.3	当 ciphertext 数量本身是瓶颈时	194
12.3.4	当参数规模允许有效 amortization 时	194
12.3.5	当应用需要保持表示一致性时	194
12.3.6	一个经验性判断	195

12.4	何时 thin / sparse 更合适	195
12.4.1	当你最在乎的是“刷新便宜”而不是“刷新后最满”	195
12.4.2	当后续计算不需要满槽并行时	195
12.4.3	当 d 太大时	196
12.4.4	当想优先验证新 <i>bootstrapping</i> 原型时	196
12.4.5	当系统更关心“峰值时延”而非“平均吞吐”时	196
12.4.6	一个经验性判断	197
12.5	子环优化	197
12.5.1	为什么子环是 <i>sparse</i> 路线的天然舞台	197
12.5.2	子环优化带来的直接收益	197
12.5.3	为什么子环优化与 <i>sparse packed</i> 高度绑定	198
12.5.4	子环优化并不等于“信息丢失”	198
12.5.5	与本文主题的直接关系	198
12.6	trace / coefficient selection 的作用	199
12.6.1	为什么会出现 <i>trace</i>	199
12.6.2	抽象定义	199
12.6.3	在 <i>BGV/BFV</i> 中它为什么重要	199
12.6.4	<i>coefficient selection</i> 是什么	200
12.6.5	为什么 <i>trace / selection</i> 对 <i>thin</i> 路线特别重要	200
12.6.6	一个结构性理解	200
12.7	partial <i>CoeffToSlot</i> 的概念与用途	201
12.7.1	为什么会想到“只做一半 <i>CoeffToSlot</i> ”	201
12.7.2	partial <i>CoeffToSlot</i> 的概念	201
12.7.3	为什么这样做可能更便宜	201
12.7.4	它的用途在哪里	202
12.7.5	与本文主题的关系	202
12.7.6	一个统一理解	202
12.8	本章小结	203

第十三章 复杂度与成本来源 204

13.1	评估指标	204
13.1.1	乘法深度	204
13.1.2	<i>automorphism</i> 数量	205
13.1.3	<i>plaintext-ciphertext multiplication</i> 数量	206
13.1.4	<i>key switching</i> 代价	207
13.2	哪些步骤最贵	208

13.2.1	不能只问“哪一步最慢”	208
13.2.2	第一类大头: <i>CoeffToSlot</i> / <i>SlotToCoeff</i>	209
13.2.3	第二类大头: <i>digit extraction</i>	209
13.2.4	第三类大头: <i>unpack</i> / <i>repack</i>	209
13.2.5	不同语境下的“最贵”不同	210
13.2.6	所以成本分析应分层做	210
13.3	fully packed 与 sparse packed 的复杂度对比	210
13.3.1	先给一个总判断	210
13.3.2	线性层规模的差别	210
13.3.3	<i>rotation</i> / <i>key switching</i> 数量的差别	211
13.3.4	中间密文数量的差别	211
13.3.5	平均每消息成本不一定谁更高	212
13.3.6	一个结构性总结	212
13.4	BSGS 带来的下降	212
13.4.1	从朴素实现到 <i>BSGS</i>	212
13.4.2	为什么这是 <i>bootstrapping</i> 里最实在的收益之一	213
13.4.3	不是所有成本都按 $\sqrt{n}$ 降	213
13.4.4	配合 <i>hoisting</i> 后的实际下降	213
13.4.5	对 <i>full</i> 与 <i>sparse</i> 路线的意义不同	214
13.5	<i>unpack</i> / <i>repack</i> 的额外成本	214
13.5.1	为什么这是 <i>fully packed</i> 的“额外账”	214
13.5.2	递归层数带来的成本	214
13.5.3	中间态爆炸的成本	215
13.5.4	为什么它们常成为 <i>fully packed</i> 的主惩罚项	215
13.6	<i>digit extraction</i> 的成本占比	215
13.6.1	为什么要单独看占比	215
13.6.2	从深度看: 它通常是第一大头	216
13.6.3	从运行时间看: 它未必总是第一大头	216
13.6.4	为什么优化 <i>digit extraction</i> 仍然最关键	216
13.7	参数选择对复杂度的影响	217
13.7.1	参数不是只影响安全性, 也直接影响 <i>bootstrapping</i> 成本	217
13.7.2	$d = \text{ord}_m(p)$ 的影响	217
13.7.3	ℓ 的影响	218
13.7.4	p^e 的影响	218
13.7.5	Q 与 <i>bootstrapping</i> 模数链的影响	218
13.7.6	<i>good</i> / <i>bad dimension</i> 相关参数的影响	219

13.7.7 一个统一视角	219
13.8 本章小结	219

附录 A 伪代码附录：从原始数据到 Bootstrapping 再回到原始数据 221

A.1 统一记号与对象约定	221
A.1.1 基础参数	221
A.1.2 打包模式	221
A.1.3 方案标志	222
A.1.4 辅助对象	222
A.1.5 密文类型	222
A.2 参数生成与预处理	223
A.2.1 算法 A.1: <i>SetupBGVBFVBootstrap</i>	223
A.3 编码与解码	225
A.3.1 算法 A.2: <i>EncodeData</i>	225
A.3.2 算法 A.3: <i>DecodeData</i>	226
A.4 加密与解密	227
A.4.1 算法 A.4: <i>Encrypt</i>	227
A.4.2 算法 A.5: <i>Decrypt</i>	228
A.5 基础同态运算与 key switching	229
A.5.1 算法 A.6: <i>Add</i>	229
A.5.2 算法 A.7: <i>MulAndRelinearize</i>	229
A.5.3 算法 A.8: <i>KeySwitch</i>	230
A.5.4 算法 A.9: <i>Rotate</i>	231
A.6 大线性变换: weighted rotations、BSGS 与 hoisting	231
A.6.1 算法 A.10: <i>EvalLinearTransform_BSGS</i>	231
A.7 同态计算主循环	232
A.7.1 算法 A.11: <i>EvalCircuitWithBootstrap</i>	232
A.8 Bootstrapping 的核心伪代码	233
A.8.1 算法 A.12: <i>Bootstrap</i>	233
A.9 unpack / repack 的伪代码	235
A.9.1 算法 A.13: <i>Unpack</i>	235
A.9.2 算法 A.14: <i>Repack</i>	236
A.10 BGV 的 digit extraction 与 BFV 的 rounding 伪代码	237
A.10.1 算法 A.15: <i>DigitExtract_ModT (BGV)</i>	237

A.10.2 算法 A.16: <i>ScaleAndRound</i> (BFV)	238
A.11 从数据到数据的端到端伪代码	239
A.11.1 算法 A.17: <i>EndToEnd_BGVBFV_BootstrapWorkflow</i>	239
A.12 附录总结: 如何读这套伪代码	240

引言

0.1 为什么要讲 Bootstrapping

Bootstrapping 是全同态加密中最核心、也最容易让人迷失的一部分。它一方面涉及 RLWE 密文、噪声增长、模数链、同态解密等密码学概念；另一方面又大量依赖抽象代数、有限域、分圆多项式、中国剩余定理（CRT）、有限 Abel 群分解、Frobenius 轨道、线性变换分解等数学工具。若直接从算法流程切入，往往会出现两个问题：

- (1) 看得见流程图，但看不懂每一步为什么合法；
- (2) 记得住符号，却不知道这些符号到底从哪来、为什么长成那样。

因此，本讲义的第一部分专门补足 BGV/BFV bootstrapping 所需的代数与数论基础。

0.2 本讲义的目标

本讲义希望在进入具体 bootstrapping 流程之前，先把下列问题讲清楚：

- (1) 为什么明文环写成

$$R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1)$$

以后，会自然出现 slot 结构；

- (2) 为什么 slot 的个数是

$$\ell = \frac{N}{d}, \quad d = \text{ord}_m(p), \quad m = 2N;$$

- (3) 为什么 slot 索引集会写成

$$S = \{g_1^{e_1} \cdots g_t^{e_t}\},$$

它与有限 Abel 群直积分解有什么关系；

- (4) 为什么会出现局部幂基、统一幂基、系数基这三套坐标；
- (5) 为什么 sparse packed 情形下会出现一个看起来像 Vandermonde/FFT 的矩阵 U ；
- (6) 为什么 fully packed 情形下不能只看 U ，还要看 M 与 M^{-1} 。

0.3 讲解路线

本讲义后续将按如下路线展开：

1. 先补齐代数与数论基础：分圆多项式、Frobenius 轨道、CRT、Abel 群分解、slot 几何；
2. 再回到 BGV/BFV 的解密关系，解释 bootstrapping 中的 noisy plaintext、CoeffToSlot、unpack、digit extraction、repack、SlotToCoeff；
3. 最后讨论 BSGS、FFT-like 分解、稀疏矩阵分解、thin/full bootstrap 等优化手段。

下面开始第一章。

代数与数论基础：从分圆环到 Slot 几何

1.1 为什么必须先补这章

1.1.1 Bootstrapping 中真正困难的地方

如果只把 bootstrapping 看成一个流程图：

旧密文 $\rightarrow$ 同态解密 $\rightarrow$ CoeffToSlot $\rightarrow$ digit extraction $\rightarrow$ SlotToCoeff $\rightarrow$ 新密文，

那么容易误以为每一步只是“黑盒变换”。但实际上，每一步背后都依赖如下事实：

- 为什么明文空间能被看成若干个 slot 的直积；
- 为什么 automorphism $a(X) \mapsto a(X^j)$ 会对应 slot permutation；
- 为什么所谓的“rotation”是一个群作用而不是随便置换；
- 为什么 sparse packed 明文天然落在某个子环 R'_{p^e} 中；
- 为什么某些线性变换长得像 Vandermonde 矩阵，另一些却必须先换基再做。

这些都不是“实现细节”，而是 bootstrapping 的数学骨架。

1.1.2 本章将解决哪些符号问题

本章将把下列符号的来源与意义统一解释清楚：

$$m, N, p, e, d, \ell, c, m', N', E, E_i, R_{p^e}, R'_{p^e}, S, g_i, \ell_i, \zeta_m, \zeta_{m,i}, \tau_j, \sigma.$$

1.2 环、商环与多项式模

1.2.1 从整数到多项式环

我们从最基本的对象开始。固定一个正整数 q ，记

$$\mathbb{Z}_q := \mathbb{Z}/q\mathbb{Z}.$$

在 BGV/BFV 中，明文系数通常不在域里，而在环 $\mathbb{Z}_{p^e}$ 中；密文系数则在 $\mathbb{Z}_Q$ 中，其中 Q 是大的密文模数。

在此基础上，我们考虑多项式环

$$\mathbb{Z}_q[X].$$

其中元素是系数在 $\mathbb{Z}_q$ 中的多项式。

1.2.2 商环的含义

若给定一个多项式 $f(X) \in \mathbb{Z}_q[X]$ ，则商环

$$\mathbb{Z}_q[X]/(f(X))$$

表示“把所有相差 $f(X)$ 的倍数的多项式视为同一个元素”。在这个商环中， $f(X) = 0$ 被视为一个恒等关系。

特别地，若

$$f(X) = X^N + 1,$$

则在商环里总有

$$X^N = -1, \quad X^{2N} = 1.$$

这意味着任何更高次的幂都能被化简成次数小于 N 的多项式。

1.2.3 BGV/BFV 的明文环

在 power-of-two cyclotomic 情形下，我们令

$$m = 2N,$$

并考虑明文环

$$R_{p^e} := \mathbb{Z}_{p^e}[X]/(X^N + 1).$$

这是 bootstrapping 讨论的核心明文空间。

它有三种重要的理解方式：

(1) **系数视角**：任一元素都可唯一写成

$$a_0 + a_1X + \cdots + a_{N-1}X^{N-1};$$

(2) **分圆视角**：因为 $X^N + 1 = \Phi_m(X)$ ，它是一个 m 次分圆多项式；

(3) **slot 视角**：经过 CRT 分解后，它会变成若干个较小环的直积。

1.2.4 子环 R'_{p^e} 的预告

后面我们会看到，若只允许出现某些稀疏幂次

$$1, X^c, X^{2c}, \dots, X^{(N'-1)c}, \quad N' = \frac{N}{c},$$

则这些元素构成一个子环，通常记作

$$R'_{p^e}.$$

它在 BGV/BFV 的 sparse packed bootstrap 中非常重要。现在先记住：大环 R_{p^e} 之外，还会自然出现更小的子环 R'_{p^e} 。

直观解释 1.1. $\mathbb{Z}_{p^e}[X]/(X^N + 1)$ 可以理解成“长度为 N 的环形系数向量空间”，但这里的“系数”不是普通整数，而是模 p^e 的数；而 $X^N = -1$ 则使这个空间带有丰富的对称性与分解结构。

1.3 分圆多项式基础

1.3.1 单位根与本原单位根

在任意交换环或其扩张中，如果元素 ζ 满足

$$\zeta^m = 1,$$

则称其为 m 次单位根。若进一步满足对所有 $1 \leq r < m$ 都有 $\zeta^r \neq 1$ ，则称 ζ 为 m 次本原单位根。

1.3.2 分圆多项式的定义

第 m 个分圆多项式定义为

$$\Phi_m(X) = \prod_{\substack{1 \leq a \leq m \\ \gcd(a, m) = 1}} (X - \zeta_m^a),$$

其中 ζ_m 是任取的一个 m 次本原单位根。它的次数是欧拉函数

$$\deg \Phi_m = \varphi(m).$$

1.3.3 power-of-two 情形

对本报告最重要的是 $m = 2^{s+1}$ 的情形。此时

$$\varphi(m) = 2^s.$$

而且分圆多项式有极其简单的形式。

定理 1.2 (power-of-two 分圆多项式). 若 $m = 2^{s+1}$ 且 $N = 2^s$, 则

$$\Phi_m(X) = X^N + 1.$$

证明. 由

$$X^{2N} - 1 = (X^N - 1)(X^N + 1)$$

可知 $X^N + 1$ 的根恰好是满足 $\xi^{2N} = 1$ 但 $\xi^N = -1$ 的那些根, 也就是 $2N = m$ 次本原单位根。故

$$\Phi_{2N}(X) = X^N + 1.$$

□

直观解释 1.3. 这一定理告诉我们, power-of-two cyclotomic ring

$$\mathbb{Z}_{p^e}[X]/(X^N + 1)$$

并不是随便挑出来的, 而是恰好对应于 $m = 2N$ 的分圆结构。这也是后面为什么一直出现“模 m ”而不是“模 N ”的根本原因。

1.4 有限域、有限环与 Galois ring 视角

1.4.1 $\mathbb{F}_p$ 与 $\mathbb{Z}_{p^e}$ 的区别

$\mathbb{F}_p$ 是域, 而 $\mathbb{Z}_{p^e}$ 在 $e > 1$ 时不是域。比如在 $\mathbb{Z}_{p^2}$ 中, $p \neq 0$ 但

$$p \cdot p^{e-1} = 0.$$

这说明它有零因子。

因此, BGV/BFV 的明文空间不是“有限域上的向量空间”, 而是“有限环上的模”。

1.4.2 为什么先研究模 p 再提升到模 p^e

虽然真正的明文模数是 p^e , 但很多分解型和轨道结构只由底层素数 p 决定。这是因为:

- Frobenius 自同构的基本形式是 $x \mapsto x^p$;
- 因子次数来自这个 Frobenius 作用的轨道长度;
- 模 p^e 下的因子结构可以看作模 p 下结构的 Hensel 提升。

1.4.3 slot algebra 的预告

后面经 CRT 分解后，每个 slot 会对应一个较小的环

$$E_i = \mathbb{Z}_{p^e}[X]/(F_i(X)).$$

这些小环彼此同构，常统一记为某个标准环 E 。这就是后面反复出现的 slot algebra。

注 1.4. “slot 值不一定是普通整数”这一点非常重要。对 sparse packed 而言，slot 里装的是 $\mathbb{Z}_{p^e}$ 的标量；对 fully packed 而言，slot 里装的一般是 E -元素。

1.5 群论基础： $\mathbb{Z}_m^\times$ 与元素的阶

1.5.1 模 m 乘法群

$$\mathbb{Z}_m^\times := \{a \in \mathbb{Z}_m : \gcd(a, m) = 1\}$$

构成一个有限乘法群，其群运算是模 m 乘法。

1.5.2 元素的阶

定义 1.5. 若 $g \in \mathbb{Z}_m^\times$ ，则其阶定义为最小正整数 r ，使得

$$g^r \equiv 1 \pmod{m}.$$

记作

$$\text{ord}_m(g) = r.$$

命题 1.6. 若 $g \in \mathbb{Z}_m^\times$ ，则 $\text{ord}_m(g)$ 存在且满足

$$\text{ord}_m(g) \mid |\mathbb{Z}_m^\times| = \varphi(m).$$

证明. $\mathbb{Z}_m^\times$ 是有限群，因此任意元素都有有限阶；再由 Lagrange 定理，元素阶整除群的阶。□

1.5.3 这里的关键量 $d = \text{ord}_m(p)$

由于 $\gcd(p, m) = 1$ ，所以 $p \in \mathbb{Z}_m^\times$ 。于是可定义

$$d := \text{ord}_m(p).$$

换句话说， d 是最小正整数，使得

$$p^d \equiv 1 \pmod{m}.$$

直观解释 1.7. 后面你会看到, d 同时控制:

- Frobenius 轨道长度;
- 不可约因子次数;
- 单个 slot 的“内部宽度”;
- slot 数量 $\ell = N/d$ 。

所以 d 是整个 slot 几何的核心参数。

1.6 Frobenius 自同构与轨道

1.6.1 Frobenius 映射

在特征 p 的环境中, 映射

$$\mathrm{Fr}_p : x \mapsto x^p$$

是一个环自同构; 在有限域扩张中, 它就是 Frobenius 自同构。

1.6.2 为什么会出现 ζ^{hp^j}

取一个 m 次本原单位根 ζ (它通常在某个扩域或 Galois ring 中), 则

$$(\zeta^h)^p = \zeta^{hp}.$$

继续作用得到

$$\zeta^h, \zeta^{hp}, \zeta^{hp^2}, \dots$$

这就是 Frobenius 轨道。

定理 1.8 (Frobenius 轨道长度). 设 ζ 是一个 m 次本原单位根, $h \in \mathbb{Z}_m^\times$, 且

$$d = \mathrm{ord}_m(p).$$

则 ζ^h 的 Frobenius 轨道长度恰为 d :

$$\{\zeta^h, \zeta^{hp}, \zeta^{hp^2}, \dots, \zeta^{hp^{d-1}}\}.$$

证明. 因为 $p^d \equiv 1 \pmod{m}$, 所以

$$\zeta^{hp^d} = \zeta^h.$$

故轨道长度整除 d 。

另一方面, 若存在 $0 < r < d$ 使

$$\zeta^{hp^r} = \zeta^h,$$

则

$$hp^r \equiv h \pmod{m}.$$

因 $h \in \mathbb{Z}_m^\times$ 可消去, 得到

$$p^r \equiv 1 \pmod{m},$$

与 d 的最小性矛盾。故轨道长度只能是 d 。 $\square$

直观解释 1.9. 这一定理解释了为什么一个 slot 的“内部维度”不是随便来的: 它就是一个根在 Frobenius 作用下绕一圈所需要的步数。

1.7 模 p^e 下分圆多项式的分解型

1.7.1 模 p 下的分解

先在 $\mathbb{F}_p[X]$ 中看 $\Phi_m(X)$ 的分解。由上面的轨道理论可得:

定理 1.10 (模 p 下的分解次数). 设 $m = 2N$, $\gcd(p, m) = 1$, 令

$$d = \text{ord}_m(p).$$

则在 $\mathbb{F}_p[X]$ 中, $\Phi_m(X)$ 分解成

$$\ell = \frac{\varphi(m)}{d} = \frac{N}{d}$$

个互不相同的不可约因子, 每个因子次数均为 d 。

证明. Φ_m 的全部根是所有 m 次本原单位根, 即

$$\{\zeta^h : h \in \mathbb{Z}_m^\times\}.$$

Frobenius 作用把它们分成若干个互不相交的轨道, 每个轨道长度为 d 。每个轨道对应一个不可约因子:

$$F_h(X) = \prod_{t=0}^{d-1} (X - \zeta^{hp^t}) \in \mathbb{F}_p[X].$$

于是不可约因子个数为

$$\frac{\varphi(m)}{d} = \frac{N}{d}.$$

$\square$

1.7.2 从模 p 提升到模 p^e

在 $\mathbb{Z}_{p^e}[X]$ 中, $\Phi_m(X) = X^N + 1$ 的分解可由 Hensel 提升得到。

定理 1.11 (模 p^e 下的提升分解). 设

$$X^N + 1 \equiv f_1(X) \cdots f_\ell(X) \pmod{p}$$

是在 $\mathbb{F}_p[X]$ 中分解成两两互素的不可约因子, 则存在唯一的两两互素单首多项式

$$F_1(X), \dots, F_\ell(X) \in \mathbb{Z}_{p^e}[X]$$

满足

$$X^N + 1 = F_1(X) \cdots F_\ell(X) \quad \text{in } \mathbb{Z}_{p^e}[X]$$

且

$$F_i(X) \equiv f_i(X) \pmod{p}.$$

每个 F_i 仍然次数为 d 。

证明思路. 这是 Hensel 引理的标准应用。由于模 p 下的因子两两互素, 所以可唯一提升到模 p^e 下的因子分解。次数在提升过程中不变。 $\square$

1.7.3 因此得到的核心参数关系

我们最终得到:

$$X^N + 1 = F_1(X) \cdots F_\ell(X) \quad \text{in } \mathbb{Z}_{p^e}[X],$$

其中

$$\deg F_i = d, \quad \ell = \frac{N}{d}.$$

直观解释 1.12. 这就是为什么一个 ring element 既能看作长度为 N 的系数向量, 又能看作 ℓ 个 slot, 每个 slot 内部再带 d 维结构:

$$N = \ell \cdot d.$$

1.7.4 power-of-two 情形下因子的稀疏形状

后面讨论 sparse packed 子环时, 需要一个更特殊的结论。对 power-of-two cyclotomic, 有如下现象:

定理 1.13 (power-of-two 情形下的稀疏因子形状). 设 $m = 2N$ 为 2 的幂, $d = \text{ord}_m(p)$, 并设

$$X^N + 1 = F_1(X) \cdots F_\ell(X) \quad \text{in } \mathbb{Z}_{p^e}[X].$$

则每个因子 $F_i(X)$ 具有稀疏形状:

(1) 若 $p \equiv 1 \pmod{4}$, 则

$$F_i(X) = X^d + b_i;$$

(2) 若 $p \equiv 3 \pmod{4}$, 则

$$F_i(X) = X^d + a_i X^{d/2} + b_i.$$

其中 $a_i, b_i \in \mathbb{Z}_{p^e}$ 。

证明思路. 这里只给出足够用于 bootstrapping 的推导思路。

情形一: $p \equiv 1 \pmod{4}$. 令

$$m' = \frac{m}{d}.$$

则

$$N' = \frac{m'}{2} = \frac{m}{2d} = \frac{N}{d} = \ell.$$

此时 $p \equiv 1 \pmod{m'}$, 因此在模 p^e 下, m' -次分圆多项式分解成一次因子。记小问题中的因子为 $F'_i(Y)$, 则

$$F'_i(Y) = Y + b_i.$$

把变量代换

$$Y = X^{N/N'} = X^d,$$

就得到

$$F_i(X) = F'_i(X^d) = X^d + b_i.$$

情形二: $p \equiv 3 \pmod{4}$. 令

$$m' = \frac{2m}{d}, \quad N' = \frac{m'}{2} = \frac{m}{d}.$$

则

$$\frac{N}{N'} = \frac{d}{2}.$$

这时 m' -次分圆多项式在模 p^e 下分解成二次因子

$$F'_i(Y) = Y^2 + a_i Y + b_i.$$

再代换

$$Y = X^{N/N'} = X^{d/2}$$

便得

$$F_i(X) = F'_i(X^{d/2}) = X^d + a_i X^{d/2} + b_i.$$

□

直观解释 1.14. 这个定理非常重要, 因为它说明 $F_i(X)$ 不是任意 d 次多项式, 而是只依赖于 X^d 或 $X^{d/2}$ 的稀疏多项式。这正是 sparse packed 明文会自动落在子环中的原因。

1.8 中国剩余定理与 Slot 分解

1.8.1 CRT 的一般形式

定理 1.15 (中国剩余定理). 设 A 为交换环, $I_1, \dots, I_\ell$ 是两两互素的理想, 则

$$A/(I_1 \cdots I_\ell) \cong A/I_1 \times \cdots \times A/I_\ell.$$

1.8.2 应用到 R_{p^e}

取

$$A = \mathbb{Z}_{p^e}[X], \quad I_i = (F_i(X)).$$

由于 F_i 两两互素, 得到

$$R_{p^e} \cong \mathbb{Z}_{p^e}[X]/(X^N + 1) \cong \prod_{i=1}^{\ell} \mathbb{Z}_{p^e}[X]/(F_i(X)).$$

记

$$E_i := \mathbb{Z}_{p^e}[X]/(F_i(X)).$$

则

$$R_{p^e} \cong E_1 \times \cdots \times E_\ell.$$

1.8.3 Slot 的定义

每个分量 E_i 就称为一个 *slot algebra*; 明文在这 ℓ 个分量上的像, 就是它的 slot 表示。

定义 1.16. 在上述 CRT 分解下, 若

$$a(X) \in R_{p^e}$$

对应到

$$(a_1, \dots, a_\ell) \in E_1 \times \cdots \times E_\ell,$$

则称 a_i 为第 i 个 slot 的值。

1.8.4 标准 slot algebra E

由于所有 E_i 彼此同构, 常固定一份标准拷贝 E , 并写

$$R_{p^e} \cong E^\ell.$$

这就是后面常见的 slot 向量视角。

直观解释 1.17. CRT 不是说“一个多项式真的变成了 ℓ 个独立东西”, 而是说: 这个大环与 ℓ 个小环的直积完全等价。因此在数学上, 你可以自由地在这两种表示之间来回切换。

1.9 Abel 群直积分解与 Slot 编号集 S

1.9.1 为什么 slot 要编号

CRT 只告诉我们有 ℓ 个分量, 但没有告诉我们如何对这些分量“按几何方式排列”。后面要定义 rotation、one-dimensional linear transform、BSGS 等算法, 就必须给 slot 一个结构化编号。

1.9.2 商群 $\mathbb{Z}_m^\times / \langle p \rangle$ 的来源

前面说过, Φ_m 的根按 Frobenius 轨道分组; 每个轨道对应一个因子, 也对应一个 slot。于是不同 slot 的标签应当是

$$\mathbb{Z}_m^\times$$

按

$$h \sim hp^t$$

的等价关系分组后的结果, 即商群

$$\mathbb{Z}_m^\times / \langle p \rangle.$$

定义 1.18. 取 $\mathbb{Z}_m^\times / \langle p \rangle$ 的一组代表元, 记为

$$S.$$

则 S 可用来给 slots 编号: 每个 $h \in S$ 对应一个 slot。

1.9.3 有限 Abel 群分解定理

定理 1.19 (有限 Abel 群结构定理). 任意有限 Abel 群 G 同构于若干循环群的直积:

$$G \cong C_{n_1} \times \cdots \times C_{n_t}, \quad n_1 \mid n_2 \mid \cdots \mid n_t.$$

本报告不证明这一经典定理, 但会使用它的结论。

1.9.4 将商群坐标化

由结构定理, 可将

$$\mathbb{Z}_m^\times / \langle p \rangle$$

写成若干循环群的直积。于是可选取生成元 $g_1, \dots, g_t$, 以及相应阶数 $\ell_1, \dots, \ell_t$, 使得

$$|S| = \ell = \prod_{i=1}^t \ell_i$$

并且每个 slot 唯一写成

$$h = g_1^{e_1} \cdots g_t^{e_t}, \quad 0 \leq e_i < \ell_i.$$

直观解释 1.20. 这就是 slot 的“多维坐标系”。不是说 slot 真的是几何空间中的点，而是说它们的索引集本身是一个有限 Abel 群，因此可以写成若干循环方向的直积。

1.9.5 为什么这套编号特别适合 rotation

若只改变某个 e_i ，就相当于只在第 i 个循环方向上移动一步。因此

$$(e_1, \dots, e_i, \dots, e_t) \mapsto (e_1, \dots, e_i + v, \dots, e_t)$$

就变成了 one-dimensional rotation 的自然定义。

1.10 power-of-two 情况下 Automorphism 群的具体结构

1.10.1 $\mathbb{Z}_{2^r}^\times$ 的分解

令 $m = 2^r$ 且 $r \geq 3$ 。则有经典结论：

定理 1.21.

$$\mathbb{Z}_{2^r}^\times \cong \langle -1 \rangle \times \langle 5 \rangle,$$

其中

$$\text{ord}_{2^r}(-1) = 2, \quad \text{ord}_{2^r}(5) = 2^{r-2}.$$

证明思路. (-1) 的阶显然为 2。对 5，可用 lifting-the-exponent 或二项式展开证明

$$5^{2^{r-2}} \equiv 1 \pmod{2^r},$$

且更小的 2 次幂不满足该同余。再由

$$|\mathbb{Z}_{2^r}^\times| = 2^{r-1}$$

即可得出分解。 □

1.10.2 $p \equiv 1 \pmod{4}$ 与 $p \equiv 3 \pmod{4}$

由于

$$\mathbb{Z}_m^\times = \langle 5 \rangle \times \langle -1 \rangle,$$

所以 p 在其中的位置只会落在两类：

(1) $p \equiv 1 \pmod{4}$ ：此时 $p \in \langle 5 \rangle$ ；

(2) $p \equiv 3 \pmod{4}$ ：此时 $p \in -\langle 5 \rangle$ 。

这将直接影响商群

$$\mathbb{Z}_m^\times / \langle p \rangle$$

是一维还是二维。

命题 1.22. 在 *power-of-two cyclotomic* 情形中：

(1) 若 $p \equiv 1 \pmod{4}$ ，则 *slot* 索引群天然带有二维结构，常取

$$g_1 = 5, \quad g_2 = -1;$$

(2) 若 $p \equiv 3 \pmod{4}$ ，则 *slot* 索引群天然是一维循环结构，常只需

$$g_1 = 5.$$

直观解释 1.23. 这就是 later literature 中所谓 rank-one / rank-two rotation group 的来源。它不是算法拍脑袋设计出来的，而是底层商群结构决定的。

1.11 Slot 几何与一维旋转

1.11.1 多维 slot 坐标

上一节给出的坐标化

$$h = g_1^{e_1} \cdots g_t^{e_t}$$

使得每个 slot 可唯一写成一个 t -维坐标

$$(e_1, \dots, e_t).$$

1.11.2 one-dimensional rotation 的定义

定义 1.24. 固定某个维度 i ，以及步长 v 。定义第 i 维旋转

$$\rho_i^v : (e_1, \dots, e_i, \dots, e_t) \mapsto (e_1, \dots, e_i + v \pmod{\ell_i}, \dots, e_t).$$

1.11.3 good dimension 与 bad dimension 的预告

后面在具体实现时，并不是每个一维旋转都能由一个 automorphism 直接完成。是否能“只用一个 automorphism”实现，取决于该维度在原群中是否真回到单位元。这会导致 good dimension / bad dimension 的区分。

1.12 自同构 $a(X) \mapsto a(X^j)$ 的代数意义

1.12.1 为什么 $j \in \mathbb{Z}_m^\times$ 对应一个自同构

在分圆视角下， X 的角色可理解为一个抽象的 m 次本原单位根 ζ_m 。若 $j \in \mathbb{Z}_m^\times$ ，则 ζ_m^j 仍然是本原 m 次单位根，因此定义了自同构

$$\tau_j : \zeta_m \mapsto \zeta_m^j.$$

在多项式表示里，这就写成

$$\tau_j : a(X) \mapsto a(X^j).$$

定理 1.25. 映射

$$\tau_j : a(X) \mapsto a(X^j)$$

给出 $R/\mathbb{Z}$ 的一个环自同构，并且

$$\text{Aut}(R/\mathbb{Z}) \cong \mathbb{Z}_m^\times.$$

证明思路. 在分圆表示 $R \cong \mathbb{Z}[\zeta_m]$ 下，任何自同构由 ζ_m 的像唯一确定；而 ζ_m 必须被送到另一个本原 m 次单位根 ζ_m^j ，其中 $j \in \mathbb{Z}_m^\times$ 。这样得到群同构。□

1.12.2 系数视角下的含义

在系数表示里， $a(X) \mapsto a(X^j)$ 表示：把多项式中每个 X 都换成 X^j ，再模 $X^N + 1$ 化简。因此它表现为“系数位置的重排与可能的符号变化”。

1.12.3 slot 视角下的含义

在 slot 视角里，若 slot 标签是 $h \in S$ ，则

$$\tau_j(a)(\zeta_m^h) = a(\zeta_m^{hj}).$$

所以 τ_j 在 slot 上就是

$$h \mapsto hj$$

的置换。

直观解释 1.26. 这说明 rotation 和 automorphism 的根本联系是：slot 的索引集本身是一个群或商群，automorphism 恰好在这个群上做乘法作用。

1.13 稀疏子环与换元 $Y = X^c$

1.13.1 为什么 sparse packed 会落到子环

由前面的稀疏因子形状定理, power-of-two 情形下每个因子满足

$$F_i(X) \in \mathbb{Z}_{p^e}[X^c], \quad c = \begin{cases} d, & p \equiv 1 \pmod{4}, \\ d/2, & p \equiv 3 \pmod{4}. \end{cases}$$

而 CRT 基底也是由这些因子构造出来的, 因此对应的 sparse packed 明文天然只依赖于 X^c 。

命题 1.27. 若一个明文 $m(X) \in R_{p^e}$ 只含幂次

$$1, X^c, X^{2c}, \dots, X^{(N'-1)c}, \quad N' = \frac{N}{c},$$

则这些元素构成一个子环, 且该子环同构于

$$R'_{p^e} := \mathbb{Z}_{p^e}[Y]/(Y^{N'} + 1), \quad Y = X^c.$$

证明. 设

$$Y = X^c.$$

则任何只含 X^{ct} 的多项式都可唯一写成

$$q(Y) = a_0 + a_1 Y + \dots + a_{N'-1} Y^{N'-1}.$$

又因为

$$X^N + 1 = 0, \quad N = cN',$$

所以

$$Y^{N'} + 1 = (X^c)^{N'} + 1 = X^N + 1 = 0.$$

因此 Y 满足关系 $Y^{N'} + 1 = 0$, 故该集合与

$$\mathbb{Z}_{p^e}[Y]/(Y^{N'} + 1)$$

同构。 □

1.13.2 为什么会出现 $q(Y)$

这一步只是换元:

$$m(X) = a_0 + a_1 X^c + \dots + a_{\ell-1} X^{c(\ell-1)} \iff q(Y) = a_0 + a_1 Y + \dots + a_{\ell-1} Y^{\ell-1},$$

其中 $Y = X^c$ 。

这里最高次数是 $\ell - 1$ ，是因为 sparse 子环的维数正好是

$$N' = \frac{N}{c} = \ell$$

(在最典型的 sparse packed 情形中确实如此)，所以标准基是

$$1, Y, Y^2, \dots, Y^{\ell-1}.$$

直观解释 1.28. $q(Y)$ 不是一个新的数学对象，只是把“稀疏的 X -多项式”改写成“普通的 Y -多项式”，这样后面的 CRT/插值矩阵会看起来非常直观。

1.14 基底问题：局部幂基、统一幂基与系数基

1.14.1 整个明文环的系数基

在

$$R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1)$$

中，自然的系数基是

$$1, X, X^2, \dots, X^{N-1}.$$

这是最终“多项式表示”使用的基底。

1.14.2 第 i 个 slot 的局部幂基

在第 i 个 slot algebra

$$E_i = \mathbb{Z}_{p^e}[T]/(F_i(T))$$

中，记

$$\zeta_{m,i} := T \bmod F_i(T).$$

则局部幂基是

$$1, \zeta_{m,i}, \zeta_{m,i}^2, \dots, \zeta_{m,i}^{d-1}.$$

因此 fully packed 时，第 i 个 slot 的一般元素写成

$$\sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j.$$

1.14.3 统一参考 slot algebra 的幂基

固定一份标准拷贝

$$E = \mathbb{Z}_{p^e}[Y]/(F_1(Y)), \quad \zeta_m := Y \bmod F_1(Y).$$

则统一幂基为

$$1, \zeta_m, \zeta_m^2, \dots, \zeta_m^{d-1}.$$

1.14.4 为什么三套基不能混

- (1) $1, X, \dots, X^{N-1}$ 是整个大环的系数基;
- (2) $1, \zeta_{m,i}, \dots, \zeta_{m,i}^{d-1}$ 是第 i 个 slot 的局部基;
- (3) $1, \zeta_m, \dots, \zeta_m^{d-1}$ 是标准 slot algebra 的统一基。

它们处在不同层次的对象上，不能不加区分地混用。

1.14.5 M 与 M^{-1} 的本质

在 fully packed 的 CoeffToSlot / SlotToCoeff 中， M 的作用是把

$$\sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j$$

统一改写成

$$\sum_{j=0}^{d-1} m_{i,j} \zeta_m^j.$$

也就是把各个 slot 的局部坐标系统一成标准坐标系； M^{-1} 则做反向变换。

直观解释 1.29. 在 fully packed 情形下， U 只处理“slot 与 slot 之间”的外层混合；而每个 slot 里面那组 $\zeta_{m,i}^j$ 与 ζ_m^j 的差异，必须由 M 和 M^{-1} 解决。

1.15 从 CRT 到插值：为什么会出现 Vandermonde/FFT-like 矩阵

1.15.1 CRT 与“在若干点上求值”的等价

在最普通的 sparse 子环情形里，一个元素写成

$$q(Y) = a_0 + a_1 Y + \dots + a_{\ell-1} Y^{\ell-1}.$$

而其 slot/CRT 表示就是在若干点

$$y_i := \zeta_{m,i}^c$$

上的取值：

$$q(y_i) = a_0 + a_1 y_i + \dots + a_{\ell-1} y_i^{\ell-1}.$$

1.15.2 因此出现 Vandermonde 矩阵

将上式对所有 i 排成矩阵，得到

$$\begin{bmatrix} q(y_0) \\ q(y_1) \\ \vdots \\ q(y_{\ell-1}) \end{bmatrix} = \begin{bmatrix} 1 & y_0 & y_0^2 & \cdots & y_0^{\ell-1} \\ 1 & y_1 & y_1^2 & \cdots & y_1^{\ell-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & y_{\ell-1} & y_{\ell-1}^2 & \cdots & y_{\ell-1}^{\ell-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{\ell-1} \end{bmatrix}.$$

这就是典型的 Vandermonde / evaluation matrix。

若把 $y_i = \zeta_{m,i}^c$ 代入，则矩阵条目为

$$U_{ik} = y_i^k = \zeta_{m,i}^{ck}.$$

当 $c = d$ 时，便得到熟悉的

$$U_{ik} = \zeta_{m,i}^{dk}.$$

1.15.3 为什么它又像 FFT

因为这些 y_i 是某组 roots of unity 的幂，所以该矩阵兼具：

- Vandermonde 矩阵的“插值/求值”性质；
- FFT 矩阵的“根的幂”结构与稀疏递归分解性质。

1.15.4 sparse 与 fully packed 的区别

- sparse packed: U 本身就可以理解为子环上的 CRT/Vandermonde 矩阵；
- fully packed: U 只处理外层 slot 索引，完整变换要看 $M^{-1}UM$ 。

1.16 参数关系总表与推导链

1.16.1 核心参数

参数	含义
m	分圆指数；power-of-two 情形下 $m = 2N$
N	大环 R_{p^e} 的系数维数； $\Phi_m(X) = X^N + 1$
p^e	明文模数
Q	密文模数

d	$d = \text{ord}_m(p)$, Frobenius 轨道长度, 也是不可约因子次数
ℓ	slot 数, $\ell = N/d$
E_i	第 i 个 slot algebra, $E_i = \mathbb{Z}_{p^e}[X]/(F_i(X))$
E	标准 slot algebra, 一般作为各 E_i 的统一拷贝
S	$\mathbb{Z}_m^\times/\langle p \rangle$ 的代表集, 用来给 slot 编号
g_i, ℓ_i	slot 索引群分解的生成元与各维长度
c	sparse 子环的步长, 通常 $c = d$ 或 $c = d/2$
m'	子环对应的更小分圆指数, $m' = m/c$
N'	子环维数, $N' = N/c$
R'_{p^e}	sparse packed 自然落入的子环

1.16.2 最重要的推导链

下面这条链建议在汇报时反复强调:

$$m = 2N \implies \Phi_m(X) = X^N + 1$$

$$d = \text{ord}_m(p) \implies \text{Frobenius 轨道长度} = d$$

$$\text{轨道长度} = d \implies \deg F_i = d, \quad \ell = \frac{N}{d}$$

$$R_{p^e} \cong \prod_{i=1}^{\ell} E_i \cong E^{\ell}$$

$$S \subset \mathbb{Z}_m^\times/\langle p \rangle \implies \text{slot 可以按群坐标编号}$$

$$F_i(X) \in \mathbb{Z}_{p^e}[X^c] \implies \text{sparse packed 明文落在 } R'_{p^e}$$

$$Y = X^c \implies \text{稀疏子环上的 CRT 退化成普通 Vandermonde/FFT 型插值问题.}$$

1.17 本章小结: 后续 Bootstrapping 需要哪些代数工具

到这里, 我们已经具备了进入 BGV/BFV bootstrapping 主流程所需的最关键代数准备。总结如下:

(1) 大环与小环:

$$R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1)$$

是大明文环, 经 CRT 可分解为若干个 slot algebra 的直积; sparse packed 时, 还会自然落入更小子环

$$R'_{p^e}.$$

(2) **核心参数**:

$$d = \text{ord}_m(p), \quad \ell = \frac{N}{d}$$

决定了 slot 的内部宽度与数量。

(3) **slot 的来源**: slot 不是多项式系数, 而是 CRT 分解得到的分量; 其索引集来自商群

$$\mathbb{Z}_m^\times / \langle p \rangle.$$

(4) **slot 的几何**: 通过有限 Abel 群分解, slot 可写成多维循环坐标, 从而自然定义 rotation。

(5) **automorphism 的作用**:

$$a(X) \mapsto a(X^j)$$

是分圆结构上的环自同构; 在 slot 视角下, 它就是 slot permutation 或 slot 内部 Frobenius。

(6) **三套基底**: 系数基、局部幂基、统一幂基分别对应不同层次的表示; fully packed 里必须用 M 与 M^{-1} 处理它们之间的转换。

(7) **sparse packed 的本质**: 它把问题压缩到一个只依赖 X^c 的子环, 因此可用

$$Y = X^c$$

把问题改写成普通多项式 $q(Y)$ 的插值/求值问题; 这正是 Vandermonde/FFT 型矩阵 U 出现的来源。

有了这一章作为基础, 下一部分再进入 BGV/BFV bootstrapping 的完整流程——同态线性解密、CoeffToSlot、unpack、digit extraction、repack、SlotToCoeff——就会清晰得多。

第二章

BGV/BFV 的基础代数框架

2.1 本章目标

上一章已经补齐了 bootstrapping 所需的代数与数论背景：分圆环、Frobenius 轨道、CRT、slot 几何、子环 R'_{p^e} 、以及局部幂基与统一幂基的区分。本章开始回到密码系统本身，目标是回答以下问题：

- (1) BGV/BFV 的明文环、密文环、密钥究竟是什么对象；
- (2) 密文的“本体”为什么始终是多项式环元素，而不是向量；
- (3) BGV 与 BFV 的解密公式分别长什么样；
- (4) 噪声在解密关系中处于什么位置，为什么会不断增长；
- (5) 为什么 BGV 的 bootstrapping 核心更像“模 p^e 约简”，而 BFV 的核心更像“缩放后取整”；
- (6) 为什么 bootstrapping 可以理解为“同态计算解密函数”。

本章不讨论具体的 CoeffToSlot / SlotToCoeff、unpack、digit extraction 细节；这些会在后续章节展开。本章只建立系统级视角：我们到底在对什么对象做运算，这些对象满足什么解密关系，*bootstrapping* 试图修复的到底是什么。

2.2 明文环与密文环

2.2.1 分圆环作为底层工作空间

在 power-of-two cyclotomic 情形中，固定

$$m = 2N, \quad \Phi_m(X) = X^N + 1.$$

于是我们考虑大环

$$R := \mathbb{Z}[X]/(X^N + 1).$$

其模 q 版本记为

$$R_q := \mathbb{Z}_q[X]/(X^N + 1).$$

定义 2.1. 在 BGV/BFV 中，典型地使用以下两个环：

$$R_{p^e} := \mathbb{Z}_{p^e}[X]/(X^N + 1),$$

$$R_Q := \mathbb{Z}_Q[X]/(X^N + 1),$$

其中 p^e 是明文模数， Q 是较大的密文模数。

注 2.2. 这里必须区分三层对象：

- (1) $\mathbb{Z}_{p^e}$ ：明文系数所在的底层环；
- (2) R_{p^e} ：明文多项式所在的分圆环；
- (3) R_Q ：密文运算所在的“大模数”分圆环。

后续的 bootstrapping，始终是在 R_Q 或更大的提升模数环里计算，但它服务于 R_{p^e} 上消息的恢复。

2.2.2 明文空间的两种视角

上一章已经说明，明文环 R_{p^e} 有两种同等重要的视角：

- (1) **系数视角**：元素写成

$$a(X) = a_0 + a_1X + \cdots + a_{N-1}X^{N-1}, \quad a_i \in \mathbb{Z}_{p^e};$$

- (2) **slot 视角**：经 CRT 分解后，

$$R_{p^e} \cong E^\ell,$$

可看成 ℓ 个 slots 的直积。

直观解释 2.3. 这两种视角都只是同一个明文环元素的不同坐标系。BGV/BFV 密文从头到尾都加密的是这个环元素；所谓 CoeffToSlot / SlotToCoeff，只是在切换“如何解释它”的坐标系，而不是把密文本体换成另一种对象。

2.2.3 子环 R'_{p^e} 的位置

为了后面讨论 sparse packed bootstrapping，我们再次强调：若明文只依赖 X^c ，则会自然落在子环

$$R'_{p^e} \cong \mathbb{Z}_{p^e}[Y]/(Y^{N'} + 1), \quad Y = X^c, \quad N' = \frac{N}{c}.$$

这个子环稍后会用于 thin/sparse bootstrap，但在本章里只把它看成一种更小的明文空间。

2.3 BGV/BFV 的密文结构

2.3.1 RLWE 视角

BGV/BFV 都属于基于环学习带误差 (RLWE) 的方案。最基本的二元密文形状是

$$\mathbf{c} = (c_0, c_1) \in R_Q^2.$$

在乘法、重线性化之前，也可能暂时出现更多分量，但核心分析总能回到“密文是若干个 R_Q 元素组成的短向量”。

注 2.4. 这也是为什么前面一直强调：密文本体始终是环元素对（或小元组），不是向量 *slots*。slot 只是明文语义上的解释视角。

2.3.2 秘密钥与公钥的角色

记秘密钥为

$$s \in R_Q.$$

在最简单的形式下，解密会用到

$$c_0 + c_1 s.$$

若密文分量更多，则解密通常会涉及

$$\sum_i c_i s^i.$$

因此 bootstrapping 中“同态解密线性部分”这一步，通常就是要同态计算这种带有 s 或 s^i 的环表达式。

2.3.3 为什么噪声是不可避免的

RLWE 方案的安全性依赖于误差项，因此密文在构造时必然含有噪声。无论是加法还是乘法，同态运算都会让噪声增长。

如果噪声增长到一定程度，解密公式中“真正的消息”与“噪声部分”将无法再被区分，于是密文失效。

定义 2.5. 称一个密文 $\mathbf{c}$ 为可正确解密，若其解密表达式仍落在“消息可恢复”的范围内。具体“范围”随 BGV 与 BFV 不同而不同。

直观解释 2.6. bootstrapping 要修复的不是“消息错了”，而是“消息虽然还在，但快被噪声淹没了”。因此 bootstrapping 的目标从来不是改变明文，而是刷新密文的可解密性。

2.4 BGV 的解密关系

2.4.1 BGV 的基本形状

在 BGV 中，明文模数是 p^e ，而噪声通常是这个明文模数的倍数。最核心的解密关系可以抽象写成

$$u := c_0 + c_1 s \equiv m + p^e r \pmod{Q},$$

其中

- $m \in R_{p^e}$ 是真正明文；
- $r \in R_Q$ 聚合了噪声与若干中间误差；
- $u \in R_Q$ 是解密线性部分得到的 noisy plaintext。

更一般地，若有多个密文分量，则写作

$$u := \sum_i c_i s^i \equiv m + p^e r \pmod{Q}.$$

注 2.7. 真正实现里，公式会伴随模数切换、重线性化、key switching 等技术细节，但从 bootstrapping 的角度，最重要的是这条结构性关系：

$$u = m + p^e r.$$

2.4.2 为什么这条公式足以说明 BGV 的核心任务

由

$$u = m + p^e r$$

可立即得到

$$u \bmod p^e = m.$$

因此，若我们能同态地从 $\text{Enc}(u)$ 恢复 $\text{Enc}(u \bmod p^e)$ ，就得到了 $\text{Enc}(m)$ 。

直观解释 2.8. 所以 BGV bootstrapping 的本质就是：

把“加密的 $m + p^e r$ ”变成“加密的 m ”。

这也是为什么后面 digit extraction 会围绕“低位保留、高位丢弃”来设计。

2.4.3 BGV 中噪声为什么是“ p^e 的倍数”特别有利

因为它给了我们一个非常明确的非线性目标：做模 p^e 约简。一旦把 noisy plaintext 展开到适合的表示（通常是 slots），后续任务就变成逐槽地从

$$u_i = m_i + p^e r_i$$

恢复

$$m_i.$$

这比“直接在整个多项式环里做复杂函数”更容易并行化。

2.5 BFV 的解密关系

2.5.1 BFV 的基本形状

与 BGV 不同，BFV 中通常先把明文 $m \in R_t$ 通过一个放大系数嵌入到大模数空间。设

$$t = p^e, \quad \Delta \approx \frac{Q}{t}.$$

则 BFV 的解密关系抽象上可写成

$$u := c_0 + c_1 s \equiv \Delta m + e \pmod{Q},$$

其中 e 是较小噪声。

2.5.2 解密步骤的性质

从上式恢复 m 的方法不是直接模 t ，而是先缩放再取整：

$$m \approx \left\lfloor \frac{t}{Q} u \right\rfloor \pmod{t}.$$

因此 BFV bootstrapping 的核心非线性更像：

把“加密的 $\Delta m + e$ ”变成“加密的 m ”。

2.5.3 与 BGV 的核心区别

BGV 的 noisy plaintext 长成

$$m + p^e r,$$

所以目标是“去掉 p^e 的倍数垃圾”。

BFV 的 noisy plaintext 长成

$$\Delta m + e,$$

所以目标是“消除缩放 Δ 并抹掉剩余误差”。

直观解释 2.9. 如果用最直白的话说：

- BGV 更像“精确地保留低位 residue”；
- BFV 更像“把一个被放大的消息缩回原范围，并正确取整”。

所以两者虽然都需要 bootstrapping，但中间的核心非线性不一样。

2.6 BGV 与 BFV：为什么都属于“同态解密”

2.6.1 同态解密的抽象模板

设旧密文为 $\mathbf{c}$ ，秘密钥为 s 。普通解密是某个函数

$$\text{Dec}_s(\mathbf{c}) = m.$$

bootstrapping 则是在密文域内同态评估这个函数，得到

$$\text{Enc}(m).$$

更抽象地写：

$$\text{Bootstrap}(\mathbf{c}) = \text{Eval}(\text{Dec}_s, \mathbf{c}).$$

2.6.2 为什么不是“先解密再加密”

如果真的先解密再加密，就必须知道秘密钥并进入明文域，这在 FHE 服务端环境下是不允许的。bootstrapping 的关键是：服务端只拿到 *bootstrapping key*，而不是明文 *secret key*，所以它只能在密文域里模拟解密电路。

2.6.3 同态解密的两部分

无论 BGV 还是 BFV，都可把同态解密分成两块：

- (1) **线性部分**：同态求出 noisy plaintext，例如

$$u = c_0 + c_1 s \quad \text{或} \quad u = \sum_i c_i s^i;$$

- (2) **非线性清理部分**：从 u 中恢复消息 m 。

注 2.10. 在现代 packed bootstrapping 里，CoeffToSlot / SlotToCoeff 往往插在第二块周围，帮助把“非线性清理”变成逐槽处理的问题。

2.7 噪声增长与 Leveled FHE

2.7.1 为什么 Leveled FHE 不够

若只使用有限层级的模数链，则系统只能支持预先给定深度的电路。对于无限深度或高层数应用，就必须使用 bootstrapping。

2.7.2 噪声增长的来源

噪声增长主要来自：

- (1) 加法带来的线性累积；
- (2) 乘法带来的乘性放大；
- (3) relinearization / key switching 的附加误差；
- (4) 复杂线性变换（如 automorphism 与 rotation）的额外代价。

2.7.3 为什么“刷新噪声”不是免费操作

bootstrapping 虽然能重置密文的有效噪声预算，但它本身需要评估一条很深的电路，因此代价极高。这就是为什么后续章节会重点优化：

- CoeffToSlot / SlotToCoeff；
- unpack / repack；
- digit extraction；
- rotation 的数量与线性变换分解。

2.8 Bootstrapping 前的表示问题

2.8.1 同一个明文有两套常用表示

在进入真正 bootstrapping 之前，必须时刻记住同一个明文有两套表示：

- (1) **coefficient** 表示：

$$a(X) = a_0 + a_1X + \cdots + a_{N-1}X^{N-1};$$

- (2) **slot** 表示：

$$(a_1, \dots, a_\ell) \in E^\ell.$$

2.8.2 为什么 bootstrapping 不会改变密文本体

无论你说一个密文“当前在 coefficient view 下”还是“当前在 slot view 下”，底层密文始终是 R_Q 中的环元素对。变化的只是被加密明文的解释坐标系。

直观解释 2.11. 你可以把密文看成一张纸上的同一个向量，只是你有时候用标准基读它，有时候用特征向量基读它。坐标不同，不代表对象变了。

2.9 BGV/BFV 与 CKKS 的 bootstrapping 差别

2.9.1 三者都包含 CoeffToSlot / SlotToCoeff

在 packed bootstrapping 中，三者都可能出现：

- CoeffToSlot;
- SlotToCoeff;
- automorphism / rotation;
- FFT-like 线性变换。

2.9.2 但核心非线性不同

CKKS. 中心步骤通常是近似的 EvalMod，即对实/复槽值做一个近似函数。

BGV. 中心步骤更像：

$$u_i = m_i + p^e r_i \mapsto m_i.$$

因此核心是 digit extraction / residue reduction。

BFV. 中心步骤更像：

$$u_i = \Delta m_i + e_i \mapsto m_i.$$

因此核心是缩放、取整、再模 t 。

2.9.3 为什么这会影晌后续优化

因为不同的核心非线性决定了：

- 需要把数据打开到什么层次；
- 逐槽操作到底是“精确整数函数”还是“近似解析函数”；
- unpack / repack 的必要性与形式；
- 最终复杂度瓶颈在哪里。

2.10 本章小结

本章完成了 BGV/BFV 密码系统层面的准备，可以概括为以下几点：

- (1) **明文与密文**：明文活在

$$R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1),$$

密文活在 R_Q^2 （或其短元组）中。

- (2) **密文本体始终是环元素**：slot 只是明文语义上的解释坐标，不是密文对象类型。

- (3) **BGV 的 noisy plaintext**：

$$u = m + p^e r,$$

所以核心非线性是“模 p^e 约简”。

- (4) **BFV 的 noisy plaintext**：

$$u = \Delta m + e,$$

所以核心非线性是“缩放并取整”。

- (5) **bootstrapping 的本质**：同态计算解密函数，从旧密文中得到同一消息的新密文。

- (6) **后续章节的逻辑起点**：真正困难的部分不是“线性解密”本身，而是从 noisy plaintext 到真消息的恢复；这正是 CoeffToSlot、unpack、digit extraction、repack、SlotToCoeff 登场的地方。

下一章将正式进入 BGV/BFV 的完整 bootstrapping 流程，重点解释：

- 为什么要先做 CoeffToSlot；
- fully packed 为什么必须 unpack；
- digit extraction 为什么可以逐槽做；
- 以及如何再通过 SlotToCoeff 回到新的 ring ciphertext。

明文空间、CRT 与 SIMD 打包

3.1 什么是 batching / SIMD

3.1.1 为什么一个密文里想装多个消息

在普通的公钥加密中，我们通常把一个明文消息 m 加密成一个密文 c 。如果要同时处理很多消息，就需要很多个密文。而在同态加密中，这样做会非常昂贵，因为：

- 每个密文都带有多项式环上的开销；
- 每次加法、乘法、旋转、key switching 都要对整个密文对象操作；
- 若一个密文只装一个标量，吞吐会很低。

因此人们希望：

在一个密文里同时打包很多个消息，并让它们尽可能并行地被处理。

这就是 batching，也常写作 SIMD (Single Instruction, Multiple Data) 的思想。

3.1.2 SIMD 的直观含义

所谓 SIMD，在这里不是硬件术语，而是一个非常直观的抽象：

同一个同态操作，一次作用到很多个槽 (slot) 上。

例如，若一个明文在 slot 视角下写成

$$(m_0, m_1, \dots, m_{\ell-1}),$$

那么一次同态加法就对应于

$$(m_0, \dots, m_{\ell-1}) + (n_0, \dots, n_{\ell-1}) = (m_0 + n_0, \dots, m_{\ell-1} + n_{\ell-1}),$$

一次同态乘法则对应于

$$(m_0, \dots, m_{\ell-1}) \cdot (n_0, \dots, n_{\ell-1}) = (m_0 n_0, \dots, m_{\ell-1} n_{\ell-1}),$$

至少在最基础的 slot 语义上是如此。

直观解释 3.1. SIMD 的核心不是“数据真的变成了一个普通向量”，而是：同一个环元素经过适当分解之后，可以被解释成很多个相互独立或半独立的分量；环上的运算在这些分量上表现为并行运算。

3.1.3 为什么 BGV/BFV 天然适合 batching

BGV/BFV 的明文空间是

$$R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1).$$

这个环并不是不可再分的。一旦 $X^N + 1$ 在 $\mathbb{Z}_{p^e}[X]$ 中分解成若干个互素因子

$$X^N + 1 = F_1(X) \cdots F_\ell(X),$$

中国剩余定理就会告诉我们：

$$R_{p^e} \cong \mathbb{Z}_{p^e}[X]/(F_1) \times \cdots \times \mathbb{Z}_{p^e}[X]/(F_\ell).$$

于是一个明文环元素，可以同时被看成 ℓ 个较小分量的组合，这就是 batching 的数学来源。

3.1.4 batching 与 bootstrapping 的关系

在 bootstrapping 中，我们通常不是只关心“能不能恢复一个消息”，而是关心：

- 能否在一个密文里同时恢复很多个 slot；
- 能否在 slot 表示下逐槽做 digit extraction / residue reduction；
- 能否在 fully packed 情况下保持满吞吐刷新。

因此，理解 batching / SIMD，不只是理解“怎么多装数据”，更是在理解：

为什么 bootstrapping 中会出现 CoeffToSlot、unpack、repack、SlotToCoeff 这些看起来像“表示变换”

3.2 CRT 分解的基本思想

3.2.1 明文环如何分裂成多个 slot

我们从上一章的结论出发。设

$$m = 2N, \quad d = \text{ord}_m(p), \quad \ell = \frac{N}{d}.$$

在 $\mathbb{Z}_{p^e}[X]$ 中,

$$X^N + 1 = F_1(X) \cdots F_\ell(X),$$

其中每个 $F_i(X)$ 都是次数为 d 的互素多项式。

于是由中国剩余定理,

$$R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1) \cong \prod_{i=1}^{\ell} \mathbb{Z}_{p^e}[X]/(F_i(X)).$$

定义 3.2. 记

$$E_i := \mathbb{Z}_{p^e}[X]/(F_i(X)).$$

则称 E_i 为第 i 个 *slot algebra*, 并把上面的直积分解理解为把明文环分裂成 ℓ 个 slots。

因此, 一个明文环元素 $a(X) \in R_{p^e}$ 可以等价地写成

$$a(X) \longleftrightarrow (a_1, \dots, a_\ell), \quad a_i \in E_i.$$

3.2.2 为什么这就叫“分裂成多个 slot”

“slot”这个词不是一个额外定义出来的密码学黑箱, 而是 CRT 分量的应用性叫法。

当你写

$$a(X) \longleftrightarrow (a_1, \dots, a_\ell),$$

其中每个 a_i 都可独立地参与加法、乘法、Frobenius、自同构等操作时, 最自然的想法就是把它们视为:

同一个大明文里的很多个槽。

这和复数 FFT 中“频率槽”或者向量 SIMD 中“lane”很像, 只不过这里的槽不是人工切出来的, 而是由 CRT 自然给出的。

3.2.3 用根来理解 slot 分裂

上一章中我们已经提到, 若 ζ_m 是某个 m 次本原单位根, 则 R_{p^e} 的 CRT 映射可写成

$$a(X) \mapsto \{a(\zeta_m^h)\}_{h \in S},$$

其中 S 是 $\mathbb{Z}_m^\times / \langle p \rangle$ 的一组代表元。

因此, 一个 slot 也可以理解成:

“在某一个 Frobenius 轨道代表根上观察多项式的值”

所以“明文环如何分裂成多个 slot”有两种同义说法:

- (1) 代数上: 通过 CRT 分解成多个商环 E_i ;
- (2) 根值上: 通过在若干个代表根 ζ_m^h 上求值来获得多个分量。

3.2.4 slot 数为什么等于 $\ell = N/d$

这里再把最核心的参数关系重述一遍。

- $X^N + 1 = \Phi_m(X)$ 的总次数是 N ;
- 模 p^e 下每个不可约因子 F_i 的次数是

$$d = \text{ord}_m(p);$$

- 因此因子个数，也就是 slot 个数，是

$$\ell = \frac{N}{d}.$$

这说明：

slot 数不是随便选择的，而是由分圆结构和 Frobenius 轨道长度共同决定的。

3.2.5 slot algebra 的含义

从定义上看，

$$E_i = \mathbb{Z}_{p^e}[X]/(F_i(X))$$

只是一个小商环。但在 bootstrapping 的语境里，它有三层含义。

3.2.5.1 第一层：它是 CRT 分量

这是最直接的含义。明文在 R_{p^e} 中的一个元素，被 CRT 分解后，第 i 个分量就落在 E_i 中。

3.2.5.2 第二层：它描述了“一个 slot 内部还装了多少东西”

由于 $\deg F_i = d$ ，所以 E_i 作为 $\mathbb{Z}_{p^e}$ -模具有维数 d 。也就是说，一个一般的 slot 值并不是一个单独的标量，而是可以写成

$$a_{i,0} + a_{i,1}\zeta_{m,i} + a_{i,2}\zeta_{m,i}^2 + \cdots + a_{i,d-1}\zeta_{m,i}^{d-1}.$$

因此单个 slot 里本身就可以装一个 d -维对象。

这就是 fully packed 与 sparse packed 的分水岭。

3.2.5.3 第三层：它是一个“局部明文空间”

当我们说“某个 slot 上做 Frobenius”“某个 slot 上做 trace”“某个 slot 上做 digit extraction 之前要 unpack”，其实都是在讨论这个小环 E_i 里的运算与结构，而不是整个大环 R_{p^e} 的运算。

直观解释 3.3. 把 R_{p^e} 想成一个“大房子”，CRT 把它分成 ℓ 个“房间”，每个房间就是一个 slot algebra E_i 。sparse packed 是每个房间里只放一个标量；fully packed 是每个房间里放一整个 d -维小向量。

3.2.6 “slot 不是系数”的正确理解

这是最重要、最容易混淆的一句话。

3.2.6.1 错误理解

很多人第一次接触 batching 时会误以为：

“slot 就是多项式的系数位置。”

比如把

$$a(X) = a_0 + a_1X + \cdots + a_{N-1}X^{N-1}$$

中的 $a_0, \dots, a_{N-1}$ 当作 slots。这在 bootstrapping 语境下通常是错误的。

3.2.6.2 正确理解

slot 是 CRT 分解下的分量，即

$$a(X) \longleftrightarrow (a_1, \dots, a_\ell), \quad a_i \in E_i.$$

因此 slot 来自：

- 多项式模分解；
- 根的 Frobenius 轨道；
- 中国剩余定理。

而系数只是大环中某一种特定基底

$$1, X, X^2, \dots, X^{N-1}$$

下的坐标。

3.2.6.3 为什么这个区分很关键

bootstrapping 中最常见的表示变换就是：

- CoeffToSlot：从系数坐标变到 slot 坐标；
- SlotToCoeff：从 slot 坐标变回系数坐标。

如果把 slot 和系数混为一谈，那这两个变换就会变得毫无意义。

注 3.4. “slot 不是系数”不代表系数表示不重要。相反，bootstrapping 的目标之一就是在 coefficient view 与 slot view 之间来回切换。正因为它们是两套不同的坐标系，切换才成为一个非平凡的大线性变换。

3.3 slot 表示与 coefficient 表示

3.3.1 coefficient view

3.3.1.1 定义

在 coefficient view 下，一个明文写成

$$a(X) = a_0 + a_1X + a_2X^2 + \cdots + a_{N-1}X^{N-1}, \quad a_i \in \mathbb{Z}_{p^e}.$$

这里我们把它理解为：

- 一个多项式；
- 或一个长度为 N 的系数向量；
- 其基底是

$$1, X, X^2, \dots, X^{N-1}.$$

3.3.1.2 这个视角最适合做什么

coefficient view 最适合解释：

- RLWE 密文真正加密的对象是什么；
- 解密关系

$$u(X) = m(X) + p^e r(X)$$

中 $m(X)$ 与 $r(X)$ 的意义；

- 多项式乘法、模 $X^N + 1$ 化简；
- 最终输出应该回到哪个“环元素表示”。

3.3.1.3 这个视角不擅长什么

coefficient view 不擅长做逐槽非线性操作。原因是：若你在环里对 $a(X)$ 施加一个一般非线性函数 f ，得到的是

$$f(a(X)),$$

这是在环中的运算，不会逐系数独立作用。

因此若后续需要做 digit extraction 这样的“标量函数”，通常必须先换到 slot view。

3.3.2 slot view

3.3.2.1 定义

在 slot view 下，一个明文经 CRT 分解被看成

$$(a_1, \dots, a_\ell) \in E_1 \times \dots \times E_\ell \cong E^\ell.$$

其中每个 a_i 是一个 slot 值。

3.3.2.2 slot 的内部结构

若是 fully packed，则

$$a_i = \sum_{j=0}^{d-1} a_{i,j} \zeta_{m,i}^j,$$

因此每个 slot 里还含有 d 个底层系数。

若是 sparse packed，则每个 slot 只装一个标量

$$a_i \in \mathbb{Z}_{p^e} \subseteq E_i.$$

3.3.2.3 这个视角最适合做什么

slot view 特别适合解释：

- SIMD 并行；
- 逐槽线性运算；
- 逐槽非线性函数；
- automorphism/rotation 在 slots 上的作用；
- digit extraction 前为什么要先 CoeffToSlot。

3.3.2.4 这个视角不擅长什么

slot view 不擅长解释：

- 密文的 RLWE 结构本体；
- 多项式系数位置；
- 最终如何把所有槽重新装回一个合法的环元素。

这也是为什么 bootstrapping 中一定会出现 SlotToCoeff。

3.3.3 两种表示为何都重要

3.3.3.1 如果只有 coefficient view

你可以清楚地知道密文本体是什么，却很难逐槽并行做 digit extraction 之类的非线性操作。

3.3.3.2 如果只有 slot view

你可以轻松解释 SIMD 与逐槽运算，却看不见密文本体的 ring 结构，也无法解释为什么结果最终必须重新回到一个多项式环元素。

3.3.3.3 bootstrapping 恰好要求两者来回切换

在 BGV/BFV bootstrapping 中，大致是：

ring ciphertext $\rightarrow$ noisy plaintext in coefficient view $\rightarrow$ slot view $\rightarrow$ 逐槽非线性处理 $\rightarrow$ coefficient

因此两种表示不是“谁更本质”的关系，而是：

它们分别服务于 bootstrapping 的不同阶段。

3.4 sparse packed 与 fully packed

3.4.1 sparse packed 的定义

定义 3.5. 若每个 slot 中只装一个来自 $\mathbb{Z}_{p^e}$ 的标量，而不是一般的 E -元素，则称该明文为 *sparsely packed*。

于是一个 sparse packed 明文在 slot view 下长成

$$(m_0, m_1, \dots, m_{\ell-1}), \quad m_i \in \mathbb{Z}_{p^e}.$$

对应到 ring side, 它可以写成

$$m(X) = \sum_{i=1}^{\ell} m_i G_i(X),$$

其中 $G_i(X)$ 是 CRT 幂等元基底。

3.4.2 fully packed 的定义

定义 3.6. 若每个 slot 中装的是一个一般的 slot algebra 元素

$$m_i \in E_i,$$

则称该明文为 *fully packed*。

更具体地, 若选取第 i 个 slot 的局部幂基, 则

$$m_i = \sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j.$$

因此 fully packed 明文在 slot view 下可写作

$$\left(\sum_{j=0}^{d-1} m_{0,j} \zeta_{m,0}^j, \sum_{j=0}^{d-1} m_{1,j} \zeta_{m,1}^j, \dots, \sum_{j=0}^{d-1} m_{\ell-1,j} \zeta_{m,\ell-1}^j \right).$$

3.4.3 两者的本质差别

3.4.3.1 自由度不同

sparse packed 的总自由度是

$$\ell.$$

fully packed 的总自由度是

$$\ell d = N.$$

也就是说:

- sparse packed 只用到整个明文环的一小部分自由度;
- fully packed 则把整个环的维度全部装满。

3.4.3.2 slot 内部结构不同

在 sparse packed 中, slot 是“标量槽”。

在 fully packed 中, slot 是“ d -维小向量槽”。

这会直接影响:

- 是否需要 M 与 M^{-1} ;
- 是否需要 unpack / repack;
- digit extraction 前是否必须先把 E -slot 拆成标量槽。

3.4.3.3 线性变换复杂度不同

对于 sparse packed, slot-to-coefficient 往往只涉及一个 Vandermonde/FFT 型矩阵 U 。

对于 fully packed, 完整变换通常是

$$M^{-1}UM$$

或其逆; 其中 U 只负责外层 slot 混合, 而 M 、 M^{-1} 负责局部幂基与统一幂基之间的切换。

3.4.4 什么时候两者会退化成同一回事

这发生在

$$d = 1$$

时。

3.4.4.1 原因

若 $d = 1$, 则每个不可约因子 F_i 都是一次的, 故

$$E_i \cong \mathbb{Z}_{p^e}.$$

因此每个 slot algebra 本身就没有内部维度了。

于是:

- fully packed: 每个 slot 里装一个一般 E_i -元素;
- sparse packed: 每个 slot 里装一个 $\mathbb{Z}_{p^e}$ -标量;

两者就变成同一件事。

3.4.4.2 等价现象

此时:

- M 与 M^{-1} 变平凡;
- unpack / repack 没有存在必要;
- slot algebra 的内部结构消失;
- 只剩一个最简单的 slot / coefficient 变换。

注 3.7. 从数论上看, $d = 1$ 等价于

$$p \equiv 1 \pmod{m}.$$

这也是 slot 数达到最大值时的极端情形。

3.5 子环、稀疏幂次与 packing 结构

3.5.1 子环的意义

上一章已经证明: power-of-two 情形下, 每个因子 $F_i(X)$ 都只依赖于 X^c , 因此 sparse packed 明文天然落在某个更小的子环中。

这件事的意义非常大:

- (1) 它说明 sparse packed 不是“任意少装一点数据”, 而是代数上真的落在一个更小的 *ring*;
- (2) 它使 sparse packed 情况下的 slot-to-coefficient 变成一个更标准、更像 Vandermonde/FFT 的问题;
- (3) 它为 thin bootstrap、partial CoeffToSlot 等优化路线提供了结构基础。

3.5.2 为什么 sparse packed 会落在子环

由上一章稀疏因子形状定理可知:

$$F_i(X) \in \mathbb{Z}_{p^e}[X^c], \quad c = \begin{cases} d, & p \equiv 1 \pmod{4}, \\ d/2, & p \equiv 3 \pmod{4}. \end{cases}$$

而 sparse packed 明文由 CRT 基底 $G_i(X)$ 的线性组合构成:

$$m(X) = \sum_{i=1}^{\ell} m_i G_i(X).$$

由于每个 $G_i(X)$ 同样属于 $\mathbb{Z}_{p^e}[X^c]$ ，所以整个 $m(X)$ 也在 $\mathbb{Z}_{p^e}[X^c]$ 中。

也就是说，sparse packed 明文不是一般的大环元素，而是满足：

只会出现 $1, X^c, X^{2c}, \dots$ 这些幂次的稀疏多项式。

3.5.3 X^c 变量替换与“外层多项式”

3.5.3.1 为什么要做换元

若一个多项式只含幂次

$$1, X^c, X^{2c}, \dots, X^{c(\ell-1)},$$

那么最自然的重写方式就是设

$$Y := X^c.$$

于是原来的稀疏多项式

$$a_0 + a_1 X^c + a_2 X^{2c} + \dots + a_{\ell-1} X^{c(\ell-1)}$$

就变成普通多项式

$$q(Y) = a_0 + a_1 Y + a_2 Y^2 + \dots + a_{\ell-1} Y^{\ell-1}.$$

3.5.3.2 为什么次数到 $\ell - 1$

因为 sparse 子环的维数正好是

$$N' = \frac{N}{c}.$$

在最典型的 sparse packed 情形，这个维数正好等于 slot 数 ℓ 。因此用 Y 作为变量时，一个子环元素只需要 ℓ 个系数，自然可写成次数不超过 $\ell - 1$ 的多项式。

3.5.3.3 “外层多项式”这个说法是什么意思

这里称 $q(Y)$ 为“外层多项式”，是因为它只描述 sparse packed 结构中 slot 之间的那一层索引，不再涉及 fully packed 时 slot 内部的 d -维小向量结构。

换句话说：

- Y^k 对应第 k 个 sparse slot 的位置；
- $q(Y)$ 的系数 $(a_0, \dots, a_{\ell-1})$ 就是这些 sparse slots 里装的标量。

3.5.3.4 为什么这能把 CRT 看成普通插值

若第 i 个 slot 对应的点记为

$$y_i = \zeta_{m,i}^c,$$

那么

$$q(y_i) = a_0 + a_1 y_i + \cdots + a_{\ell-1} y_i^{\ell-1}.$$

因此从系数向量 $(a_0, \dots, a_{\ell-1})$ 到 slot 值 $(q(y_0), \dots, q(y_{\ell-1}))$ 的变换，正是一个标准的 Vandermonde 型求值矩阵。

这就是为什么 sparse packed 情况下的 U 矩阵，会看起来与 CKKS 中的 FFT/Vandermonde 矩阵非常相似。

直观解释 3.8. $q(Y)$ 不是什么新发明。它只是提醒你：

当明文只依赖 X^c 时，问题已经被压缩成“一个普通多项式在若干点上的取值/插值问题”。

也正因为这样，sparse packed 的 CoeffToSlot / SlotToCoeff 比 fully packed 简洁得多。

3.6 本章小结

本章围绕“明文空间是如何被解释成 slots 的”这一问题，建立了后续 bootstrapping 的表示层基础。主要结论如下：

- (1) **SIMD / batching 的来源：** BGV/BFV 之所以能在一个密文中并行处理多个消息，是因为明文环

$$R_{p^e}$$

经 CRT 可分解成多个 slot algebra 的直积。

- (2) **slot 的本质：** slot 是 CRT 分量，不是多项式系数位置。slot view 与 coefficient view 是同一个明文环元素的两套不同坐标系。

- (3) **slot algebra 的作用：** 每个 slot 都不是“抽象的盒子”，而是一个实际的小环

$$E_i = \mathbb{Z}_{p^e}[X]/(F_i(X)).$$

fully packed 时，每个 slot 里甚至还带着 d -维内部结构。

- (4) **sparse 与 fully packed 的区别：** sparse packed 每个 slot 只装一个 $\mathbb{Z}_{p^e}$ -标量；fully packed 每个 slot 装一个一般 E_i -元素。前者只用到部分自由度，后者用满整个明文环维度。

- (5) **何时两者退化一致**: 当 $d = 1$ 时, 每个 slot algebra 都退化成 $\mathbb{Z}_{p^e}$, 于是 sparse packed 与 fully packed 变成同一回事。
- (6) **子环与稀疏幂次**: sparse packed 明文天然落在某个只依赖 X^c 的子环中, 因此可设

$$Y = X^c$$

把问题改写为一个普通多项式

$$q(Y)$$

的求值/插值问题。

- (7) **对后续 bootstrapping 的意义**: 本章解释了为什么后面会出现:

- CoeffToSlot / SlotToCoeff;
- sparse 与 fully packed 两条不同路线;
- U 型 Vandermonde/FFT-like 矩阵;
- unpack / repack 这类专门针对 fully packed 的操作。

下一章将进入 *Automorphism*、*rotation* 与线性变换, 解释:

- $a(X) \mapsto a(X^j)$ 到底意味着什么;
- 为什么 slot permutation 能由自同构实现;
- one-dimensional rotation 从何而来;
- 为什么一般线性变换能写成 weighted rotations 的和。

Automorphism、rotation 与线性变换

4.1 自同构群的基本概念

4.1.1 为什么这一章重要

在 bootstrapping 的线性变换实现里，最常见也最昂贵的基本操作之一，就是所谓的 *rotation*。但如果直接把 *rotation* 理解成“把 slot 往左挪一格”，很容易忽略它背后的真正来源：

rotation 并不是凭空定义的一种操作，而是分圆环自同构在 slot 视角下的表现。

因此，要理解：

- 为什么同一个密文可以做 slot permutation；
- 为什么 *rotation* 通常需要配合 key switching；
- 为什么有些 *rotation* 能直接由单个 automorphism 实现，而有些不行；
- 为什么一般线性变换能写成 weighted rotations 的和；

必须先搞清楚分圆环的自同构群。

4.1.2 抽象定义

定义 4.1. 设 A 是一个交换环。若映射

$$\varphi : A \rightarrow A$$

同时满足：

- (1) φ 是双射；
- (2) 对任意 $a, b \in A$ ，有

$$\varphi(a + b) = \varphi(a) + \varphi(b), \quad \varphi(ab) = \varphi(a)\varphi(b);$$

(3) $\varphi(1) = 1$;

则称 φ 是 A 的一个环自同构。

所有环自同构在映射复合下构成一个群，记作

$$\text{Aut}(A).$$

4.1.3 我们关心哪个环的自同构

在 BGV/BFV 的分圆框架里，底层对象通常是

$$R = \mathbb{Z}[X]/(\Phi_m(X))$$

或它的模 q 版本

$$R_q = \mathbb{Z}_q[X]/(\Phi_m(X)).$$

在 power-of-two 情形中，

$$\Phi_m(X) = X^N + 1, \quad m = 2N.$$

因此我们最关心的是这些分圆环的自同构群。

4.1.4 为什么分圆环的自同构很特殊

分圆环并不是任意商环。它由一个“本原单位根”生成，因此一个自同构只要决定了这个本原单位根的像，就决定了整个环的作用方式。

更直观地说：

整个自同构群的自由度，几乎都浓缩在“ ζ_m 要被送到哪个本原 m 次单位根”这件事上。

这也是为什么后面所有 automorphism 都会长成

$$a(X) \mapsto a(X^j)$$

这样的形式。

4.1.5 分圆环自同构群的结构

定理 4.2. 设

$$R = \mathbb{Z}[\zeta_m] \cong \mathbb{Z}[X]/(\Phi_m(X)).$$

则

$$\text{Aut}(R/\mathbb{Z}) \cong \mathbb{Z}_m^\times.$$

更具体地，对每个 $j \in \mathbb{Z}_m^\times$ ，存在唯一一个自同构 τ_j ，使得

$$\tau_j(\zeta_m) = \zeta_m^j.$$

证明. 分圆环 R 由 ζ_m 生成, 因此任一 $\mathbb{Z}$ -代数自同构都由 ζ_m 的像唯一决定。而 ζ_m 的像必须仍是 $\Phi_m(X)$ 的根, 也即某个本原 m 次单位根 ζ_m^j , 其中 $j \in \mathbb{Z}_m^\times$ 。反过来, 每个 $j \in \mathbb{Z}_m^\times$ 都定义了一个映射

$$\zeta_m \mapsto \zeta_m^j,$$

并由生成元延拓为整个环上的自同构。复合关系满足

$$\tau_j \circ \tau_k = \tau_{jk},$$

因此得到群同构

$$\text{Aut}(R/\mathbb{Z}) \cong \mathbb{Z}_m^\times.$$

□

直观解释 4.3. 这一定理说明:

- 自同构并不是很多“杂乱无章”的变换;
- 它们恰好被模 m 的可逆剩余类参数化;
- 后面出现的 τ_j 、rotation、Frobenius、slot permutation, 本质上都在这个群里。

4.1.6 power-of-two 情形下的群结构回顾

上一章已经提到, 当 $m = 2^r$ 且 $r \geq 3$ 时,

$$\mathbb{Z}_m^\times \cong \langle -1 \rangle \times \langle 5 \rangle.$$

因此分圆环的自同构群可理解为:

- 一个由 -1 生成的长度为 2 的方向;
- 一个由 5 生成的长循环方向。

这将在后面 slot 几何与 one-dimensional rotation 的讨论中发挥核心作用。

4.2 $a(X) \mapsto a(X^j)$ 的真正含义

4.2.1 从 $\zeta_m \mapsto \zeta_m^j$ 到 $a(X) \mapsto a(X^j)$

在分圆表示中, 一个元素 $a \in R$ 可看作

$$a(\zeta_m)$$

的形式，其中 $a(X) \in \mathbb{Z}[X]$ 是某个代表多项式。若自同构满足

$$\tau_j(\zeta_m) = \zeta_m^j,$$

则

$$\tau_j(a(\zeta_m)) = a(\tau_j(\zeta_m)) = a(\zeta_m^j).$$

在多项式商环表示

$$R = \mathbb{Z}[X]/(\Phi_m(X))$$

下，这恰好对应于：

$$\tau_j : a(X) \longmapsto a(X^j).$$

因此，式子

$$a(X) \mapsto a(X^j)$$

不是一种“额外定义的魔法操作”，而只是把

$$\zeta_m \mapsto \zeta_m^j$$

翻译回多项式记号之后的写法。

4.2.2 为什么 j 必须属于 $\mathbb{Z}_m^\times$

若 $j \notin \mathbb{Z}_m^\times$ ，则 $\gcd(j, m) \neq 1$ ，这会导致 ζ_m^j 不再是本原 m 次单位根。于是 ζ_m 被送到的对象不再是 $\Phi_m(X)$ 的“同类根”，也就不能定义分圆环的自同构。

因此只有

$$j \in \mathbb{Z}_m^\times$$

才合法。

4.2.3 在多项式系数视角下会发生什么

若

$$a(X) = \sum_{t=0}^{N-1} a_t X^t,$$

则

$$a(X^j) = \sum_{t=0}^{N-1} a_t X^{jt}.$$

由于我们工作在

$$R_q = \mathbb{Z}_q[X]/(X^N + 1)$$

中，所有高次项都还要继续利用

$$X^N = -1$$

进行化简。

因此 τ_j 在系数视角下通常表现为：

- 系数位置重排；
- 某些项额外带上符号；
- 最终仍得到一个次数小于 N 的多项式。

4.2.4 一个简单例子

设 $N = 4$ ，于是

$$R_q = \mathbb{Z}_q[X]/(X^4 + 1).$$

取

$$a(X) = a_0 + a_1X + a_2X^2 + a_3X^3.$$

若取 $j = 3$ ，则

$$a(X^3) = a_0 + a_1X^3 + a_2X^6 + a_3X^9.$$

在商环中有

$$X^4 = -1, \quad X^6 = X^2X^4 = -X^2, \quad X^9 = X(X^4)^2 = X.$$

故

$$a(X^3) = a_0 + a_3X - a_2X^2 + a_1X^3.$$

这说明： τ_3 在系数视角下看起来像一种带符号的置换。

注 4.4. 后面在实现 rotation 时，人们常说 automorphism 是“便宜的置换”，这句话有一半对、一半不够精确。更准确地说，automorphism 在系数层面是“由幂指数映射诱导的线性变换”，在很多 power-of-two 场景下确实表现为带符号的稀疏重排。

4.2.5 为什么这一步对密文实现有意义

由于密文是 R_Q 中元素组成的向量，施加 τ_j 意味着：

- 对密文分量的每个环元素都执行 $a(X) \mapsto a(X^j)$ ；
- 得到一个“在新秘密钥 $\tau_j(s)$ 下有效”的密文；
- 因而通常还要做一次 key switching，把它切回原秘密钥下。

因此一个“rotation 操作”在实现层面往往包含：

$$\text{automorphism} + \text{key switching}.$$

4.3 在 slot 视角下的作用

4.3.1 slot permutation

上一章已经建立了 slot 视角：

$$R_{p^e} \cong E^\ell, \quad a(X) \mapsto \{a(\zeta_m^h)\}_{h \in S},$$

其中 S 是 $\mathbb{Z}_m^\times / \langle p \rangle$ 的一组代表元。

现在考察自同构 τ_j 的作用。对任意 $h \in S$ ，有

$$\tau_j(a)(\zeta_m^h) = a(\zeta_m^{hj}).$$

因此在 slot 视角下，自同构 τ_j 的作用是：

把标签为 h 的 slot 送到标签为 hj 的 slot。

也就是说，它本质上是一个 slot permutation。

4.3.2 为什么是乘法而不是加法

因为 slot 标签来自商群

$$\mathbb{Z}_m^\times / \langle p \rangle,$$

而 automorphism 参数 $j \in \mathbb{Z}_m^\times$ 的作用来自

$$\zeta_m^h \mapsto \zeta_m^{hj}.$$

所以它天然就是群乘法作用，而不是简单的整数加法平移。

后面人们把某些特殊乘法作用解释成“往某一维平移一步”，那是建立在适当选取生成元、把群写成多维循环坐标之后的结果，而不是原始定义本身。

4.3.3 在商群上的良定义性

如果两个代表元满足

$$h' = hp^t,$$

那么

$$h'j = hp^tj = hjp^t.$$

也就是说， hj 与 $h'j$ 仍落在同一个 Frobenius 轨道里。因此

$$h \mapsto hj$$

在商群

$$\mathbb{Z}_m^\times / \langle p \rangle$$

上是良定义的。

这就解释了为什么 automorphism 真正作用的是 slot 索引群，而不是某个任意选择的代表元集合。

4.3.4 slot 内部 Frobenius

除了“把一个 slot 送到另一个 slot”，自同构在某些情况下还会在单个 slot 内部表现为 Frobenius 作用。

4.3.4.1 原因

某个 slot 实际上对应一个 Frobenius 轨道

$$\zeta_m^h, \zeta_m^{hp}, \zeta_m^{hp^2}, \dots, \zeta_m^{hp^{d-1}}.$$

若 j 的作用没有把轨道代表送到另一个新轨道，而只是落在同一轨道内部不同位置，那么 τ_j 在该 slot 上的表现就是一个 Frobenius 幂。

换句话说：

- 外层看，它可能没离开这个 slot；
- 内层看，它可能把该 slot 中的 E -元素做了一个非平凡自同构。

4.3.4.2 特别重要的例子： $\sigma = \tau_p$

当

$$j = p$$

时，自同构

$$\sigma := \tau_p$$

在每个 slot 上正是 Frobenius 作用。因此 fully packed 场景下， σ 常被看成“slot 内部的基础对称操作”。

直观解释 4.5. 从 slot 视角看，一个 automorphism 可能同时做两件事：

- (1) 在外层 permute slots；
- (2) 在内层对某个 slot 的 E -元素做 Frobenius。

这也正是 later literature 中“rotation 不一定等于单个 automorphism”的根源。

4.4 one-dimensional rotation 的定义

4.4.1 为什么需要“坐标化”的 slot 几何

前一节已经说明，automorphism 原始上是

$$h \mapsto hj$$

的群作用。但在算法实现中，我们更希望把 slots 看成一个一维或多维数组，然后谈：

- 沿第 1 维转一步；
- 沿第 2 维转 v 步；
- 只在某一维做线性变换。

这要求我们先把 slot 索引群坐标化。

4.4.2 slot 索引群的直积表示

设

$$\mathbb{Z}_m^\times / \langle p \rangle \cong C_{\ell_1} \times \cdots \times C_{\ell_t}.$$

选取生成元 $g_1, \dots, g_t$ ，则任一 slot 标签可唯一写成

$$h = g_1^{e_1} \cdots g_t^{e_t}, \quad 0 \leq e_i < \ell_i.$$

因此每个 slot 都可表示为坐标

$$(e_1, \dots, e_t).$$

4.4.3 定义 one-dimensional rotation

定义 4.6. 固定某个维度 $i \in \{1, \dots, t\}$ 与步长 $v \in \mathbb{Z}$ 。第 i 维 rotation ρ_i^v 定义为：

$$\rho_i^v(e_1, \dots, e_i, \dots, e_t) = (e_1, \dots, e_i + v \pmod{\ell_i}, \dots, e_t).$$

更简洁地说，它只改变第 i 个坐标，其余坐标保持不变。

4.4.4 为什么这看起来像“循环移位”

因为每一维坐标 e_i 都活在一个循环群 C_{ℓ_i} 中。因此

$$e_i \mapsto e_i + v \pmod{\ell_i}$$

本质上就是一个循环移位。

特别地：

- 若 $t = 1$ ，所有 slots 排成一个环；
- 若 $t = 2$ ，slots 排成一个二维循环网格；
- 若继续重解释成很多个大小为 2 的维度，则会出现 FFT-like 的二分蝶形结构。

4.4.5 与 automorphism 的关系

若存在某个 $j \in \mathbb{Z}_m^\times$ ，使得在 slot 索引群中，

$$h \mapsto hj$$

恰好等同于

$$(e_1, \dots, e_i, \dots, e_t) \mapsto (e_1, \dots, e_i + v, \dots, e_t),$$

那么我们就说：该 rotation 由 automorphism τ_j 实现。

但是，这种对应不总是如此简单。接下来要解释：为什么某些 one-dimensional rotation 不能直接由单个 automorphism 完成。

4.5 为什么 rotation 不一定等于单个 automorphism

4.5.1 表面上的困惑

我们已经知道：

- slot 可以用 $(e_1, \dots, e_t)$ 坐标表示；
- rotation 只是某个坐标加一；
- automorphism 在 slot 上对应群作用 $h \mapsto hj$ 。

于是很自然会想：

既然 slot 索引群本身就是由生成元 g_i 组成的，那沿第 i 维转一步，难道不就是“乘上 g_i ”吗？

答案是：在商群里是这样，但在原群里不一定完全如此。

4.5.2 关键区别：商群里的阶 vs 原群里的值

当我们说 g_i 在 slot 索引群中对应长度为 ℓ_i 的那一维时，这真正表示的是：

$$g_i^{\ell_i} \in \langle p \rangle.$$

也就是说， g_i 在商群

$$\mathbb{Z}_m^\times / \langle p \rangle$$

中的阶为 ℓ_i 。

但这并不意味着

$$g_i^{\ell_i} \equiv 1 \pmod{m}.$$

注 4.7. 这两个条件有本质区别：

$$g_i^{\ell_i} \in \langle p \rangle \quad \text{vs.} \quad g_i^{\ell_i} \equiv 1 \pmod{m}.$$

前者说的是“在商群里回到单位元”；后者说的是“在原群里真回到 1”。

4.5.3 为什么这会影响 rotation 的实现

如果某个维度满足更强条件

$$g_i^{\ell_i} \equiv 1 \pmod{m},$$

那么沿这一维转一圈后，在原群里也真的回到起点，因此该维度上的 rotation 往往可由一个单独 automorphism 干净实现。

但若只有

$$g_i^{\ell_i} \in \langle p \rangle,$$

则意味着沿这一维转一圈以后，在外层 slot 索引上虽然回到原位置，但在同一 slot 内部可能多了一个 Frobenius 幂。于是“理想 rotation”与“单个 automorphism”的作用之间会出现偏差。

直观解释 4.8. 外层看，你似乎回到了原 slot；内层看，你其实还转动了 slot 里面的坐标系。这就是“rotation 不一定等于单个 automorphism”的本质。

4.5.4 good dimension 与 bad dimension 的预告

这种偏差导致了一个重要分类：

- **good dimension**：一圈真的回到 1，所以 rotation 很干净；
- **bad dimension**：一圈只回到 $\langle p \rangle$ ，因此 rotation 需要补偿。

接下来我们把这件事写得更具体。

4.6 掩码、两支 automorphism 与 bad/good dimension

4.6.1 good dimension 的定义

定义 4.9. 设第 i 维由生成元 g_i 与长度 ℓ_i 描述。若

$$g_i^{\ell_i} \equiv 1 \pmod{m},$$

则称该维度为 *good dimension*。

在这种情况下, 第 i 维 rotation 的实现较为简单, 往往可以直接由一个 automorphism 完成。

4.6.2 bad dimension 的定义

定义 4.10. 若某维只满足

$$g_i^{\ell_i} \in \langle p \rangle$$

但

$$g_i^{\ell_i} \not\equiv 1 \pmod{m},$$

则称该维度为 *bad dimension*。

4.6.3 bad dimension 中发生了什么

设我们想沿第 i 维做 rotation。若单独使用 automorphism τ_{g_i} , 那么它确实会在大多数位置上把 slot 送到“下一格”, 但在绕回边界时, 会出现一个偏移:

- 外层 slot 索引应该绕回开头;
- 实际上却多带了一个 $\langle p \rangle$ 内的因子;
- 这个额外因子在 slot 内部表现为 Frobenius。

因此, 一个“理想的循环平移”不能再由单个 automorphism 精确给出。

4.6.4 为什么会出现“掩码 + 两支 automorphism”

解决方法是把 slot 分成两部分:

- (1) 那些不会越过边界的部分;
- (2) 那些会在边界处绕回的部分。

对第一部分，可以直接用某个 automorphism；对第二部分，则用另一个 automorphism，并配合掩码把它只作用在需要绕回的位置。

于是 bad dimension 中的 rotation 常写成类似

$$\rho_i^v(m) = \mu \cdot \tau_{j_1}(m) + (1 - \mu) \cdot \tau_{j_2}(m),$$

其中：

- μ 是一个 plaintext mask；
- τ_{j_1} 负责“不绕回”的那一段；
- τ_{j_2} 负责“绕回”的那一段。

直观解释 4.11. 这就像你在做一个循环数组移位，但实现底层只有“线性平移”而没有“自动绕回”。于是你必须：

- 先把中间段正常平移；
- 再把从尾巴掉出去的那一段单独搬回头部；
- 最后把两段拼起来。

在 FHE 里，这种“拼起来”就靠 mask 实现。

4.6.5 为什么 good dimension 可以省掉一半麻烦

若某维是 good dimension，则无边界偏差，因而不需把 rotation 拆成两支 automorphism。这会直接减少：

- automorphism 的数量；
- key switching 的数量；
- mask 的数量；
- 后续 BSGS 中内层求和的成本。

因此，在 bootstrapping 的复杂度分析中，good/bad dimension 是一个非常关键的结构性区别。

4.7 一般线性变换如何写成 rotations 的加权和

4.7.1 问题背景

到了这一节，我们终于可以回答一个极其核心的问题：

为什么 bootstrapping 里的大线性变换，经常能写成“若干个 rotations 的加权和”？

这件事不是实现技巧，而是一个线性代数事实。

4.7.2 先从最简单的循环情形说起

设有一个长度为 n 的向量

$$x = (x_0, x_1, \dots, x_{n-1})^T.$$

定义循环移位矩阵 P_k ，使得

$$P_k x$$

表示把向量循环移位 k 步。

则任意矩阵

$$A \in M_n(\mathbb{F})$$

都可以唯一写成

$$A = \sum_{k=0}^{n-1} D_k P_k,$$

其中 D_k 是对角矩阵。

定理 4.12 (循环置换基分解). 设 P_k 为大小为 n 的循环移位矩阵，则对任意

$$A \in M_n(\mathbb{F}),$$

存在唯一的一组对角矩阵 $D_0, \dots, D_{n-1}$ ，使得

$$A = \sum_{k=0}^{n-1} D_k P_k.$$

证明. 矩阵 $D_k P_k$ 的非零元恰好位于第 k 条循环对角线上。任意矩阵 A 的每个元素 A_{ij} 都唯一落在某条循环对角线

$$j - i \equiv k \pmod{n}$$

上。因此可将 A 按循环对角线分组，每一组由一个对角矩阵 D_k 提供系数，从而得到唯一分解。 $\square$

直观解释 4.13. 这一定理的意思非常朴素：

“任意矩阵 = 若干种循环移位 + 每种移位前面乘一个按位置变化的权重。”

这正是 later literature 中“weighted rotations”这个说法的线性代数基础。

4.7.3 把这个思想搬到 slot 几何里

在 BGV/BFV 中, rotation 不是普通的二维矩阵循环移位, 而是 slot 索引群上的 one-dimensional rotation ρ_i^v 。但本质思想一样:

- 先固定某一维 i ;
- 沿这维有若干种 rotation ρ_i^v ;
- 任意在这维上作用的线性变换, 都可写成这些 rotations 的加权和。

因此一个第 i 维的线性变换通常可写成

$$L(m) = \sum_{v=0}^{\ell_i-1} \kappa(v) \cdot \rho_i^v(m).$$

这里 $\kappa(v)$ 表示与第 v 个 rotation 相关的权重; 在实际实现里, 它往往对应某个对角掩码或 slot-wise plaintext multiplier。

4.7.4 为什么这件事对 bootstrapping 至关重要

bootstrapping 中的 CoeffToSlot、SlotToCoeff、unpack、repack、trace 等, 都本质上是大线性变换。而在 FHE 里我们最擅长实现的基本操作之一就是:

- automorphism / rotation;
- 乘一个 plaintext mask;
- 然后求和。

因此, 如果一个大线性变换能写成

$$L = \sum_v \kappa(v) \rho^v,$$

那么它就能被同态实现为:

- (1) 对密文做若干个 rotations;
- (2) 每个 rotation 后乘一个 plaintext 权重;
- (3) 把所有项加起来。

这就是后面 BSGS、hoisting、稀疏矩阵分解等优化的共同起点。

4.7.5 bad dimension 下的细化

若 ρ_i^v 在 bad dimension 中不能由一个 automorphism 直接实现，那么上式里的每个 ρ_i^v 还要继续写成

$$\rho_i^v = \mu_v \cdot \tau_{j_1(v)} + (1 - \mu_v) \cdot \tau_{j_2(v)}.$$

于是整个线性变换变成

$$L(m) = \sum_v \kappa(v) \mu_v \tau_{j_1(v)}(m) + \sum_v \kappa(v) (1 - \mu_v) \tau_{j_2(v)}(m).$$

这就解释了为什么 bad dimension 会显著增加 automorphism 数量。

4.7.6 从 weighted rotations 到 BSGS 的桥梁

一旦把一般线性变换写成

$$L(m) = \sum_{v=0}^{\ell_i-1} \kappa(v) \rho_i^v(m),$$

后面 BSGS 要做的事就很清楚了：

不是改变变换本身，而是重排求和顺序，减少昂贵的 automorphism 次数。

也就是说：

- 线性代数层面：先有 weighted rotations 分解；
- 算法层面：再对这些 rotations 做 baby-step / giant-step 优化。

4.8 本章小结

本章建立了 bootstrapping 线性变换的“群作用视角”。主要结论如下：

(1) **自同构群来源**：分圆环的自同构群同构于

$$\mathbb{Z}_m^\times,$$

每个 $j \in \mathbb{Z}_m^\times$ 对应一个自同构

$$\tau_j : a(X) \mapsto a(X^j).$$

(2) **slot 视角下的作用**： τ_j 在 slot 上表现为

$$h \mapsto hj,$$

因而本质上是 slot permutation；同时在 slot 内部还可能表现为 Frobenius。

- (3) **rotation 的来源**: 在把 slot 索引群写成有限 Abel 群直积后, one-dimensional rotation 就是只改变某一维坐标的循环移位。
- (4) **rotation 与 automorphism 的差别**: rotation 是 slot 几何上的理想平移; automorphism 是原始分圆群上的乘法作用。两者只有在 good dimension 中才能简单一致, 在 bad dimension 中往往需要 mask 加两支 automorphism 拼起来。
- (5) **good/bad dimension 的意义**: 它决定某个 rotation 是否能由单个 automorphism 直接实现, 从而直接影响 bootstrapping 中 automorphism 和 key switching 的成本。
- (6) **一般线性变换的表示**: 任意沿某一维的线性变换都可以写成

$$L(m) = \sum_v \kappa(v) \rho_i^v(m),$$

即 weighted rotations 的加权和。这正是后续 BSGS、hoisting、FFT-like 分解的线性代数起点。

下一章将正式进入 *BGV/BFV* 中的 *CoeffToSlot* 与 *SlotToCoeff*, 重点解释:

- 这两个变换到底从哪到哪;
- sparse packed 下为什么只剩一个 U ;
- fully packed 下为什么必须写成 $M^{-1}UM$;
- unpack / repack 为什么会不可避免地出现。

BGV/BFV 中的 CoeffToSlot 与 SlotToCoeff

5.1 这两个变换到底是什么

5.1.1 从表示方向理解

上一章已经反复强调：在 BGV/BFV 中，同一个明文环元素可以用两种方式来理解：

(1) **coefficient view**：视为

$$a(X) = a_0 + a_1X + \cdots + a_{N-1}X^{N-1} \in R_{p^e},$$

即大环

$$R_{p^e} = \mathbb{Z}_{p^e}[X]/(X^N + 1)$$

中的系数表示；

(2) **slot view**：经 CRT 分解后视为

$$(a_1, \dots, a_\ell) \in E^\ell,$$

其中每个分量属于某个 slot algebra。

因此，所谓 CoeffToSlot 与 SlotToCoeff，本质上就是同一个明文元素在两套坐标系之间的转换。

定义 5.1 (CoeffToSlot). 称

$$\text{CoeffToSlot} : R_{p^e} \longrightarrow E^\ell$$

为从 coefficient view 到 slot view 的变换。也就是说，它把一个大环元素

$$a(X)$$

送到它的 CRT 分量

$$(a_1, \dots, a_\ell).$$

定义 5.2 (SlotToCoeff). 称

$$\text{SlotToCoeff} : E^\ell \longrightarrow R_{p^e}$$

为从 slot view 到 coefficient view 的变换。它是上面 CoeffToSlot 的逆映射。

注 5.3. 这两个变换在数学上互为逆映射：

$$\text{SlotToCoeff} = \text{CoeffToSlot}^{-1}.$$

因此它们都只是 表示变换，不改变消息本身，只改变同一个明文环元素的坐标系。

5.1.2 从 bootstrapping 流程角色理解

虽然从数学定义上看，CoeffToSlot 与 SlotToCoeff 只是两套坐标之间的变换，但在 bootstrapping 流程中，它们分别承担不同角色。

5.1.2.1 CoeffToSlot 的流程角色

在 BGV 的同态解密线性部分之后，我们会得到一个 noisy plaintext

$$u(X) = m(X) + p^e r(X),$$

它最自然地处于 coefficient view 下。

如果后续想对它做 digit extraction 这样的“逐槽标量函数”，通常必须先把它打开到 slots：

$$u(X) \xrightarrow{\text{CoeffToSlot}} (u_1, \dots, u_\ell).$$

于是 CoeffToSlot 的作用是：

把环元素变成方便逐槽处理的表示。

5.1.2.2 SlotToCoeff 的流程角色

在 digit extraction、trace 或其它逐槽处理完成之后，我们得到的是“每个 slot 中都被修复”的结果：

$$(v_1, \dots, v_\ell).$$

但最终系统仍需要一个合法的环元素密文，因此必须把它重新装回

$$R_{p^e}.$$

这就是 SlotToCoeff 的作用：

$$(v_1, \dots, v_\ell) \xrightarrow{\text{SlotToCoeff}} v(X).$$

SlotToCoeff 的作用是把逐槽修复后的结果重新组合成一个大环元素。

5.1.2.3 在 bootstrapping 中它们为什么不可省

如果不做 `CoeffToSlot`，那么你面对的是整个环元素 $u(X)$ ，而不是若干可独立处理的槽值；`digit extraction` 这种单变量函数就很难并行地施加。如果不做 `SlotToCoeff`，那么最终只得到 slot 视角下的数据，而得不到一个能继续参与后续 FHE 运算的合法密文。

因此，在 packed bootstrapping 中，这两个变换通常都是不可省的核心步骤。

5.1.3 容易混淆的命名问题

这部分非常容易混，必须单独澄清。

5.1.3.1 按“表示方向”命名

最严格、最不容易出错的命名方式是：

- `CoeffToSlot`：系数表示 $\rightarrow$ slot 表示；
- `SlotToCoeff`：slot 表示 $\rightarrow$ 系数表示。

这就是本讲义采用的标准。

5.1.3.2 按“bootstrapping 流程角色”命名

在一些文献或口头讨论里，人们会把

- 把密文中的 noisy plaintext “打开”到 slots 的步骤称为“同态解码”；
- 把逐槽处理后的结果重新装回多项式系数的步骤称为“同态编码”。

这会导致：

$$\text{CoeffToSlot} \leftrightarrow \text{decode}, \quad \text{SlotToCoeff} \leftrightarrow \text{encode},$$

从而与“表示方向”命名产生交错。

5.1.3.3 本讲义的约定

为了避免混乱，本章始终坚持以下约定：

`CoeffToSlot` / `SlotToCoeff` 一律只按“表示方向”命名。

至于“编码/解码”这两个词，只作为流程直觉使用，不作为正式定义。

5.2 sparse packed 下的 CoeffToSlot / SlotToCoeff

5.2.1 为什么只剩一个 U

5.2.1.1 关键原因：每个 slot 里只剩标量

在 sparse packed 情况下，每个 slot 中只装一个来自 $\mathbb{Z}_{p^e}$ 的标量：

$$(m_0, \dots, m_{\ell-1}), \quad m_i \in \mathbb{Z}_{p^e}.$$

也就是说，slot algebra 的内部 d -维结构并没有真的被使用。

因此：

- 不需要再区分局部幂基

$$1, \zeta_{m,i}, \dots, \zeta_{m,i}^{d-1}$$

与统一幂基

$$1, \zeta_m, \dots, \zeta_m^{d-1};$$

- 也就不需要前后两个基变换 M 与 M^{-1} 。

于是完整变换退化成了一个单独的矩阵 U （或其逆）即可完成。

5.2.1.2 从维数上看为什么合理

在 sparse packed 下，总自由度是

$$\ell.$$

而对应的子环 R'_{p^e} 维数也是 ℓ 。因此，此时 coefficient 表示与 slot 表示之间就是两个 ℓ -维线性空间之间的同构，自然只需要一个 $\ell \times \ell$ 矩阵。

而 fully packed 时，总自由度是

$$\ell d = N,$$

内部还含有额外的 d -维结构，这时就不能只靠一个 $\ell \times \ell$ 矩阵解决全部问题。

5.2.1.3 一个精确说法

命题 5.4. 在 sparse packed 情况下， CoeffToSlot 与 SlotToCoeff 都可由某个 $\ell \times \ell$ 可逆矩阵 U 及其逆 U^{-1} 实现，不需要额外的基变换矩阵 M, M^{-1} 。

证明. 由于 sparse packed 明文天然落在某个 ℓ -维子环 R'_{p^e} 中，而其 slot 表示也是 ℓ -维标量向量空间，因此两者之间的坐标变换是一个 ℓ -维线性同构，必可表示为某个可逆矩阵 U 。又因为没有使用 slot algebra 的内部 d -维结构，所以无需额外处理局部基与统一基的差异。 $\square$

5.2.2 为什么这时 U 可以理解成 CRT/Vandermonde 矩阵

5.2.2.1 先回顾子环结构

上一章已经说明，sparse packed 明文天然落在子环

$$R'_{p^e} \cong \mathbb{Z}_{p^e}[Y]/(Y^\ell + 1)$$

(这里写最典型、最直观的情形；更一般的写法是 $Y^{N'} + 1$)。

因此任意 sparse packed 明文都可唯一写成

$$q(Y) = a_0 + a_1Y + \cdots + a_{\ell-1}Y^{\ell-1}.$$

这里：

- $q(Y)$ 是同一个明文在 coefficient view 下的“外层多项式”；
- 系数向量

$$(a_0, \dots, a_{\ell-1})$$

就是 coefficient 表示。

5.2.2.2 slot 值就是在若干点上的取值

记第 i 个 slot 对应的点为

$$y_i.$$

在 BGV/BFV 的 sparse 子环里，这些点通常就是

$$y_i = \zeta_{m,i}^c,$$

其中 $c = d$ 或 $d/2$ ，取决于具体情形。

于是第 i 个 slot 的值就是

$$q(y_i) = a_0 + a_1y_i + a_2y_i^2 + \cdots + a_{\ell-1}y_i^{\ell-1}.$$

因此，如果写成矩阵形式，就得到

$$\begin{bmatrix} q(y_0) \\ q(y_1) \\ \vdots \\ q(y_{\ell-1}) \end{bmatrix} = \begin{bmatrix} 1 & y_0 & y_0^2 & \cdots & y_0^{\ell-1} \\ 1 & y_1 & y_1^2 & \cdots & y_1^{\ell-1} \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 1 & y_{\ell-1} & y_{\ell-1}^2 & \cdots & y_{\ell-1}^{\ell-1} \end{bmatrix} \begin{bmatrix} a_0 \\ a_1 \\ \vdots \\ a_{\ell-1} \end{bmatrix}.$$

5.2.2.3 这就是 Vandermonde / CRT 矩阵

上式中间那个矩阵，正是标准的 Vandermonde 型矩阵。而从代数角度，它正是“把一个多项式送到若干个 CRT 分量上的取值”的矩阵，因此也可视为该子环上的 CRT 矩阵。

定理 5.5. 设

$$q(Y) = a_0 + a_1Y + \cdots + a_{\ell-1}Y^{\ell-1} \in \mathbb{Z}_{p^e}[Y]/(Y^\ell + 1),$$

并设 $y_0, \dots, y_{\ell-1}$ 是该环对应的 ℓ 个 slot 点，且两两不同。则

$$(a_0, \dots, a_{\ell-1})^T \mapsto (q(y_0), \dots, q(y_{\ell-1}))^T$$

由一个 Vandermonde 型可逆矩阵表示。

证明. 定义矩阵

$$U = (U_{ik})_{0 \leq i, k < \ell}, \quad U_{ik} = y_i^k.$$

则

$$(Ua)_i = \sum_{k=0}^{\ell-1} y_i^k a_k = q(y_i),$$

故 U 正是所求矩阵。

只需证明 U 可逆。若 $Ua = 0$ ，则

$$q(y_i) = 0, \quad i = 0, \dots, \ell - 1.$$

也就是说，多项式 $q(Y)$ 在 ℓ 个两两不同的点上全为零。但 $\deg q \leq \ell - 1$ ，因此只有

$$q(Y) \equiv 0,$$

也即

$$a_0 = \cdots = a_{\ell-1} = 0.$$

故 U 核为零，从而可逆。 □

直观解释 5.6. 这说明 sparse packed 情况下的 CoeffToSlot / SlotToCoeff，其实就是最普通的：

“多项式系数 $\leftrightarrow$ 在若干点上的值”

因此它长得像 CRT，又长得像 Vandermonde，还长得像 FFT/DFT 矩阵，这三种说法在这里并不矛盾。

5.2.2.4 为什么会长成 $\zeta_{m,i}^{ck}$ 的样子

若

$$y_i = \zeta_{m,i}^c,$$

则

$$U_{ik} = y_i^k = (\zeta_{m,i}^c)^k = \zeta_{m,i}^{ck}.$$

这就是文献中常见的那种“矩阵条目是根的幂”的来源。

特别地，在最常用的 sparse packed 情形中 $c = d$ ，于是

$$U_{ik} = \zeta_{m,i}^{dk}.$$

5.3 fully packed 下的 CoeffToSlot / SlotToCoeff

5.3.1 局部幂基与统一幂基

5.3.1.1 fully packed 的起点长什么样

在 fully packed 情形下，第 i 个 slot 中不是一个标量，而是一个一般的 E_i -元素：

$$m_i = \sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j.$$

因此整个 slot 向量写成

$$\vec{m} = \left(\sum_{j=0}^{d-1} m_{0,j} \zeta_{m,0}^j, \sum_{j=0}^{d-1} m_{1,j} \zeta_{m,1}^j, \dots, \sum_{j=0}^{d-1} m_{\ell-1,j} \zeta_{m,\ell-1}^j \right).$$

这里每个 slot 使用的都是自己的局部幂基

$$1, \zeta_{m,i}, \zeta_{m,i}^2, \dots, \zeta_{m,i}^{d-1}.$$

5.3.1.2 为什么局部幂基会造成麻烦

如果我们想用一個统一的矩阵把不同 slots 混起来，那么最自然的要求是：

所有 slot 分量应该先写在同一种内部坐标系里。

但 fully packed 时，第 0 个 slot 用的是 $\zeta_{m,0}$ 的幂基，第 1 个 slot 用的是 $\zeta_{m,1}$ 的幂基，……这意味着你不能直接把“第 k 个 slot 里的系数”与“第 i 行矩阵的权重”做统一线性代数运算，因为它们不在同一套内部坐标里。

5.3.1.3 统一幂基的引入

为了解决这个问题，固定一份标准 slot algebra E ，及其统一幂基

$$1, \zeta_m, \zeta_m^2, \dots, \zeta_m^{d-1}.$$

然后用一个 slot-wise 线性变换 M 把每个局部表达

$$\sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j$$

改写成统一幂基表达

$$\sum_{j=0}^{d-1} m_{i,j} \zeta_m^j.$$

这一步并不改变系数 $m_{i,j}$ ，改变的只是“同一个 slot 元素在不同基下的坐标表达”。

5.3.2 M 、 U 、 M^{-1} 的分工

这三个矩阵各自负责不同层次的问题，必须明确区分。

5.3.2.1 M 的任务：统一内部基

M 只做一件事：

$$\sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j \mapsto \sum_{j=0}^{d-1} m_{i,j} \zeta_m^j.$$

因此 M 是一个逐 slot 的基变换，它不混不同 slots，只统一它们内部的坐标系。

5.3.2.2 U 的任务：只处理外层 slot 混合

一旦所有 slot 都写成统一幂基形式：

$$v_k = \sum_{j=0}^{d-1} m_{k,j} \zeta_m^j,$$

此时才能把它们当作“同一种类型”的向量分量。接下来矩阵 U 负责的是：

只处理第 k 个 slot 应当如何贡献到第 i 个目标位置。

也就是说， U 只管外层 slot 索引，不管内部幂基的定义来源。

5.3.2.3 M^{-1} 的任务：把统一基送回局部基

做完 U 以后，表达仍然处在统一幂基

$$1, \zeta_m, \dots, \zeta_m^{d-1}$$

下。若最终目标是回到每个 slot 自己的局部幂基表达，就必须再施加一次 M^{-1} 。

5.3.2.4 整体结构

因此 fully packed 情况下，完整的 SlotToCoeff 不是单独一个 U ，而是

$$M^{-1}UM.$$

相应地，完整的 CoeffToSlot 则是

$$M^{-1}U^{-1}M.$$

注 5.7. 这并不意味着 $M^{-1}UM$ 是某个“神秘复合”。它只是三件很自然的事按顺序叠加：

- (1) 先统一内部基；
- (2) 再做外层 slot 混合；
- (3) 最后恢复局部基。

5.3.3 为什么不能直接乘 U

这是 fully packed 中最常见的困惑。

5.3.3.1 直接乘 U 的想法

若忽略基底问题，你可能会想直接做

$$U\vec{m}.$$

但此时 $\vec{m}$ 的第 k 个分量长成

$$\sum_{j=0}^{d-1} m_{k,j} \zeta_{m,k}^j.$$

也就是说，不同分量属于“不同局部幂基表示”的 E_k 。

5.3.3.2 哪里出了问题

假设为了简单起见取 $\ell = 2, d = 2$ ，则

$$\vec{m} = \begin{bmatrix} a_0 + a_1 \zeta_{m,0} \\ b_0 + b_1 \zeta_{m,1} \end{bmatrix}.$$

若直接乘

$$U = \begin{bmatrix} 1 & \zeta_{m,0}^2 \\ 1 & \zeta_{m,1}^2 \end{bmatrix},$$

第一行会产生

$$(a_0 + a_1 \zeta_{m,0}) + \zeta_{m,0}^2 (b_0 + b_1 \zeta_{m,1}) = a_0 + a_1 \zeta_{m,0} + b_0 \zeta_{m,0}^2 + b_1 \zeta_{m,0}^2 \zeta_{m,1}.$$

最后一项

$$\zeta_{m,0}^2 \zeta_{m,1}$$

同时混入了两个不同 slot 的局部根，因此你无法把结果整齐地写回“第 0 个 slot 的幂基展开”。

5.3.3.3 为什么先做 M 就没问题

若先做 M ，则向量变成

$$\vec{m}' = \begin{bmatrix} a_0 + a_1 \zeta_m \\ b_0 + b_1 \zeta_m \end{bmatrix}.$$

现在每个分量都写在统一幂基下。于是乘 U 后虽然会出现

$$\zeta_m^j \zeta_{m,i}^{dk},$$

但这里只有“统一基中的 ζ_m^j ”与“矩阵权重 $\zeta_{m,i}^{dk}$ ”的乘积，不再混入另一套局部基。最后施加 M^{-1} 时，再把

$$\zeta_m^j \mapsto \zeta_{m,i}^j,$$

于是就能合并成

$$\zeta_{m,i}^{j+dk}.$$

定理 5.8. 在 *fully packed* 情况下，若不先做局部幂基到统一幂基的基变换，则外层 slot 混合矩阵 U 不能一般性地单独实现合法的 *slot-to-coefficient* 变换。

证明. 上述 $\ell = 2, d = 2$ 的具体例子已经给出反例：直接乘 U 会产生同时包含 $\zeta_{m,0}$ 与 $\zeta_{m,1}$ 的混合项，无法整理成单一局部幂基下的表达。而合法的 *slot-to-coefficient* 变换必须在最终得到一个属于正确环表示的结果，因此必须先统一内部基。□

直观解释 5.9. U 不是“坏矩阵”，它只是只负责外层的那一部分。*fully packed* 比 *sparse packed* 多出来的困难，正是每个 slot 内部还带着一套自己的基底坐标。

5.4 unpack / repack 的必要性

5.4.1 为什么 *fully packed* 时要 unpack

5.4.1.1 digit extraction 想处理的对象是什么

在 BGV bootstrapping 中，digit extraction 要处理的是类似

$$u_i = m_i + p^e r_i$$

这样的标量关系。例如在 base- p 展开中，我们想提取某个整数的低若干位 digit。

5.4.1.2 fully packed 下每个 slot 不是标量

但是 fully packed 时，一个 slot 的值是

$$u_i = \sum_{j=0}^{d-1} u_{i,j} \zeta_{m,i}^j,$$

也就是一个 E_i -元素，而不是一个普通的 $\mathbb{Z}_{p^e}$ -标量。

因此，如果后续的非线性操作设计成“对每个标量做某个函数”，那么它不能直接作用于一般 E_i -元素。

5.4.1.3 这就是为什么要 unpack

unpack 的作用是：

把“每个 slot 里的 d -维小向量”拆成若干个只含标量槽的密文。

拆完以后，每个新密文都只保留原来 fully packed 结构中的一部分内部坐标，从而使后续 digit extraction 真正面对的是标量而不是 E_i -元素。

5.4.2 unpack 之后发生了什么

5.4.2.1 直观图像

若原来一个 slot 里装的是

$$(u_{i,0}, u_{i,1}, \dots, u_{i,d-1}),$$

那么 unpack 之后，你不再保留“一个 slot 里同时装这 d 个数”，而是把它们拆到多个新的密文里：

第一个密文保留 $u_{i,0}$ ， 第二个密文保留 $u_{i,1}$ ， ...

于是每个新密文在 slot 视角下都变成 sparse packed。

5.4.2.2 even/odd splitting 的基本思想

最常见的 unpack 思想是递归偶奇分裂。如果某个 slot 在统一基下写成

$$f(\zeta_m) = a_0 + a_1 \zeta_m + a_2 \zeta_m^2 + \dots + a_{d-1} \zeta_m^{d-1},$$

则对某个适当 automorphism $\sigma^{d/2}$ 有

$$\sigma^{d/2}(\zeta_m) = -\zeta_m.$$

因此：

$$f(\zeta_m) + f(-\zeta_m)$$

会消掉所有奇次项，只留下偶次部分；

$$f(\zeta_m) - f(-\zeta_m)$$

会消掉所有偶次项，只留下奇次部分。

这样就能把一个 d -维向量拆成两个 $d/2$ -维向量。重复 $\log_2 d$ 次后，就能拆成标量。

5.4.2.3 为什么 unpack 代价很高

因为它通常需要：

- 多次 automorphism；
- 多次 mask；
- 多次 plaintext multiplication；
- 以及中间层的重新组织。

这也是 fully packed bootstrap 显著比 sparse packed 更复杂的根源之一。

5.4.3 repack 在哪里出现

5.4.3.1 为什么 unpack 后不能直接结束

经过 unpack 和逐槽 digit extraction 后，你往往得到的是很多个 sparse packed 密文，每个密文只承载原 fully packed 数据的一部分。

但最终为了继续维持 fully packed 吞吐量，你不能永远停留在“拆散状态”。必须把它们重新组合成一个 fully packed 密文。

5.4.3.2 repack 的作用

repack 就是 unpack 的逆过程：

把若干个 sparse packed 的结果，重新合并成一个 fully packed 的 slot 结构。

通常它会在：

- digit extraction 之后；
- SlotToCoeff 之前；
- 或与某些线性变换一起融合。

5.4.3.3 为什么 repack 也是非平凡步骤

因为它不是简单地“把几个向量拼起来”，而是要保证：

- 各个部分回到正确的内部幂基位置；
- outer slot index 与 inner basis index 重新对齐；
- 最终结果仍然落在合法的 R_{p^e} 表示里。

因此 repack 与 unpack 一样，通常也是由一系列 automorphism、mask 和线性组合构成的。

5.5 本章小结

本章围绕 BGV/BFV 中的 CoeffToSlot 与 SlotToCoeff 建立了完整的表示变换框架。主要结论如下：

- (1) **定义层面：** CoeffToSlot 是从 coefficient view 到 slot view 的变换；SlotToCoeff 是其逆变换。
- (2) **流程层面：** CoeffToSlot 用于把 noisy plaintext 打开到可逐槽处理的表示；SlotToCoeff 用于把逐槽处理完的结果重新装回大环。
- (3) **sparse packed 情况：** 由于每个 slot 中只装一个标量，不存在内部幂基差异，因此完整变换退化成单个矩阵 U （及其逆）即可。
- (4) **U 的本质：** 在 sparse 子环中， U 就是一个 CRT/Vandermonde 型求值矩阵，其条目形如

$$U_{ik} = y_i^k = \zeta_{m,i}^{ck}.$$

- (5) **fully packed 情况：** 每个 slot 都是一个 d -维 E_i -元素，因此必须先用 M 把局部幂基统一，再用 U 做外层 slot 混合，最后用 M^{-1} 恢复局部基。完整变换因此长成

$$M^{-1}UM$$

或

$$M^{-1}U^{-1}M.$$

- (6) **为什么不能直接乘 U ：** 因为未统一基时，不同 slots 的局部根会在乘法中混在一起，导致结果无法整理成合法的单一局部幂基表示。
- (7) **unpack / repack 的必要性：** digit extraction 等后续非线性操作通常要求输入是标量槽，因此 fully packed 时必须先 unpack，把每个 slot 内的 d -维结构拆成标量槽流；处理完成后再通过 repack 恢复 fully packed 结构。

下一章将正式进入 *BGV Bootstrapping* 的完整流程，把这里的表示变换放回总体上下文中，依次解释：

- 模数提升；
- 同态线性解密；
- `CoeffToSlot`；
- `unpack`；
- digit extraction / modular reduction；
- `repack`；
- `SlotToCoeff`；
- 输出新密文。

BGV Bootstrapping 的完整流程

6.1 总体框图

本章开始，我们不再只停留在代数结构与表示变换本身，而是把这些内容全部放回到一个完整的 BGV bootstrapping 过程里，回答最关键的问题：

一个快要失效的 BGV 密文，究竟是怎样一步一步被刷新成一个同消息、低噪声的新密文的？

在抽象层面，BGV bootstrapping 可以概括为：

旧密文 $\rightarrow$ 模数提升 $\rightarrow$ 同态线性解密 $\rightarrow$ CoeffToSlot $\rightarrow$ unpack（如适用） $\rightarrow$ digit extraction

其中：

- “旧密文”中已经包含了同一个消息，但噪声接近可解密边界；
- “模数提升”给后续的深电路运算腾出模数空间；
- “同态线性解密”把密文中的消息拉成一个显式的 noisy plaintext；
- “CoeffToSlot / SlotToCoeff”负责在 coefficient view 与 slot view 之间切换；
- “unpack / repack”只在 fully packed 情况下需要；
- “digit extraction / modular reduction”是真正清除 p^e -倍噪声部分的核心非线性步骤；
- “新密文”加密的消息与旧密文相同，但噪声重新回到较低水平。

注 6.1. 在不同论文与实现中，上述步骤可能会被合并、重排或部分融合。例如：

- CoeffToSlot 可能与某些 trace/selection 步骤融合；
- repack 可能与 SlotToCoeff 前后的线性变换交织实现；
- digit extraction 可能用不同的 base decomposition 方式呈现。

但从原理上看，这条链条是最清晰、最稳健的理解方式。

6.2 模数提升 (modulus raising / lifting)

6.2.1 为什么 bootstrapping 之前必须先“变大模数”

设当前旧密文活在

$$R_q^2,$$

其中 q 已经是模数链中较靠下的一层。此时虽然它仍然加密着正确的消息，但若我们立刻在它上面评估一条很深的 bootstrapping 电路，就会遇到两个问题：

- (1) 当前密文模数太小，无法承载足够多的中间误差；
- (2) 后续步骤中需要大量 automorphism、key switching、plaintext multiplication 与 digit extraction 电路，这些都需要额外的模数预算。

因此 bootstrapping 的第一步通常是：

把旧密文从当前模数 q 提升到一个更大的 bootstrapping 模数 Q 。

6.2.2 模数提升的抽象形式

抽象上，可以把这一步看成从

$$\mathbf{c} \in R_q^2$$

得到

$$\mathbf{c}^\uparrow \in R_Q^2, \quad Q \gg q.$$

它满足：

- 加密的消息不变；
- 当前噪声不会神奇变小；
- 但后续运算可使用的大模数空间增大了。

注 6.2. 模数提升不是 bootstrapping 的“刷新步骤”，而是“为刷新步骤准备空间”的步骤。它本身不会恢复消息质量，只是防止你在真正做同态解密之前就先把模数链耗尽。

6.2.3 为什么模数提升在 BGV 中是自然的

BGV 的消息位于

$$R_{p^e},$$

而密文计算位于

$$R_q \text{ 或 } R_Q.$$

消息与密文模数是分离的，因此只要提升后的密文仍对应同一条解密关系，消息并不会改变。

也就是说，BGV 中的模数提升更像是：

给同一条 noisy plaintext 关系换一个更宽裕的“工作平台”。

6.2.4 模数提升不等于噪声刷新

这点必须单独强调。

若旧密文解密关系可写成

$$u = m + p^e r,$$

那么模数提升之后，本质上只是把这条关系搬到了更大的模数空间里。也就是说，仍然是

$$u^\uparrow = m + p^e r'$$

这样的关系，只不过它现在活在一个更大的模数环中。消息 m 没变，噪声项也没有被清除，只是后面有能力继续计算一条更深的电路了。

直观解释 6.3. 模数提升就像你准备写一篇很长的证明，发现原来的纸太小了，于是把内容誊写到更大的纸上。证明本身并没有变短，只是你有空间把它完整写出来。

6.3 同态线性解密

6.3.1 从密文得到加密的 noisy plaintext

6.3.1.1 普通解密的线性部分

在最基本的二元 BGV 密文中，解密会先计算

$$u = c_0 + c_1 s.$$

若是更高阶的密文表示，则更一般地写成

$$u = \sum_i c_i s^i.$$

这一部分叫“线性解密”或“解密的线性部分”，因为秘密钥只以线性或低次幂的形式进入表达式。

6.3.1.2 bootstrapping 中不能直接知道 s

在服务端执行 bootstrapping 时，不允许直接持有明文秘密钥 s 。因此我们不能真的“明文算出 u ”，只能借助 bootstrapping key 同态地评估这条表达式。

也就是说，bootstrapping 要做的是：

$$\text{Enc}(\mathbf{c}) \rightsquigarrow \text{Enc}\left(\sum_i c_i s^i\right).$$

6.3.1.3 同态线性解密的结果是什么

BGV 的核心关系是：对正确构造的密文，总存在

$$u = \sum_i c_i s^i = m + p^e r$$

(在适当模数意义下成立)，其中：

- $m \in R_{p^e}$ 是真正明文；
- r 汇总旧噪声与中间误差；
- u 是 noisy plaintext。

因此，同态线性解密之后，得到的不是“直接的新密文消息”，而是：

一个加密的 noisy plaintext: $\text{Enc}(m + p^e r)$ 。

6.3.2 为什么这一步还没有刷新噪声

6.3.2.1 消息已经显式，但噪声仍在

注意，经过同态线性解密后，我们已经成功把旧密文中“隐含在 RLWE 结构里的消息”抽成了一个显式环元素

$$u = m + p^e r.$$

这当然非常重要，因为后面 digit extraction 的目标对象正是这个 u 。

但这一步还远远不能算刷新完成，因为：

$$u \neq m.$$

真正的问题恰恰是：它还带着一个 p^e -倍噪声项。

6.3.2.2 为什么单靠线性部分无法清噪

从线性代数角度，若你只能做线性变换，则从

$$u = m + p^e r$$

中恢复 m 是不可能的，因为 r 是未知的、随消息与噪声变化而变化。恢复

$$u \bmod p^e$$

是一个本质上的非线性任务。

因此：

同态线性解密只是把问题暴露出来，并没有解决问题。

6.3.2.3 这一步与后续步骤的逻辑衔接

所以 bootstrapping 的真正结构是：

$$\text{密文} \xrightarrow{\text{线性解密}} \text{Enc}(m + p^e r) \xrightarrow{\text{非线性清理}} \text{Enc}(m).$$

而 CoeffToSlot、unpack、digit extraction、repack、SlotToCoeff 这些后续步骤，全部都是为这个“非线性清理”服务的。

6.4 CoeffToSlot

6.4.1 为什么需要先开到 slots

6.4.1.1 我们此时手里有什么

经过同态线性解密之后，我们手里有的是一个加密的环元素

$$u(X) = m(X) + p^e r(X) \in R_Q.$$

它最自然的解释是 coefficient view 下的 noisy plaintext。

6.4.1.2 为什么不能直接对 $u(X)$ 做 digit extraction

digit extraction 想要处理的是“一个标量的低位”，例如：

$$u_i = m_i + p^e r_i \mapsto m_i.$$

但现在我们面对的是一个大环元素 $u(X)$ ，不是一个单独标量。

若你在系数表示下直接施加某个环多项式函数 f ，算出来的是

$$f(u(X)),$$

它是环上的函数值，不会逐系数独立作用。

因此若希望后面做“逐个值的低位提取”，就必须先把 $u(X)$ 展开成若干个可并行处理的分量，也就是 slots：

$$u(X) \xrightarrow{\text{CoeffToSlot}} (u_1, \dots, u_\ell).$$

6.4.1.3 CRT 视角下为什么这是自然的

因为我们已经知道

$$R_{p^e} \cong E^\ell.$$

因此把 $u(X)$ 送到 slots，并不是做了什么“外来技巧”，而只是利用 CRT 把同一个环元素改写成若干个分量的表示。

这一步之所以对 bootstrapping 特别关键，是因为：

很多后续非线性步骤在 slot 表示下可以逐槽并行地定义。

6.4.2 如果不做会发生什么

6.4.2.1 非线性会在环里纠缠整个多项式

如果你不做 `CoeffToSlot`, 而试图直接在 `coefficient view` 下对 $u(X)$ 做 `digit extraction`, 那么你面对的是:

- 一个大环元素;
- 环乘法会混合各个系数;
- `digit extraction` 这种标量函数无法逐槽分离。

换句话说, 后续的“去掉 p^e 倍噪声”将不再是

$$u_i \mapsto u_i \bmod p^e$$

这样的单变量问题, 而会退化成整个环上的复杂非线性问题, 代价和复杂度都难以接受。

6.4.2.2 为什么 `packed bootstrapping` 都绕不开它

现代 `packed bootstrapping` 的根本思想就是:

- (1) 先把 `noisy plaintext` 摊成很多个并行槽;
- (2) 再对每个槽独立做所需的非线性处理;
- (3) 最后把结果重新装回环里。

因此, 如果不做 `CoeffToSlot`, 就等于放弃了整个 `SIMD` 思路。

直观解释 6.4. `CoeffToSlot` 的作用可以概括成一句话:

把“一个难处理的大对象”, 变成“很多个容易逐个处理的小对象”。

6.5 unpack (如适用)

6.5.1 什么时候需要 `unpack`

这取决于当前采用的是哪种 `packing` 模式。

sparse packed. 若 `CoeffToSlot` 后, 每个 `slot` 里本来就只装一个标量

$$u_i \in \mathbb{Z}_{p^e},$$

则后续 `digit extraction` 可以直接逐槽进行, 不需要 `unpack`。

fully packed. 若 CoeffToSlot 后, 每个 slot 里装的是一般的 E_i -元素

$$u_i = \sum_{j=0}^{d-1} u_{i,j} \zeta_{m,i}^j,$$

则 digit extraction 无法直接作用在这样的 E_i -元素上。必须先把每个 slot 内部的 d -维结构拆开, 变成多个 sparse/标量槽流, 这一步就叫 unpack。

6.5.2 unpack 的目标

unpack 的最终目标是把

$$\left(\sum_{j=0}^{d-1} u_{0,j} \zeta_{m,0}^j, \dots, \sum_{j=0}^{d-1} u_{\ell-1,j} \zeta_{m,\ell-1}^j \right)$$

转化为若干个新密文, 使得每个新密文里每个 slot 只含一个标量系数 $u_{i,j}$ 。

也就是说, unpack 把“slot 内部的小向量”拆散。

6.5.3 unpack 的递归结构

最常见的 unpack 原理是 even/odd splitting。

设统一基下一个 slot 元素写成

$$f(\zeta_m) = a_0 + a_1 \zeta_m + \dots + a_{d-1} \zeta_m^{d-1}.$$

若存在某个 automorphism $\sigma^{d/2}$ 使

$$\sigma^{d/2}(\zeta_m) = -\zeta_m,$$

则

$$f(\zeta_m) + f(-\zeta_m)$$

只保留偶数次项,

$$f(\zeta_m) - f(-\zeta_m)$$

只保留奇数次项。

于是一个 d -维结构被拆成两个 $d/2$ -维结构。继续递归, 就能在 $\log_2 d$ 层后拆成标量。

6.5.4 为什么 bootstrapping 中 unpack 很昂贵

因为它并不是一个“简单重排”, 而是需要:

- automorphism;
- key switching;

- mask;
- 若干层次的线性组合。

这也是 fully packed bootstrapping 的主要成本来源之一。

6.6 digit extraction / modular reduction

6.6.1 这一步要解决什么问题

经过同态线性解密，再经过 CoeffToSlot（以及必要时的 unpack）之后，我们终于把问题压缩成了很多个标量关系：

$$u_i = m_i + p^e r_i.$$

这里：

- m_i 是真正想保留的消息；
- $p^e r_i$ 是我们想去掉的“高位垃圾”。

因此 digit extraction / modular reduction 的核心目标就是：

$$u_i \mapsto u_i \bmod p^e = m_i.$$

这一步才是 BGV bootstrapping 真正“刷新噪声”的核心。

6.6.2 为什么可以逐槽做

6.6.2.1 来自 CRT 的逐槽关系

在大环里，noisy plaintext 满足

$$u = m + p^e r.$$

对 CRT 分解同时作用，得到

$$u_i = m_i + p^e r_i$$

对所有 slots 成立。

因此，“ p^e 的倍数垃圾”这个关系会逐槽传递。这说明：

$$u \bmod p^e = m$$

在大环里成立，等价于

$$u_i \bmod p^e = m_i$$

在每个 slot 里成立。

6.6.2 为什么这不是因为 slot 是系数

这里要特别强调：

逐槽模 p^e 成立，不是因为 slot 像多项式系数，而是因为 CRT 给的是环同构。

换句话说，合法性来自：

$$R_{p^e} \cong E^\ell$$

这条代数事实，而不是来自某种“slot 就是数组元素”的直觉。

6.6.3 与 CRT/slot 的关系

digit extraction 之所以总和 CRT/slots 绑在一起出现，是因为它需要：

(1) 先把 noisy plaintext 写成

$$u = m + p^e r$$

的形式；

(2) 再利用 CRT 把这个关系送到每个 slot；

(3) 然后把每个 slot 当成一个小标量问题来处理。

所以 CRT 在这里的作用不是“完成 digit extraction”，而是：

把 digit extraction 变成一堆互相并行的小问题。

6.6.4 digit extraction 的基本思想

最常见的理解方式是进位制展开。

设我们按 base- p 展开一个整数 u_i ：

$$u_i = d_0 + d_1 p + d_2 p^2 + \cdots + d_L p^L, \quad 0 \leq d_t < p.$$

则

$$u_i \bmod p^e = d_0 + d_1 p + \cdots + d_{e-1} p^{e-1}.$$

也就是说，digit extraction 的本质就是：

抽出低 e 个 base- p digits，丢掉更高位。

更一般地，也可以按 base- p^k 分块做 digit extraction，把低位按 block 抽出后再重组。

6.6.5 为什么它是非线性核心

因为“取低位 digit”“对 p^e 取模”都不是线性运算。这一步无法只靠加法和固定矩阵完成，因此必须评估某种非线性电路或函数。

这就是为什么 bootstrapping 的成本核心通常不在同态线性解密，而在 digit extraction / modular reduction。

6.7 repack (如适用)

6.7.1 为什么 digit extraction 之后还不能直接结束

若前面做了 unpack，那么 digit extraction 通常是在多个“拆开的标量槽流”上进行的。这时虽然每个标量都已经恢复干净，但整体数据结构仍是分散的，不是一个 fully packed 明文。

因此还必须把这些 sparse/拆开的结果重新合成一个 fully packed 的结构，这一步就是 repack。

6.7.2 repack 的作用

repack 是 unpack 的逆过程。它负责：

- 把拆开的内部坐标重新并入单个 slot；
- 恢复每个 slot 的 d -维结构；
- 使结果再次适合后续的 fully packed 表示与高吞吐运算。

6.7.3 为什么 repack 常与后续线性变换融合

在实际实现中，repack 往往不会作为一个完全独立的大步骤存在，而常与：

- SlotToCoeff；
- 某些 trace；
- basis change；
- key switching

进行融合，以节省层数和 automorphism 开销。

6.8 SlotToCoeff

6.8.1 它在流程中的位置

经过 digit extraction（以及必要时的 repack）之后，我们已经得到了“每个 slot 都已修复”的结果：

$$(v_1, \dots, v_\ell).$$

但这仍然只是 slot view 下的描述。最终系统需要的是一个新的环元素密文，因此必须再做

$$(v_1, \dots, v_\ell) \xrightarrow{\text{SlotToCoeff}} v(X).$$

6.8.2 sparse packed 与 fully packed 的形式差别

sparse packed. 此时 SlotToCoeff 就是 sparse 子环上的 Vandermonde/CRT 逆变换，通常由 U 或 U^{-1} 实现，取决于你采用的矩阵约定。

fully packed. 此时完整变换不是单独的 U ，而是

$$M^{-1}UM$$

或相应逆形式，因为还需要处理局部幂基与统一幂基之间的切换。

6.8.3 为什么 bootstrapping 必须回到 coefficient view

因为：

- 密文本体始终是 ring ciphertext；
- 后续同态计算仍将在大环里继续进行；
- 若停留在 slot 表示，就只是“解释上已恢复”，并没有得到一个可继续使用的标准密文表示。

所以 SlotToCoeff 不是附加步骤，而是 bootstrapping 闭环的必要一半。

6.9 输出新密文与后处理

6.9.1 key switching / relinearization

6.9.1.1 为什么需要 key switching

在 bootstrapping 过程中，大量 automorphism 与辅助变换会把密文临时送到不同的秘密钥表示下。为了让最终输出回到原始系统可继续使用的标准密钥下，通常需要做 key

switching。

6.9.1.2 为什么可能需要 relinearization

若中间步骤生成了更高阶的密文表示（例如乘法、部分乘积堆叠等），则还要通过 relinearization 把它压回标准的短密文形式，例如二元密文。

6.9.2 回到正常计算模数层

6.9.2.1 为什么不能一直留在 bootstrapping 模数 Q

在 bootstrapping 开始时，我们特意把密文提升到了较大的 bootstrapping 模数 Q 。但后续普通同态计算通常不希望一直在这么大的模数上运行，因为：

- 开销更大；
- 参数更重；
- 不利于后续层级管理。

因此 bootstrapping 输出后，通常会把结果切回一个适合正常运算的模数层。

6.9.2.2 这一步的意义

这一步意味着：

bootstrapping 不是“把密文永久搬到另一个世界”，而是“借助一个更大的工作模数完成刷新，再带着

6.10 full bootstrap 与 thin bootstrap 的区别

6.10.1 full bootstrap

所谓 full bootstrap，是指输出仍保持 fully packed 结构，即尽可能保住全部 slot 容量与内部打包能力。

其特点是：

- 吞吐高；
- 需要处理完整的 CoeffToSlot / unpack / repack / SlotToCoeff；
- 通常代价更高。

6.10.2 thin bootstrap

所谓 thin bootstrap，是指输出不一定恢复全部 fully packed 结构，而是压缩到更小子环或更少自由度的表示。

其特点是：

- 代价更低；
- 结构更简单；
- 但 packing 能力与后续并行度下降。

6.10.3 两者的选择逻辑

是否采用 full 还是 thin，通常取决于：

- 应用是否极度依赖满吞吐 SIMD；
- 当前参数下 unpack / repack 成本是否过高；
- 是否可以接受后续用更多密文分担计算；
- 目标是“尽快刷新一次”还是“维持长期高吞吐”。

注 6.5. 因此，full/thin 的区别并不只是一种“实现风格偏好”，而是 bootstrapping 设计中的一个核心架构选择。

6.11 本章小结

本章给出了 BGV bootstrapping 的完整主流程。其逻辑链可以总结为：

旧密文 → 模数提升 → 同态线性解密 → CoeffToSlot → unpack（如适用） → digit extraction /
--

其中最关键的思想是：

- (1) 先把旧密文的消息拉成一个显式 noisy plaintext

$$u = m + p^e r;$$

- (2) 再把它切换到适合逐槽处理的表示；
- (3) 最终对每个槽做“去掉 p^e 倍垃圾”的非线性清理；
- (4) 再重新装回一个合法的 ring ciphertext。

本章也澄清了：

- 模数提升不是刷新噪声，只是为刷新腾出空间；
- 同态线性解密并没有真正清噪，它只是把问题显式化；
- `CoeffToSlot` 是为逐槽非线性服务的；
- `unpack` / `repack` 只在 `fully packed` 情况下必要；
- `digit extraction` 才是真正的噪声刷新核心；
- `SlotToCoeff` 则保证最终结果重新回到标准 `ring ciphertext` 表示。

下一章将单独展开一个 *BGV* 例子：从头到尾走一遍，把本章中的抽象流程具体化，逐步写出：

- `noisy plaintext` 到底长什么样；
- `CoeffToSlot` 之后各个 `slot` 的值是什么；
- `unpack` 如何把 `fully packed` 结构拆成标量槽；
- `digit extraction` 到底如何作用于这些标量；
- `repack` 和 `SlotToCoeff` 如何把结果重新组回去。

BGV 例子：从头到尾走一遍

7.1 参数设定与问题目标

本章不给出真实工业实现中的全部参数细节，而是选取一个足够小、但又能体现 *fully packed* 结构的示例，把前面章节中的所有概念串起来。

7.1.1 一个教学型参数组

我们固定：

$$m = 16, \quad N = \frac{m}{2} = 8.$$

取明文底层素数

$$p = 3,$$

并令

$$e = 2, \quad p^e = 9.$$

因此明文环是

$$R_9 = \mathbb{Z}_9[X]/(X^8 + 1).$$

现在计算

$$d = \text{ord}_{16}(3).$$

因为

$$3^1 \equiv 3 \pmod{16}, \quad 3^2 \equiv 9 \pmod{16}, \quad 3^4 \equiv 81 \equiv 1 \pmod{16},$$

且 $3^1, 3^2 \not\equiv 1 \pmod{16}$ ，故

$$d = 4.$$

于是 slot 数为

$$\ell = \frac{N}{d} = \frac{8}{4} = 2.$$

所以这是一个最小的 *fully packed* 例子之一：

- 总系数维数 $N = 8$;

- slot 数 $\ell = 2$;
- 每个 slot 的内部维数 $d = 4$ 。

7.1.2 这个例子会展示什么

这个例子的目标不是做所有参数最优，而是让你能清楚看见以下层次：

- (1) BGV 解密线性部分如何得到

$$u = m + 9r;$$

- (2) 为什么 CoeffToSlot 后得到的是两个 E -slot;

- (3) 为什么 fully packed 时必须 unpack;

- (4) digit extraction 究竟是在什么对象上做;

- (5) repack 与 SlotToCoeff 如何把结果重新组回一个合法环元素;

- (6) 为什么最终输出的确加密的是同一个明文 m 。

7.1.3 本章使用的两种层次

为了避免符号过载，本章会并行使用两层视角。

抽象层。 始终保持符号

$$u = m + 9r$$

来说明 bootstrapping 的结构。

标量 toy 层。 在 digit extraction 处，会拿一个具体数

$$u_i = 68$$

来演示“为什么模 9 之后会得到正确消息”。

注 7.1. 把抽象环元素和具体整数例子分开，是为了避免误以为所有 slot 值都天然都是普通整数。在 fully packed 里，slot 值首先是 E -元素；只有经过 unpack 之后，才会真的变成适合做 digit extraction 的标量流。

7.2 解密表达式的写法

7.2.1 旧密文的抽象形式

设旧密文为

$$\mathbf{c} = (c_0, c_1) \in R_q^2,$$

其中 q 是 bootstrapping 前某一层密文模数。为简洁起见，我们只写二元密文；若有更多分量，思路完全相同，只是解密表达式会变成 $\sum_i c_i s^i$ 。

记秘密钥为

$$s \in R_q.$$

则普通 BGV 解密线性部分首先形成

$$u = c_0 + c_1 s.$$

7.2.2 BGV 的核心关系

BGV 的关键在于， u 不会直接等于消息 m ，而是具有结构

$$u = m + p^e r.$$

在本例中 $p^e = 9$ ，因此

$$u = m + 9r.$$

这里：

- $m \in R_9$ 是真正的明文；
- r 聚合了原密文噪声与中间误差；
- u 称为 noisy plaintext。

7.2.3 为什么这条公式是 bootstrapping 的起点

若能从加密的 u 中同态恢复

$$u \bmod 9 = m,$$

那么就得到了同一个消息 m 的新密文。

所以 BGV bootstrapping 的主问题可压缩成：

从 $\text{Enc}(m + 9r)$ 同态得到 $\text{Enc}(m)$ 。

7.2.4 提升到 bootstrapping 模数后的写法

实际执行时，我们通常会先把旧密文提升到更大的 bootstrapping 模数 Q 。因此更准确地说，在更大的环 R_Q 中，我们同态求值得到：

$$\text{Enc}_Q(u) = \text{Enc}_Q(m + 9r).$$

模数提升不会改变这条关系的形状，只是让后面的电路有足够的模数预算。

7.2.5 本章的记号约定

从这里开始，我们把“同态线性解密之后得到的加密 noisy plaintext”统一记为

$$\text{Enc}(u), \quad u = m + 9r.$$

不再反复写出下标 Q 。

7.3 noisy plaintext 的结构

7.3.1 coefficient view 下的结构

在 coefficient view 中，

$$u(X) = m(X) + 9r(X)$$

是 R_Q 中的一个环元素。写成系数形式可表示为

$$u(X) = u_0 + u_1X + \cdots + u_7X^7.$$

其中每个 u_t 都是模 Q 的数。

同样，

$$m(X) = m_0 + m_1X + \cdots + m_7X^7$$

是目标明文，而

$$r(X) = r_0 + r_1X + \cdots + r_7X^7$$

是某个误差多项式。

因此

$$u_t = m_t + 9r_t$$

对每个系数位置都成立。

注 7.2. 这一点容易让人误以为“那直接逐系数模 9 不就行了”。问题在于：同态计算发生在环上，若不切换到适合的表示，后续 digit extraction 这类非线性函数不会逐系数独立作用。因此 bootstrapping 中要做的不是“纯逻辑上看系数关系是否成立”，而是“如何同态实现这个关系的恢复”。

7.3.2 slot view 下的结构

经过 CRT 分解后,

$$R_9 \cong E_1 \times E_2.$$

因此同一个 noisy plaintext 也可写成

$$u \longleftrightarrow (u_1, u_2), \quad m \longleftrightarrow (m_1, m_2), \quad r \longleftrightarrow (r_1, r_2).$$

并且关系逐 slot 传递为

$$u_i = m_i + 9r_i, \quad i = 1, 2.$$

7.3.3 在本例中为什么每个 slot 不是标量

因为 $d = 4$, 每个 slot algebra 的维数是 4。所以每个 slot 中的元素一般长成

$$m_i = m_{i,0} + m_{i,1}\zeta_{m,i} + m_{i,2}\zeta_{m,i}^2 + m_{i,3}\zeta_{m,i}^3.$$

同理,

$$u_i = u_{i,0} + u_{i,1}\zeta_{m,i} + u_{i,2}\zeta_{m,i}^2 + u_{i,3}\zeta_{m,i}^3.$$

因此即使已经到了 slot view, 这里也还不是“两个普通整数槽”, 而是“两个四维小环槽”。

7.3.4 为什么这会迫使我们 unpack

digit extraction 想处理的是像

$$\text{某个整数 } \alpha = \beta + 9\gamma$$

这样的标量关系。但当前 fully packed 的 slot 值是一般 E_i -元素, 不是普通标量, 所以还不能直接 digit extract。

这就解释了:

CoeffToSlot

之后为何还要加

unpack.

7.4 CoeffToSlot 的结果长什么样

7.4.1 先写出变换前后的形式

在本例中, CoeffToSlot 把 coefficient view 下的 noisy plaintext

$$u(X) \in R_Q$$

送到 slot view:

$$u(X) \xrightarrow{\text{CoeffToSlot}} (u_1, u_2) \in E_1 \times E_2.$$

更具体地,

$$u_i = m_i + 9r_i, \quad i = 1, 2,$$

且每个 u_i 都可写成

$$u_i = u_{i,0} + u_{i,1}\zeta_{m,i} + u_{i,2}\zeta_{m,i}^2 + u_{i,3}\zeta_{m,i}^3.$$

7.4.2 用统一幂基写法表达

为了方便后续的线性变换, 可以先通过 M 把局部幂基统一成标准幂基。于是同样的两个 slot 也可以写成

$$u'_i = u_{i,0} + u_{i,1}\zeta_m + u_{i,2}\zeta_m^2 + u_{i,3}\zeta_m^3.$$

因此 CoeffToSlot 之后、unpack 之前, 我们手里本质上有的是:

$$(u_{1,0} + u_{1,1}\zeta_m + u_{1,2}\zeta_m^2 + u_{1,3}\zeta_m^3, u_{2,0} + u_{2,1}\zeta_m + u_{2,2}\zeta_m^2 + u_{2,3}\zeta_m^3).$$

7.4.3 这一步在流程中真正得到什么

这一步最重要的不是具体矩阵, 而是语义变化:

我们不再面对一个大环元素 $u(X)$, 而是面对两个 slot, 每个 slot 中装着一个四维对象。

因此:

- ring 上的“全局问题”被拆成了两个并行的小问题;
- 但每个小问题内部仍有 4 个坐标, 还不能直接做标量 digit extraction。

7.4.4 和 sparse packed 的对比

若这是 sparse packed 例子, 则 CoeffToSlot 之后就会直接得到

$$(u_1, u_2), \quad u_i \in \mathbb{Z}_9,$$

然后可以马上进入 digit extraction。而 fully packed 多出来的麻烦正是:

$$u_i \in E_i, \quad \dim_{\mathbb{Z}_9} E_i = 4.$$

7.5 fully packed 时的 unpack

7.5.1 目标：把四维 slot 拆成标量流

在本例中，每个 slot 长成

$$u_i = u_{i,0} + u_{i,1}\zeta_m + u_{i,2}\zeta_m^2 + u_{i,3}\zeta_m^3.$$

我们希望最终把它拆成 4 条标量流：

$$(u_{1,0}, u_{2,0}), \quad (u_{1,1}, u_{2,1}), \quad (u_{1,2}, u_{2,2}), \quad (u_{1,3}, u_{2,3}).$$

也就是说，从 1 个 fully packed 密文，拆成 4 个 sparse packed 密文。

7.5.2 第一层：偶奇分裂

记

$$f(\zeta_m) = a_0 + a_1\zeta_m + a_2\zeta_m^2 + a_3\zeta_m^3.$$

取能满足

$$\sigma^2(\zeta_m) = -\zeta_m$$

的自同构（在本例 $d = 4$ ，所以用 $\sigma^{d/2} = \sigma^2$ ）。则

$$f(-\zeta_m) = a_0 - a_1\zeta_m + a_2\zeta_m^2 - a_3\zeta_m^3.$$

于是：

$$f(\zeta_m) + f(-\zeta_m) = 2(a_0 + a_2\zeta_m^2),$$

$$f(\zeta_m) - f(-\zeta_m) = 2(a_1\zeta_m + a_3\zeta_m^3).$$

由于 2 在 $\mathbb{Z}_9$ 中可逆，我们可以定义

$$f_{\text{even}} := \frac{f(\zeta_m) + f(-\zeta_m)}{2} = a_0 + a_2\zeta_m^2,$$

$$f_{\text{odd}} := \frac{f(\zeta_m) - f(-\zeta_m)}{2\zeta_m} = a_1 + a_3\zeta_m^2.$$

这样，一个四维对象被拆成了两个二维对象。

7.5.3 第二层：继续拆成标量

现在

$$f_{\text{even}} = a_0 + a_2\zeta_m^2, \quad f_{\text{odd}} = a_1 + a_3\zeta_m^2.$$

再对 ζ_m^2 重复同样的 even/odd splitting，就可以分别得到：

$$a_0, a_2 \quad \text{以及} \quad a_1, a_3.$$

因此经过两层递归拆分后，原来

$$a_0 + a_1\zeta_m + a_2\zeta_m^2 + a_3\zeta_m^3$$

可以被拆成 4 个标量。

7.5.4 作用到两个 slots 上的结果

对本例的两个 slots 同时做上述递归，就能从

$$(u_1, u_2)$$

得到四个 sparse packed 密文，其 slot 视角分别近似为：

$$(u_{1,0}, u_{2,0}),$$

$$(u_{1,1}, u_{2,1}),$$

$$(u_{1,2}, u_{2,2}),$$

$$(u_{1,3}, u_{2,3}).$$

每一条现在都只是两个 $\mathbb{Z}_9$ -标量构成的向量。

7.5.5 为什么这样就可以 digit extract 了

因为此时每个槽值都已经是真正的标量，而不再是一般的 E_i -元素。所以后续可以对每条 sparse packed 密文逐槽做 digit extraction，即处理：

$$u_{i,j} = m_{i,j} + 9r_{i,j}.$$

7.6 digit extraction 的逐步操作

7.6.1 核心目标

经过 unpack 以后，我们把 fully packed 的问题压缩成了许多个标量问题。对于任意一个标量槽值，我们都面对同样的关系：

$$u_{i,j} = m_{i,j} + 9r_{i,j}.$$

digit extraction 的目标就是从 $u_{i,j}$ 中恢复

$$m_{i,j} = u_{i,j} \bmod 9.$$

接下来我们分别用两种视角看这件事。

7.6.2 base- p 视角

在本例中，

$$p = 3, \quad e = 2, \quad p^e = 9.$$

因此 base- p 视角就是 base-3 视角。

任意一个标量 u 都可写成

$$u = d_0 + d_1 \cdot 3 + d_2 \cdot 3^2 + d_3 \cdot 3^3 + \cdots, \quad 0 \leq d_t < 3.$$

由于

$$u = m + 9r = m + 3^2 r,$$

所以 m 正好由最低两位 d_0, d_1 决定：

$$m = d_0 + d_1 \cdot 3.$$

7.6.2.1 一个具体数值例子

取某个槽值

$$u = 68.$$

把它写成三进制：

$$68 = 2 \cdot 3^0 + 1 \cdot 3^1 + 1 \cdot 3^2 + 2 \cdot 3^3.$$

也就是

$$68 = (2112)_3.$$

最低两位是

$$(12)_3 = 1 \cdot 3 + 2 = 5.$$

因此

$$68 \bmod 9 = 5.$$

这就是 digit extraction 在本例中的最直接含义：

从三进制展开中只保留最低两位。

7.6.3 base- p^k 视角

有时不按 base- p 一位一位提取，而是按更大的块提取。例如本例中直接按

$$p^k = 9$$

来分块。

则一个标量 u 写成

$$u = D_0 + 9D_1 + 9^2D_2 + \cdots.$$

这时恢复

$$u \bmod 9$$

就等价于直接保留最低 block D_0 。

7.6.3.1 同一个数值例子

仍然取

$$u = 68.$$

按 base-9 展开：

$$68 = 5 + 9 \cdot 7.$$

所以最低 block 就是

$$D_0 = 5.$$

也就是说

$$68 \bmod 9 = 5.$$

7.6.3.2 两种视角的关系

- base-3 视角：保留最低 2 位 digit；
- base-9 视角：保留最低 1 个 block。

数学上是同一件事，只是分块粒度不同。

7.6.4 低位保留与高位丢弃

7.6.4.1 为什么丢掉高位是合法的

设

$$u = m + 9r.$$

由于 $9r$ 一定是 9 的倍数，所以它不会影响模 9 的 residue。因此所有高于 9 的位或 block 都属于“噪声携带的高位部分”，丢掉之后不会改变消息本身。

7.6.4.2 在 toy 例子中的解释

对

$$u = 68 = (2112)_3,$$

高位部分

$$(21)_3 \cdot 9$$

对应的正是“ $9r$ ”那部分。而真正消息只活在低两位

$$(12)_3.$$

7.6.4.3 digit extraction 的本质

因此，不管你用逐位还是逐块说法，digit extraction 的本质都可以统一成：

保留低位（或低块），丢弃高位（或高块）。

在 BGV 中，这等价于做

$$u \longmapsto u \bmod p^e.$$

7.7 repack 与 SlotToCoeff

7.7.1 digit extraction 之后我们拥有什么

经过 unpack 和逐槽 digit extraction 之后，我们不再有原来的 fully packed 密文，而是有若干个 sparse packed 密文，分别承载：

$$(m_{1,0}, m_{2,0}), \quad (m_{1,1}, m_{2,1}), \quad (m_{1,2}, m_{2,2}), \quad (m_{1,3}, m_{2,3}).$$

这些都已经都是“干净的消息系数流”，但它们还没有重新合成为 fully packed slot。

7.7.2 repack: 把四条标量流重新装回两个 fully packed slots

repack 的作用是逆向恢复:

$$\begin{aligned}m_1 &= m_{1,0} + m_{1,1}\zeta_{m,1} + m_{1,2}\zeta_{m,1}^2 + m_{1,3}\zeta_{m,1}^3, \\m_2 &= m_{2,0} + m_{2,1}\zeta_{m,2} + m_{2,2}\zeta_{m,2}^2 + m_{2,3}\zeta_{m,2}^3.\end{aligned}$$

因此 repack 之后, 我们又得到了一个 fully packed slot 向量:

$$(m_1, m_2).$$

7.7.3 SlotToCoeff: 把 slot 向量重新装回环元素

最后一步是

$$(m_1, m_2) \xrightarrow{\text{SlotToCoeff}} m(X) \in R_9.$$

在本例 fully packed 情况下, 这一步不是简单的单个 Vandermonde 矩阵, 而是完整的基变换组合:

$$M^{-1}UM$$

或其逆向版本, 视具体定义方向而定。

其作用是把两个 E -slot 中的消息重新嵌入回一个系数多项式

$$m(X) = m_0 + m_1X + \cdots + m_7X^7.$$

7.7.4 最终输出的对象是什么

这时我们得到的不再是“若干个单独的槽值”, 而是一个新的、合法的、大环中的明文表示。对应到密文层, 这一步完成后得到的是一个新的 BGV 密文:

$$\text{Enc}(m(X)).$$

这就是 bootstrapping 输出。

7.8 为什么最终得到的是同一明文的新密文

7.8.1 逻辑链条

从头到尾, 这个例子里发生的事情可总结为:

$$\begin{aligned}\text{ct} &\xrightarrow{\text{同态线性解密}} \text{Enc}(u), \quad u = m + 9r; \\ \text{Enc}(u) &\xrightarrow{\text{CoeffToSlot}} \text{Enc}((u_1, u_2));\end{aligned}$$

$$\begin{aligned}
& \text{Enc}((u_1, u_2)) \xrightarrow{\text{unpack}} \text{Enc}(u_{i,j}) \text{ 的若干条标量流;} \\
& u_{i,j} = m_{i,j} + 9r_{i,j} \xrightarrow{\text{digit extraction}} m_{i,j}; \\
& \text{Enc}(m_{i,j}) \xrightarrow{\text{repack+SlotToCoeff}} \text{Enc}(m).
\end{aligned}$$

7.8.2 关键事实：除了 digit extraction，其他步骤都不改变消息语义

CoeffToSlot / SlotToCoeff. 它们只是 coordinate change，即表示变换。

unpack / repack. 它们只是重新组织 fully packed 的内部结构。

真正改变“数值内容”的只有 **digit extraction**。而 digit extraction 所做的恰恰是

$$u_{i,j} \mapsto u_{i,j} \bmod 9 = m_{i,j}.$$

因此整个流程唯一的非平凡语义动作，就是把 noisy plaintext 还原成 message。

定理 7.3. 设某次 *BGV bootstrapping* 流程中，同态线性解密得到

$$u = m + p^e r,$$

且后续 *CoeffToSlot*、*unpack*、*repack*、*SlotToCoeff* 都是正确实现的线性同构或其逆，而 *digit extraction* 对每个标量槽都正确计算

$$x \mapsto x \bmod p^e.$$

则 *bootstrapping* 的最终输出密文加密的明文恰为原始消息 m 。

证明。由同态线性解密得到的明文关系是

$$u = m + p^e r.$$

CoeffToSlot、*unpack*、*repack*、*SlotToCoeff* 均是表示变换或重组变换，不改变所表示的抽象环元素，只改变其坐标表达方式。

因此在逐槽标量层面上，总存在表示

$$u_{i,j} = m_{i,j} + p^e r_{i,j}.$$

digit extraction 正确计算

$$u_{i,j} \bmod p^e = m_{i,j},$$

故经过 *digit extraction* 后恢复了所有标量消息分量 $m_{i,j}$ 。再通过 *repack* 和 *SlotToCoeff* 把这些消息分量重新组合为原明文 m 。因此最终输出密文加密的消息即为 m 。□

直观解释 7.4. 这个定理的含义非常简单：

整个 *bootstrapping* 流程中，真正“修复消息”的只有一次：把 $m + p^e r$ 变成 m 。

其余所有步骤，都是为了让这次修复能在密文里、可实现地、低深度地完成。

7.9 这个例子里每一步的代价来源

7.9.1 模数提升的代价

模数提升本身需要：

- 扩展到更大的模数表示；
- 可能伴随若干预处理与规范化；
- 后续所有运算都在更大模数上进行，代价整体抬升。

它的代价主要体现在：后面每一步都变重了。

7.9.2 同态线性解密的代价

这一步的成本来源于：

- 用 bootstrapping key 评估

$$c_0 + c_1 s \quad \text{或} \quad \sum_i c_i s^i;$$

- 相关的 key switching / external product；
- 若密文阶数较高，还可能涉及进一步线性整理。

它通常不是最深的非线性部分，但常常是很重的一步。

7.9.3 CoeffToSlot 的代价

CoeffToSlot 的成本主要来自：

- 大规模线性变换；
- automorphism / rotation；
- key switching；
- plaintext mask multiplication；
- 若用 BSGS，还涉及 baby-step / giant-step 的分组代价。

在大多数 packed bootstrapping 中，这一步本身就是主要瓶颈之一。

7.9.4 unpack 的代价

本例中 $d = 4$ ，所以 unpack 只需做两层偶奇分裂。但一般 d 越大，unpack 的层数大约是

$$\log_2 d.$$

每一层都需要：

- automorphism;
- key switching;
- mask 或简单系数变换;
- 线性组合。

因此 fully packed 时，unpack 往往是相当昂贵的。

7.9.5 digit extraction 的代价

这一步的成本取决于：

- 用 base- p 还是 base- p^k ;
- 每个 digit/block 的提取电路深度;
- 需要提取多少位或多少个 block;
- 是否能把若干中间步骤并行融合。

从 bootstrapping 的本质看，这通常是最核心的非线性成本来源。

7.9.6 repack 与 SlotToCoeff 的代价

repack 和 SlotToCoeff 的成本主要来自：

- 再次进行大规模线性变换;
- 恢复 fully packed 结构;
- 需要 automorphism 与 key switching;
- 若与其它步骤不能融合，会额外增加层数与旋转次数。

7.9.7 总成本的结构性来源

把整个例子压缩后，真正昂贵的来源可以归纳为三类：

- (1) **大线性变换**：CoeffToSlot、SlotToCoeff、unpack、repack；
- (2) **automorphism 相关成本**：rotation + key switching；
- (3) **核心非线性**：digit extraction / modular reduction。

注 7.5. 这也是为什么后续优化工作通常集中在：

- 用 BSGS 降低 rotation 数量；
- 用 FFT-like 稀疏分解降低线性变换成本；
- 用更好的 digit extraction 电路降低非线性深度；
- 用 thin/sparse 路线避免 fully packed 的 unpack/repack 开销。

7.10 本章小结

本章用一个具体的教学型 BGV 例子，把前面所有抽象概念串成了一条完整流水线。核心收获如下：

- (1) 在本例中，

$$m = 16, \quad N = 8, \quad p^e = 9, \quad d = 4, \quad \ell = 2,$$

因此明文有两个 slots，每个 slot 内部是 4-维结构。

- (2) 同态线性解密后得到 noisy plaintext

$$u = m + 9r,$$

这是 bootstrapping 真正要修复的对象。

- (3) CoeffToSlot 之后，问题被拆成两个 E -slot，但这仍然不是标量问题，因此 fully packed 情况下必须进一步 unpack。
- (4) unpack 把每个 4-维 slot 递归拆成四条 sparse packed 标量流。
- (5) digit extraction 的本质是对每个标量槽做

$$u_{i,j} \mapsto u_{i,j} \bmod 9,$$

即“保留低位、丢掉高位”。

(6) `repack` 和 `SlotToCoeff` 再把这些已修复的标量流重新组装成一个合法的明文环元素，从而得到同一消息的新密文。

(7) 这个例子清楚展示了：`bootstrapping` 中昂贵的部分并不只是一个，而是：

- 同态线性解密；
- `CoeffToSlot` / `SlotToCoeff`；
- `unpack` / `repack`；
- `digit extraction`。

下一章将专门把 `unpack` / `repack` 拿出来单独讨论，系统分析：

- 为什么 `fully packed` 一定会遇到它们；
- `even/odd splitting` 到底在做什么；
- 为什么递归分裂最终能把 `E-slot` 拆成标量槽；
- `repack` 又如何作为其逆过程把结构恢复回来。

unpack / repack 专题

8.1 fully packed 为什么一定会遇到它们

8.1.1 问题的根源：slot 里装的不是标量

在前面的章节中，我们已经强调过多次：在 fully packed 情况下，第 i 个 slot 的值不是一个单独的 $\mathbb{Z}_{p^e}$ -标量，而是一个一般的 slot algebra 元素：

$$m_i \in E_i \cong \mathbb{Z}_{p^e}[X]/(F_i(X)).$$

若选取第 i 个 slot 的局部幂基，则它可以写成

$$m_i = m_{i,0} + m_{i,1}\zeta_{m,i} + m_{i,2}\zeta_{m,i}^2 + \cdots + m_{i,d-1}\zeta_{m,i}^{d-1}.$$

这意味着：

一个 fully packed slot 本身就是一个 d -维对象。

而 bootstrapping 中后续的核心非线性（尤其是 digit extraction / modular reduction）通常希望处理的是

$$u = m + p^e r$$

这种标量关系，而不是一般 E_i -元素关系。

8.1.2 为什么 sparse packed 没有这个问题

若是 sparse packed，则每个 slot 中只装一个标量：

$$m_i \in \mathbb{Z}_{p^e} \subseteq E_i.$$

因此 CoeffToSlot 之后，得到的就是一组真正的标量槽：

$$(m_1, \dots, m_\ell).$$

后续 digit extraction 可以直接逐槽进行。

换句话说：

- sparse packed: slot 已经是标量，不必拆；
- fully packed: slot 是 d -维对象，必须先拆。

8.1.3 为什么不能直接对 E_i -元素做 digit extraction

这是最核心的问题。

digit extraction 的目标是从

$$u_i = m_i + p^e r_i$$

中恢复

$$m_i = u_i \bmod p^e.$$

如果这里的 u_i 是普通整数或 $\mathbb{Z}_{p^e}$ -标量，那么“取低位”“按 base- p 展开”“保留最低若干 digits”这些说法都非常直观。

但当 u_i 是一般 E_i -元素时，它并不是一个单独的数，而是一个

d -维 $\mathbb{Z}_{p^e}$ -模元素。

此时如果你说“对它做模 p^e ”或“提取低位 digits”，必须先明确：

- 你是对哪一个坐标做 digit extraction；
- 你是在什么基底下看这些坐标；
- 这些坐标之间是否会因环乘法和 Frobenius 纠缠在一起。

因此，为了让后续非线性成为真正的“标量函数”，必须先把每个 slot 的内部 d -维结构拆开。这就是 unpack 的必要性。

8.1.4 从 bootstrapping 流程看 unpack 的位置

对于 fully packed BGV bootstrapping，核心链条是：

旧密文 $\rightarrow$ 同态线性解密 $\rightarrow$ CoeffToSlot $\rightarrow$ unpack $\rightarrow$ digit extraction $\rightarrow$ repack $\rightarrow$ SlotToCoeff

这里 unpack 的位置并不是偶然的。它恰好位于：

- **CoeffToSlot 之后**：因为只有先到了 slot 视角，才有“每个 slot 内部结构”可言；
- **digit extraction 之前**：因为 digit extraction 想要面对的是标量流，而不是 E -slot。

8.1.5 从维数角度看为什么一定要拆

fully packed 明文总自由度是

$$N = \ell d.$$

而 digit extraction 如果逐槽进行，理想上每次只想看到一个标量自由度。因此你必须把原来“ ℓ 个 slot、每个 slot d 维”的组织方式，拆成“若干条每条只含标量 slot 的流”。

这不是某种实现偏好，而是维数匹配和函数输入类型强制出来的：

后续非线性是标量级的，因此 fully packed 的向量级/小环级结构必须先被摊平。

8.2 even/odd splitting 的思想

8.2.1 拆分问题的抽象形式

设统一幂基下，一个 slot 中的元素写成

$$f(\zeta_m) = a_0 + a_1\zeta_m + a_2\zeta_m^2 + \cdots + a_{d-1}\zeta_m^{d-1}.$$

我们希望把其中的系数

$$a_0, a_1, \dots, a_{d-1}$$

逐步拆出来。

最朴素的想法是：能否先把所有偶数次项与奇数次项分开？答案是肯定的，而其基础正是某个 automorphism 能把 ζ_m 送到 $-\zeta_m$ 。

8.2.2 一个最基本的代数恒等式

设

$$f(T) = a_0 + a_1T + a_2T^2 + \cdots + a_{d-1}T^{d-1}.$$

则

$$f(T) + f(-T) = 2(a_0 + a_2T^2 + a_4T^4 + \cdots),$$

$$f(T) - f(-T) = 2(a_1T + a_3T^3 + a_5T^5 + \cdots).$$

这两个恒等式说明：

- 把 $f(T)$ 与 $f(-T)$ 相加，会消掉所有奇次项；
- 把 $f(T)$ 与 $f(-T)$ 相减，会消掉所有偶次项。

因此只要能在环里合法构造“把 T 换成 $-T$ ”这件事，就能把一个 d -维对象一分为二。

8.2.3 为什么在这里 T 对应 ζ_m

在 fully packed slot 的统一幂基表示中，我们正是把某个 slot 写成

$$f(\zeta_m) = \sum_{j=0}^{d-1} a_j \zeta_m^j.$$

因此，上面的抽象恒等式可以直接用到这里。关键在于：要保证存在某个 automorphism，使得

$$\zeta_m \mapsto -\zeta_m.$$

在 power-of-two cyclotomic 的结构下，这通常由适当的 Frobenius 幂或其组合给出。在常见叙述中，会写成某个 $\sigma^{d/2}$ 满足

$$\sigma^{d/2}(\zeta_m) = -\zeta_m.$$

8.2.4 even/odd splitting 的基本构造

因此定义：

$$f_{\text{even}}(\zeta_m) := \frac{f(\zeta_m) + f(-\zeta_m)}{2},$$
$$f_{\text{odd}}(\zeta_m) := \frac{f(\zeta_m) - f(-\zeta_m)}{2\zeta_m}.$$

则有

$$f_{\text{even}}(\zeta_m) = a_0 + a_2\zeta_m^2 + a_4\zeta_m^4 + \cdots,$$
$$f_{\text{odd}}(\zeta_m) = a_1 + a_3\zeta_m^2 + a_5\zeta_m^4 + \cdots.$$

也就是说：

- f_{even} 只保留原来偶数索引的系数；
- f_{odd} 只保留原来奇数索引的系数；
- 两者都只依赖于 ζ_m^2 ，因此其“内部维度”减半。

直观解释 8.1. 这一步可以想成“把一排系数按奇偶下标分箱子”。原来一个 d -维向量：

$$(a_0, a_1, a_2, a_3, \dots)$$

现在被拆成两个 $d/2$ -维向量：

$$(a_0, a_2, a_4, \dots) \quad \text{和} \quad (a_1, a_3, a_5, \dots).$$

8.2.5 为什么这里要除以 2 和 ζ_m

8.2.5.1 除以 2

因为

$$f(T) + f(-T)$$

和

$$f(T) - f(-T)$$

都带了一个系数 2。在 bootstrapping 的明文环中，我们通常要求这个 2 在底层系数环 $\mathbb{Z}_{p^e}$ 中可逆。在大多数讨论 BGV/BFV fully packed bootstrap 的场景中，底层明文素数 p 是奇素数，因此 2 可逆。

8.2.5.2 除以 ζ_m

在 odd 部分中,

$$f(T) - f(-T) = 2T(a_1 + a_3T^2 + a_5T^4 + \cdots),$$

因此若要把它重新写成一个依赖于 T^2 的“新对象”, 需要再除以 T 。在我们的场景里, 这就是除以 ζ_m 。

这一除法发生在相应的 slot algebra 中, 是合法的, 因为 ζ_m 作为单位根是可逆元。

8.3 递归分裂的结构

8.3.1 从 d -维到 $d/2$ -维

上一节已经看到, even/odd splitting 能把

$$a_0 + a_1\zeta_m + \cdots + a_{d-1}\zeta_m^{d-1}$$

拆成两个只依赖 ζ_m^2 的对象:

$$f_{\text{even}}(\zeta_m) = a_0 + a_2\zeta_m^2 + a_4\zeta_m^4 + \cdots,$$

$$f_{\text{odd}}(\zeta_m) = a_1 + a_3\zeta_m^2 + a_5\zeta_m^4 + \cdots.$$

如果原来维数是 d , 那么新对象的有效维数就是

$$\frac{d}{2}.$$

8.3.2 继续递归: 从 $d/2$ -维到 $d/4$ -维

由于

$$f_{\text{even}}, f_{\text{odd}}$$

现在都只依赖于 ζ_m^2 , 可以把

$$\eta := \zeta_m^2$$

当成新的“内部变量”。于是我们再次对它们做 even/odd splitting, 就可以再把每个 $d/2$ -维对象拆成两个 $d/4$ -维对象。

例如:

$$f_{\text{even}}(\eta) = b_0 + b_1\eta + b_2\eta^2 + \cdots$$

再次拆分为

$$(f_{\text{even}})_{\text{even}}, \quad (f_{\text{even}})_{\text{odd}}.$$

8.3.3 递归到标量为止

若

$$d = 2^r,$$

则重复上述过程 r 次后，维数会依次变为：

$$d, \frac{d}{2}, \frac{d}{4}, \dots, 2, 1.$$

最终每个叶子都只剩一个标量系数。

这时原来的一个 slot 中的 d 个系数

$$a_0, \dots, a_{d-1}$$

就被彻底拆成了 d 条独立的标量流。

定理 8.2. 设 $d = 2^r$ 。通过对统一幂基表示的 *slot* 元素反复施加 *even/odd splitting*，可在 $r = \log_2 d$ 层后，把一个 d -维 *slot* 元素分解成 d 个标量分量。

证明. 每次 *even/odd splitting* 都将当前对象的有效内部维数减半。初始维数为 d 。经过 t 次后，维数为

$$\frac{d}{2^t}.$$

当 $t = r = \log_2 d$ 时，维数变为

$$\frac{d}{2^{\log_2 d}} = 1.$$

因此共得到 $2^r = d$ 个标量分量。 □

8.3.4 递归树视角

从算法结构上看，*unpack* 可以画成一棵二叉树：

一个 d -维 slot $\rightarrow$ 两个 $d/2$ -维对象 $\rightarrow$ 四个 $d/4$ -维对象 $\rightarrow \dots \rightarrow d$ 个标量。

每一层做的都是同一种事情：

- 选取适当 automorphism；
- 构造偶部分；
- 构造奇部分；
- 对结果继续递归。

直观解释 8.3. 这和 FFT 中“不断把问题二分”的风格非常类似。区别在于，FFT 是把一个大向量分解成频域结构；这里的递归分裂，是把一个 slot 的内部坐标逐步拆到标量层。

8.3.5 系数索引如何被递归编码

每次 even/odd splitting, 本质上是在看系数下标的最低一位 (二进制意义下):

- 偶数索引对应最低位为 0;
- 奇数索引对应最低位为 1。

继续递归时, 就是继续读取更高位。

因此, 经过 $\log_2 d$ 层之后, 每个最终标量都对应于一个长度为 $\log_2 d$ 的 0/1 序列, 也即原始系数索引 j 的二进制展开。

这说明 unpack 实际上是按二进制索引树在重排系数。

8.4 从 E -slot 到标量 slot 的过程

8.4.1 起点: 一个 E -slot

在 fully packed 情况下, 第 i 个 slot 一般长成

$$u_i = \sum_{j=0}^{d-1} u_{i,j} \zeta_{m,i}^j.$$

若先切换到统一幂基, 则可写成

$$u'_i = \sum_{j=0}^{d-1} u_{i,j} \zeta_m^j.$$

在这个阶段, 它是一个 E -slot: 即一个 slot 中装着一个 d -维对象。

8.4.2 目标: 标量 slot

我们希望最终把 fully packed 密文拆成若干个 sparse packed 密文, 使得第 t 个新密文中, 第 i 个 slot 只装原来第 i 个 slot 的第 t 个系数:

$$u_{i,t} \in \mathbb{Z}_{p^e}.$$

这样就从

E -slot

变成了真正的

scalar slot.

8.4.3 对所有 slots 同时递归分裂

要注意, unpack 并不是“对单个 slot 单独操作”。在密文中, 我们面对的是一个 ℓ -slot 向量:

$$(u_1, \dots, u_\ell).$$

当我们做一次 even/odd splitting 时, 实际上是对每个 slot 同时应用同样的内部拆分规则。

因此, 经过第一层后, 不是得到“每个 slot 各拆成两个槽位”, 而是得到两个新密文:

- 一个密文收集所有 slots 的偶坐标部分;
- 一个密文收集所有 slots 的奇坐标部分。

继续递归下去, 最终得到 d 个新密文, 每个密文都变成 sparse packed。

8.4.4 形式化描述

设原 fully packed 密文在 slot 视角下为

$$(u_1, \dots, u_\ell), \quad u_i = \sum_{j=0}^{d-1} u_{i,j} \zeta_m^j.$$

则 unpack 的输出可抽象写成

$$\mathbf{u}^{(0)}, \mathbf{u}^{(1)}, \dots, \mathbf{u}^{(d-1)},$$

其中第 t 个新密文满足:

$$\mathbf{u}^{(t)} \longleftrightarrow (u_{1,t}, u_{2,t}, \dots, u_{\ell,t}).$$

因此:

- 输入: 1 个 fully packed 密文;
- 输出: d 个 sparse packed 密文。

8.4.5 为什么这正是 digit extraction 需要的格式

digit extraction 后续想做的是:

$$u_{i,t} = m_{i,t} + p^e r_{i,t} \quad \longmapsto \quad m_{i,t}.$$

这现在已经是一个真正的标量级关系。因此每个新密文都可以被视为“一条可逐槽 digit extract 的 sparse 流”。

注 8.4. 这一步非常关键: digit extraction 不是从 fully packed 直接做起, 而是从“unpack 之后的 scalar-slot 流”做起。不理解这一点, 就会误以为 digit extraction 可以直接作用在一般 E -元素上。

8.5 repack 为什么是逆向重组

8.5.1 为什么 unpack 之后不能停下

unpack 之后，我们手里通常有 d 个 sparse packed 密文：

$$\mathbf{u}^{(0)}, \mathbf{u}^{(1)}, \dots, \mathbf{u}^{(d-1)}.$$

每个密文承载一个内部坐标流。

如果 digit extraction 已经对它们分别完成，那么现在拥有的是：

$$\mathbf{m}^{(0)}, \mathbf{m}^{(1)}, \dots, \mathbf{m}^{(d-1)},$$

其中

$$\mathbf{m}^{(t)} \longleftrightarrow (m_{1,t}, m_{2,t}, \dots, m_{\ell,t}).$$

这些信息从语义上已经足够恢复原消息，但它们还不是一个 fully packed 明文。

8.5.2 repack 的目标

repack 的目标是把这 d 条标量流重新组合成一个 fully packed 密文，使得第 i 个 slot 回到

$$m_i = \sum_{t=0}^{d-1} m_{i,t} \zeta_{m,i}^t.$$

也就是说，repack 需要完成从

$$\{\mathbf{m}^{(0)}, \dots, \mathbf{m}^{(d-1)}\}$$

到

$$(m_1, \dots, m_\ell)$$

的逆向重组。

8.5.3 为什么说它是 unpack 的逆过程

unpack 的逻辑是：

一个 d -维 slot $\rightarrow$ 两个 $d/2$ -维对象 $\rightarrow \dots \rightarrow d$ 个标量.

repack 则正好反过来：

d 个标量 $\rightarrow$ 两两合并成 $d/2$ -维对象 $\rightarrow \dots \rightarrow$ 一个 d -维 slot.

也就是说：

- unpack 是“递归拆树”；
- repack 是“递归回收并重构这棵树”。

8.5.4 为什么 repack 不是简单拼接

repack 不只是把

$$(m_{i,0}, m_{i,1}, \dots, m_{i,d-1})$$

写在一行就完事。它还必须保证：

- (1) 这些标量重新落回正确的局部幂基位置；
- (2) 各个 slot 的内部坐标顺序正确；
- (3) 所得结果是一个合法的 E_i -元素，而不仅仅是“一个列表”；
- (4) 整个组合过程在密文域中通过 automorphism、mask 和线性组合实现。

因此 repack 是一个真正的代数重构过程，而不是字面上的“数组拼接”。

8.5.5 repack 与 SlotToCoeff 的关系

实践中，repack 往往和 SlotToCoeff 紧密耦合。原因是：

- repack 先把 scalar-slot 流重构成 fully packed slot；
- SlotToCoeff 再把 fully packed slot 变回大环中的多项式系数表示。

二者在概念上不同，但在实现上常常可以融合，以减少额外层数和 automorphism 数量。

直观解释 8.5. 你可以把 repack 理解成“先把拆散的零件组装回模块”，而 SlotToCoeff 是“再把各模块装回整台机器”。前者修复的是 slot 内部结构，后者修复的是整个大环坐标结构。

8.6 unpack / repack 的复杂度与瓶颈

8.6.1 复杂度的三个主要来源

unpack / repack 的代价主要来自三类基本操作：

- (1) **automorphism / rotation**：每次 even/odd splitting 或其逆过程都需要对密文做自同构；
- (2) **key switching**：automorphism 之后通常要把密文切回标准秘密钥；
- (3) **plaintext mask 与线性组合**：用于抽取偶支、奇支，或把多个部分重新组合。

8.6.2 递归层数带来的复杂度

若内部维数为 d ，则 unpack/repack 的递归深度大约是

$$\log_2 d.$$

在每一层上，都需要同时处理当前所有分支。因此总操作数量往往与 d 和 $\log_2 d$ 同时相关，而不只是简单线性增长。

8.6.3 为什么 fully packed 显著更贵

在 sparse packed 中：

- 没有内部 d -维结构；
- 不需要 unpack；
- 不需要 repack。

在 fully packed 中：

- CoeffToSlot 后还必须 unpack；
- digit extraction 后还必须 repack；
- 且二者都伴随大量 automorphism 和 key switching。

因此 fully packed bootstrapping 的代价往往大幅高于 sparse / thin bootstrap。

8.6.4 为什么它们是 bootstrapping 的主要瓶颈之一

在完整流程中，主要代价来自：

- CoeffToSlot / SlotToCoeff；
- unpack / repack；
- digit extraction。

其中 unpack / repack 的特殊之处在于：它们不是“业务逻辑不可替代”的核心非线性，而是 fully packed 结构所强加的代表代价。

也就是说：

如果你想保住 fully packed 吞吐量，就必须为 unpack / repack 付费。

这也是 why many optimization works 会在以下方向用力：

- 让 unpack / repack 与其它线性变换融合；
- 选择更适合的 slot 几何，减少 automorphism 数量；
- 采用 thin bootstrap，直接避开 fully packed 的内部拆装成本；
- 用 partial CoeffToSlot 等思路，只“打开到足够的程度”，而不做完整 fully packed 展开。

8.6.5 复杂度的结构性总结

粗略地说，unpack / repack 的复杂度瓶颈在于：

- (1) **层数**：来自 $\log_2 d$ 级递归；
- (2) **宽度**：来自每层要处理多个分支；
- (3) **每个分支的成本**：来自 automorphism + key switching + mask；
- (4) **与其它步骤的耦合**：若实现不巧妙，会额外增加 multiplicative level 与临时密文数量。

8.7 本章小结

本章围绕 fully packed bootstrapping 中最容易让人困惑、但又极其关键的 unpack / repack 展开，核心结论如下：

- (1) fully packed 情况下一定会遇到 unpack / repack，因为每个 slot 里装的是一般 E -元素，而后续 digit extraction 想处理的是标量槽。
- (2) unpack 的基本思想是 even/odd splitting：利用

$$f(T) \pm f(-T)$$

把一个 d -维对象拆成两个 $d/2$ -维对象。

- (3) 递归重复这一过程，就能在 $\log_2 d$ 层后把一个 E -slot 拆成 d 条标量流。
- (4) 从整个密文层面看，unpack 不是“把一个 slot 拆成多个 slot 位置”，而是“把一个 fully packed 密文拆成多个 sparse packed 密文”。
- (5) repack 正是 unpack 的逆过程：把 digit extraction 后得到的 d 条标量流重新组装成 fully packed 结构。

(6) unpack / repack 的复杂度瓶颈主要来自递归层数、automorphism、key switching 和 mask，因此它们是 fully packed bootstrapping 昂贵的根本原因之一。

下一章将单独讨论 *digit extraction* 专题，系统说明：

- digit extraction 在数学上到底想做什么；
- 为什么“模 p^e ”在 slot 视角下是合法的；
- 为什么它必须在标量槽上实现；
- 常见的 base- p 与 base- p^k 两类思路；
- 以及它为什么成为 BGV bootstrapping 最核心的非线性步骤。

digit extraction 专题

9.1 digit extraction 的数学目标

9.1.1 从 BGV 的 noisy plaintext 出发

在前面的章节中，我们已经把 BGV bootstrapping 的核心问题压缩到了如下形式：

$$u = m + p^e r,$$

其中

- m 是真正的消息；
- r 是某个未知的误差/噪声项；
- p^e 是明文模数；
- u 是同态线性解密之后得到的 noisy plaintext。

因此，bootstrapping 中最关键的非线性步骤，就是从 u 中恢复 m 。

9.1.2 最核心的一句话

由于

$$u = m + p^e r,$$

所以在数学上最直接的恢复公式是

$$m = u \bmod p^e.$$

因此，所谓 digit extraction，在最本质的层面上就是：

从一个加密的 u 中，同态地计算出 $u \bmod p^e$ 。

9.1.3 为什么叫 digit extraction

因为“模 p^e ”可以等价地理解成：

- 把 u 按 base- p 展开时，保留最低 e 位 digits；
- 或者把 u 按 base- p^k 展开时，保留最低若干个 blocks；
- 或者说，去掉所有高位携带的 p^e -倍噪声部分。

因此这个步骤常被称作：

- digit extraction；
- modular reduction；
- residue extraction；
- low-digit recovery。

在不同文献里叫法略有区别，但目标是同一个。

9.1.4 一个最基本的数值直觉

设

$$p = 3, \quad e = 2, \quad p^e = 9.$$

若

$$u = 68,$$

则

$$68 = 5 + 9 \cdot 7.$$

因此

$$68 \bmod 9 = 5.$$

这就是说，若消息是 5，噪声把它抬到了 68，那么 digit extraction 要做的事就是把 68 再拉回 5。

9.1.5 抽象的代数表达

在更一般的环论视角下，digit extraction 处理的是：

$$u \in R_{p^e} \quad \text{或其某种 slot/子环表示,}$$

并恢复其在模 p^e 意义下的 residue class。

因此它并不只是一个“数论取余”的小技巧，而是 BGV bootstrapping 中唯一真正改变消息表达数值的核心步骤。

注 9.1. CoeffToSlot、SlotToCoeff、unpack、repack 等步骤都只是组织、搬运、重排、换基。真正把

$$m + p^e r$$

变成

$$m$$

的，只有 digit extraction。

9.2 “模 p^e ” 到底是在什么对象上做

9.2.1 最容易混淆的问题

很多人第一次接触 BGV bootstrapping 时会问：

既然 noisy plaintext 一开始是个多项式 $u(X)$ ，那“模 p^e ”到底是在对什么做？是对整个多项式做？对系数做？对 slot 做？还是对 slot 里的内部坐标做？

这个问题必须分层回答，否则后面的 digit extraction 会一直不清楚。

9.2.2 第一层：在大环里的意思

在 coefficient view 下，noisy plaintext 是

$$u(X) = m(X) + p^e r(X) \in R_Q.$$

若只从抽象环同余关系出发，则可以说：

$$u(X) \equiv m(X) \pmod{p^e}.$$

这里的“ $\text{mod } p^e$ ”意味着：把所有系数按模 p^e 约简。换句话说，它是商映射

$$R_Q \longrightarrow R_{p^e}$$

的作用。

因此在大环层面，“模 p^e ”是系数环约简，而不是“把 X 也除掉”之类的意思。

9.2.3 第二层：在 CRT 分解后的意思

由 CRT，

$$R_{p^e} \cong E^\ell.$$

如果在大环里有

$$u = m + p^e r,$$

那么对 CRT 同构两边同时作用，就得到

$$u_i = m_i + p^e r_i, \quad i = 1, \dots, \ell.$$

因此“模 p^e ”也可以逐 slot 地理解为：

$$u_i \bmod p^e = m_i.$$

但这一步必须小心：

这里的“逐 slot 取模”之所以合法，不是因为 slot 像数组元素，而是因为 CRT 是环同构。

9.2.4 第三层：在 E -slot 里的问题

若是 fully packed，则某个 slot 值一般形如

$$u_i = \sum_{j=0}^{d-1} u_{i,j} \zeta_{m,i}^j \in E_i.$$

它不是一个单独的整数，而是一个 d -维 $\mathbb{Z}_{p^e}$ -模元素。

这时如果写

$$u_i \bmod p^e,$$

严格说，它表示的是：

对 E_i 中这个元素的各底层系数按模 p^e 约简。

这在代数上当然是合法的，因为 E_i 的系数本来就来自 $\mathbb{Z}_{p^e}$ 。但这不等于你可以直接把它当“一个整数”去做 digit extraction。

9.2.5 为什么 bootstrapping 实现里还要进一步拆成标量槽

digit extraction 的算法视角通常不是“对一个抽象模元素做坐标约简”，而是：

先把目标对象变成真正的标量，再对这个标量做位/块提取。

因此，在 fully packed 情况下：

- 从抽象代数上， $u_i \bmod p^e$ 当然有意义；
- 但从算法实现上，digit extraction 一般希望面对

$$u_{i,j} \in \mathbb{Z}_{p^e}$$

这样的标量。

这就是为什么要先 unpack。

9.2.6 结论：三层对象不要混

可以把“模 p^e ”分成三层来理解：

(1) 大环层：

$$u(X) \mapsto u(X) \bmod p^e$$

是系数环的商映射；

(2) CRT / slot 层：

$$u_i \mapsto u_i \bmod p^e$$

是逐 slot 的 residue 恢复；

(3) 实现层：通常要进一步通过 unpack，把 E -slot 变成标量流

$$u_{i,j},$$

然后在这些标量上做 digit extraction。

9.3 为什么 CRT 后可以逐槽处理

9.3.1 核心命题

digit extraction 能够逐槽进行，其根本原因不是“slot 很像数组”，而是下面这条代数事实：

定理 9.2. 设

$$R_{p^e} \cong E^\ell$$

是一个环同构。若在 R_{p^e} 中有

$$u = m + p^e r,$$

则在每个 CRT 分量中都有

$$u_i = m_i + p^e r_i.$$

因此

$$u \bmod p^e = m$$

等价于对所有 i 都有

$$u_i \bmod p^e = m_i.$$

证明. 设

$$\phi : R_{p^e} \rightarrow E^\ell$$

为给定的环同构。则

$$\phi(u) = \phi(m + p^e r) = \phi(m) + p^e \phi(r),$$

因为 ϕ 保持加法与标量乘法。将

$$\phi(u) = (u_1, \dots, u_\ell), \quad \phi(m) = (m_1, \dots, m_\ell), \quad \phi(r) = (r_1, \dots, r_\ell)$$

代入，就得到

$$u_i = m_i + p^e r_i$$

对每个 i 成立。于是

$$u_i \equiv m_i \pmod{p^e}$$

对每个 i 成立，从而与

$$u \equiv m \pmod{p^e}$$

在同构下完全等价。 □

9.3.2 这一定理到底说明了什么

它说明：

大环里的“差一个 p^e 倍数”关系，会逐个 slot 传递下去。

因此 digit extraction 完全可以“逐个 slot 清掉 p^e -倍垃圾”。

9.3.3 为什么这正是 packed bootstrapping 的基础

若没有 CRT 分解，我们只能在整个大环元素 $u(X)$ 上做一个复杂的非线性处理。而有了 CRT 之后，问题被拆成了很多个小问题：

$$u_i = m_i + p^e r_i.$$

于是你可以：

- 对每个 u_i 单独做相同的操作；
- 在 SIMD 模式下并行处理它们；
- 最后再通过 SlotToCoeff 把结果组合回去。

9.3.4 为什么这仍不意味着“slot 就是系数”

这里再强调一次：

- 系数视角下，

$$u(X) = u_0 + u_1X + \cdots + u_{N-1}X^{N-1};$$

- CRT 视角下，

$$u \longleftrightarrow (u_1, \dots, u_\ell).$$

这两个 u_i 的含义完全不同。前者是多项式系数位置，后者是 CRT 分量。digit extraction 逐槽进行，依赖的是后者，而不是前者。

9.3.5 为什么实现上还要注意 slot 的类型

即使 CRT 保证了“逐 slot 处理”在数学上合法，算法实现仍取决于 slot 本身是什么：

- 若 slot 是标量，则可直接 digit extract；
- 若 slot 是 E -元素，则要先 unpack。

所以正确表述应该是：

CRT 让 digit extraction 逐槽合法；unpack 让它逐槽可实现。

9.4 标量槽与 E -槽的区别

9.4.1 标量槽是什么

当一个 slot 中的值直接属于

$$\mathbb{Z}_{p^e},$$

我们称之为标量槽。也就是说，第 i 个 slot 看起来就是

$$u_i \in \mathbb{Z}_{p^e}.$$

这正是 sparse packed 的情形。

9.4.2 E -槽是什么

当一个 slot 中的值属于

$$E_i = \mathbb{Z}_{p^e}[X]/(F_i(X)), \quad \deg F_i = d,$$

则我们称之为 E -槽或一般 slot algebra 槽。

这时它一般写成

$$u_i = \sum_{j=0}^{d-1} u_{i,j} \zeta_{m,i}^j.$$

这正是 fully packed 的情形。

9.4.3 二者的本质差别

9.4.3.1 自由度差别

一个标量槽只包含 1 个底层自由度。一个 E -槽包含 d 个底层自由度。

9.4.3.2 适合的算法差别

对标量槽, digit extraction 可直接视为一个单变量函数:

$$x \mapsto x \bmod p^e.$$

对 E -槽, 若不先指定基底并拆成坐标, 你甚至无法明确“最低位 digit”是什么意思。

9.4.3.3 线性变换差别

对标量槽, slot-to-coefficient 与 coefficient-to-slot 常常只是一个 Vandermonde/FFT 型矩阵。对 E -槽, 必须额外考虑:

- 局部幂基;
- 统一幂基;
- M 、 U 、 M^{-1} 的分工;
- unpack / repack。

9.4.4 为什么 fully packed 的核心困难就在这里

fully packed 的优势是把总自由度用满了, 但代价是:

每个 slot 不再是“一个数字”, 而是“一个小环元素”。

这导致后续非线性步骤不再能直接逐槽进行, 而必须先把 slot 内部结构摊平。

9.4.5 一个统一记忆方式

可以把两类槽总结成：

- **标量槽**： slot 本身就是 digit extraction 的输入对象；
- **E-槽**： slot 本身只是一个中间打包对象，digit extraction 之前还要拆开。

9.5 常见 digit extraction 思路

9.5.1 base- p 逐位提取

9.5.1.1 基本想法

若

$$u = m + p^e r,$$

则把 u 按 base- p 展开：

$$u = d_0 + d_1 p + d_2 p^2 + \cdots + d_L p^L, \quad 0 \leq d_t < p.$$

那么

$$u \bmod p^e = d_0 + d_1 p + \cdots + d_{e-1} p^{e-1}.$$

因此一种最直接的 digit extraction 思路是：

- (1) 先提取最低位 d_0 ；
- (2) 把它减掉并除以 p ；
- (3) 再提取下一位 d_1 ；
- (4) 重复 e 次；
- (5) 最后重组成消息。

9.5.1.2 递归形式

定义

$$u^{(0)} = u.$$

然后递归：

$$\begin{aligned} d_t &= u^{(t)} \bmod p, \\ u^{(t+1)} &= \frac{u^{(t)} - d_t}{p}. \end{aligned}$$

最终

$$u \bmod p^e = d_0 + d_1 p + \cdots + d_{e-1} p^{e-1}.$$

9.5.1.3 优点与缺点

优点

- 概念最直观；
- 每一步只处理一位；
- 与“逐位提取低位”直觉完全一致。

缺点

- 需要做 e 轮；
- 若 e 较大，电路深度和中间开销会累积；
- 逐位提取可能在实现上不够高效。

9.5.2 base- p^k 分块提取

9.5.2.1 基本想法

另一类思路不是一位一位提取，而是按更大的块提取。设

$$B = p^k.$$

则可把 u 写成

$$u = D_0 + D_1B + D_2B^2 + \cdots, \quad 0 \leq D_i < B.$$

若我们要恢复 $u \bmod p^e$ ，那么相当于保留最低若干个 base- B blocks。

9.5.2.2 例子

若

$$p = 3, \quad e = 4, \quad k = 2,$$

则

$$B = 3^2 = 9.$$

要恢复

$$u \bmod 3^4 = u \bmod 81,$$

那么可把 u 看成 base-9 展开：

$$u = D_0 + 9D_1 + 9^2D_2 + \cdots.$$

此时只需保留最低两个 blocks：

$$D_0, D_1.$$

9.5.2.3 优点与缺点

优点

- 提取轮数减少；
- 可减少递归层数；
- 更适合与某些 block-wise correction 技术结合。

缺点

- 单个 block 的提取更复杂；
- 需要在“块大小”和“电路复杂度”之间折中。

9.5.3 carry / correction 视角

9.5.3.1 为什么会出现 carry

把 u 写成 digit 或 block 形式时，真正困难的不只是“看见最低位”，而是：

- 高位可能通过进位影响低位表示；
- 同态电路里不能直接做传统 CPU 那种整数取模；
- 因而要显式地控制 carry 或用 correction 消除高位污染。

9.5.3.2 另一种理解：先近似、再修正

很多 digit extraction 算法并不直接一步算出

$$u \bmod p^e,$$

而是先得到某个近似的低位部分，再通过 correction 或 carry cancellation 把结果修正到正确 residue。

因此，你也可以把 digit extraction 看成：

“低位恢复 + carry 清理 + 最终修正”

9.5.3.3 为什么这只是同一问题的不同实现视角

无论是：

- base- p 逐位提取，
- base- p^k 分块提取，
- 还是 carry/correction 视角，

最终目的都一样：

$$u = m + p^e r \quad \mapsto \quad m.$$

不同思路只是把这件事分解成不同的中间步骤而已。

9.6 digit extraction 的电路深度来源

9.6.1 为什么 digit extraction 是 bootstrapping 的非线性瓶颈

与 CoeffToSlot、SlotToCoeff、unpack、repack 不同，digit extraction 不是一个纯线性变换。它必须实现诸如：

- mod p ;
- mod p^e ;
- digit 提取;
- carry 判定与修正;

这些都不是线性函数，也不能仅靠固定矩阵完成。因此 digit extraction 通常决定了 bootstrapping 的乘法深度瓶颈。

9.6.2 深度来自哪里

digit extraction 的电路深度通常来自以下几类操作：

- (1) **digit / block 识别**：需要确定当前数属于哪个 residue 或哪个 digit 区间；
- (2) **归一化与缩放**：把中间值转到适合提取 digits 的范围；
- (3) **carry / correction**：清理高位对低位的影响；
- (4) **重组**：把提取出的 digits 或 blocks 重新合成为最终 residue。

9.6.3 $\text{base-}p$ 与 $\text{base-}p^k$ 对深度的影响

$\text{base-}p$ 逐位提取 每一位的提取可能相对简单，但总轮数大。因此深度来源更像“层数多”。

$\text{base-}p^k$ 分块提取 总轮数减少，但单块处理更复杂。因此深度来源更像“每层更重”。

9.6.4 为什么这和线性变换优化不同

线性变换优化（如 BSGS、FFT-like 分解）主要减少的是：

- automorphism 数量；
- key switching 数量；
- 线性组合成本。

而 digit extraction 的优化更关注：

- 非线性电路深度；
- 乘法层数；
- 中间值范围控制；
- carry/correction 的复杂度。

因此在 bootstrapping 里，它们属于两类不同的优化问题。

9.6.5 为什么 fully packed 还会额外放大 digit extraction 的代价

digit extraction 自身是在标量槽上做的。如果前面必须经过 unpack，而后面还要 repack，那么 digit extraction 的上下游会带来大量额外成本。

因此 fully packed 中 digit extraction 的“总成本”不只是它本身的非线性深度，还包括：

- 为了让它可做而付出的 unpack 成本；
- 为了把结果重新利用而付出的 repack 成本。

9.7 digit extraction 与 BFV / CKKS 中对应步骤的对比

9.7.1 与 BFV 的对比

9.7.1.1 BGV 的目标

BGV 中 noisy plaintext 的结构是

$$u = m + p^e r.$$

因此 digit extraction 的目标是：

$$u \mapsto u \bmod p^e = m.$$

9.7.1.2 BFV 的目标

BFV 中 noisy plaintext 更像

$$u = \Delta m + e, \quad \Delta \approx \frac{Q}{p^e}.$$

因此它的核心非线性不是单纯“模 p^e ”，而是更像：

$$u \mapsto \left\lfloor \frac{p^e}{Q} u \right\rfloor \bmod p^e.$$

9.7.1.3 本质区别

所以：

- BGV 的 digit extraction 更偏向“低位 residue 恢复”；
- BFV 的对应步骤更偏向“缩放、舍入、再取模”。

尽管两者都可能使用 digit decomposition、block decomposition、carry correction 等技术，但其目标函数并不一样。

9.7.2 与 CKKS 的对比

9.7.2.1 CKKS 的中心步骤不是精确 residue 恢复

CKKS bootstrapping 的核心步骤通常是 EvalMod，即对复数/实数槽值近似计算某个模函数或周期函数。

因此 CKKS 的核心非线性是一个近似解析函数评估问题，而不是一个精确的整数 residue 恢复问题。

9.7.2.2 为什么这与 BGV digit extraction 完全不同

在 BGV 中，我们关心的是：

$$m \in \mathbb{Z}_{p^e}$$

的精确恢复。在 CKKS 中，我们关心的是近似数值的重新规范化与噪声压回。

因此：

- BGV：是精确整数/模环语义；
- CKKS：是近似实数/复数语义。

9.7.2.3 共同点与不同点

共同点 三者（BGV/BFV/CKKS）都可能出现：

- CoeffToSlot；
- SlotToCoeff；
- rotation；
- 大线性变换分解。

不同点 它们在中间最贵的非线性步骤上完全不同：

- BGV：digit extraction / modular reduction；
- BFV：scale-and-round；
- CKKS：EvalMod。

直观解释 9.3. 所以如果把 bootstrapping 比作“同态地把 noisy plaintext 重新洗干净”，那么：

- BGV 是在精确地刷掉高位泥点；
- BFV 是先缩尺、再对齐、再刷；
- CKKS 则更像用一个光滑近似函数把整体形状重新拉回正确区间。

9.8 本章小结

本章把 digit extraction 作为 BGV bootstrapping 的核心非线性步骤单独剖开，主要结论如下：

- (1) digit extraction 的数学目标就是从

$$u = m + p^e r$$

中恢复

$$m = u \bmod p^e.$$

- (2) “模 p^e ”要分层理解：在大环里是系数环约简；在 CRT 后是逐 slot residue 恢复；在实现层面通常要进一步把 E -slot 拆成标量槽再做。
- (3) CRT 之所以允许逐槽处理，不是因为 slot 像数组元素，而是因为它给出了一个环同构，把“大环里差一个 p^e 倍数”变成“每个 slot 里都差一个 p^e 倍数”。
- (4) 标量槽与 E -槽的区别是 digit extraction 能否直接实施的关键：前者可直接做，后者必须先 unpack。
- (5) 常见 digit extraction 思路包括：
- base- p 逐位提取；
 - base- p^k 分块提取；
 - carry / correction 视角。
- (6) digit extraction 的电路深度来源于它的非线性本质，因此它是 bootstrapping 中最核心的非线性瓶颈。
- (7) 与 BFV、CKKS 对比时要牢记：BGV 的 digit extraction 是精确 residue 恢复，而 BFV 更偏 scale-and-round，CKKS 更偏近似 EvalMod。

下一章将进入 线性变换实现与 BSGS，讨论：

- 为什么 bootstrapping 中的大线性变换可以写成 weighted rotations；
- baby-step/giant-step 如何减少 automorphism 数量；
- good/bad dimension 如何影响复杂度；
- hoisting / double-hoisting 为什么重要；
- 以及这些技术如何直接服务于 CoeffToSlot、SlotToCoeff、unpack、repack 的实现优化。

线性变换实现与 BSGS

10.1 为什么线性变换是 bootstrapping 的大头

10.1.1 bootstrapping 中不止有一个“大矩阵”

到了这一章，我们已经知道 bootstrapping 中至少有以下几类“重步骤”：

- 同态线性解密；
- CoeffToSlot；
- SlotToCoeff；
- unpack / repack；
- digit extraction。

其中 digit extraction 是核心非线性瓶颈；但另一方面，前后那些看起来“只是线性代数”的步骤，往往同样非常昂贵。原因在于：在 FHE 中，一个线性变换并不是像普通数值计算里那样，直接乘一个矩阵就完事了。

在普通线性代数里，若

$$y = Ax,$$

那么我们只是对一个向量做了一次矩阵乘法。但在同态加密中， x 不是普通向量，而是密文所承载的 slot 向量；矩阵 A 也不能直接作用，而必须拆成：

- rotation / automorphism；
- plaintext mask multiplication；
- 多项求和；
- 每次 automorphism 之后的 key switching。

因此，线性变换的真实成本往往体现在：

为了实现“一个矩阵乘法”，需要做多少次 automorphism、多少次 key switching、多少次明文乘法。

10.1.2 为什么 CoeffToSlot / SlotToCoeff 特别贵

CoeffToSlot 和 SlotToCoeff 的本质都是大规模线性同构：

- sparse packed 时，它们对应一个 $\ell \times \ell$ 的 Vandermonde/FFT 型矩阵；
- fully packed 时，它们往往包含

$$M, U, M^{-1}$$

三类结构，且还常与 unpack / repack 交织。

这些矩阵在纸面上看起来只是“固定常数矩阵”，但在 FHE 中实现它们，需要：

- (1) 对 slot 做大量 rotation；
- (2) 每个 rotation 后做 key switching；
- (3) 再乘以对角系数；
- (4) 最后把很多项相加。

因此，一旦 slot 数 ℓ 稍大，这些步骤就会迅速成为开销大头。

10.1.3 为什么 unpack / repack 同样属于线性变换问题

虽然 unpack / repack 经常被单独讨论，但从线性代数角度看，它们本质上仍然是线性变换。例如：

- even/odd splitting 由

$$f(\zeta_m) \pm f(-\zeta_m)$$

构成；

- 每一层递归拆分都是线性组合；
- repack 则是这些线性映射的逆向重组。

因此 unpack / repack 的成本，也可以统一放进“如何高效实现大线性变换”这个框架来分析。

10.1.4 为什么要单独研究 BSGS

既然很多 bootstrapping 步骤都能被看成“某种线性变换”，那自然会问：

有没有统一办法，减少这些线性变换实现中的 rotation 数量？

答案就是 BSGS (baby-step / giant-step)。

BSGS 并不改变线性变换本身，它做的是：

重排求和顺序，把“很多次旋转”组织成更少、更规整的旋转。

这也是为什么 BSGS 在 CoeffToSlot、SlotToCoeff、unpack、repack、trace 等实现里都非常核心。

10.2 weighted rotations 形式

10.2.1 从普通矩阵乘法到 rotation 分解

先考虑最简单的一维 slot 场景。设

$$x = (x_0, x_1, \dots, x_{n-1})^T$$

是一个长度为 n 的 slot 向量， $A \in M_n(\mathbb{F})$ 是一个线性变换。我们希望实现

$$y = Ax.$$

在 FHE 中，自然的基本操作不是“任意矩阵乘法”，而是：

- 先对 x 做某种循环移位；
- 再对不同位置乘不同权重；
- 最后把若干项加起来。

这就导向了所谓的 weighted rotations 表示。

10.2.2 循环置换基分解回顾

记 P_k 为长度 n 的循环移位矩阵，即

$$P_k x$$

表示把向量循环移位 k 步。则上一章已经给出：

定理 10.1 (循环置换基分解). 对任意矩阵

$$A \in M_n(\mathbb{F}),$$

存在唯一的一组对角矩阵 $D_0, \dots, D_{n-1}$, 使得

$$A = \sum_{k=0}^{n-1} D_k P_k.$$

因此

$$Ax = \sum_{k=0}^{n-1} D_k (P_k x).$$

10.2.3 weighted rotations 的 FHE 解释

这条分解在同态加密中的含义极其直接:

- $P_k x$: 对应一次 rotation;
- D_k : 对应一个 plaintext mask / 对角权重;
- 对所有 k 求和: 对应把所有加权 rotation 的结果相加。

所以, 一个线性变换可以被同态实现为:

$$L(x) = \sum_k \kappa_k \cdot \rho^k(x),$$

其中:

- ρ^k 表示第 k 个 rotation;
- κ_k 是对应的对角权重 (slot-wise plaintext multiplier)。

10.2.4 多维 slot 几何中的写法

若 slots 不是单一循环, 而是多维坐标

$$(e_1, \dots, e_t),$$

那么固定某个维度 i , 第 i 维线性变换可写成

$$L_i(m) = \sum_{v=0}^{\ell_i-1} \kappa(v) \rho_i^v(m).$$

这时:

- ρ_i^v 是沿第 i 维平移 v 步;
- $\kappa(v)$ 是与该平移对应的权重。

10.2.5 为什么这是 bootstrapping 的统一语言

很多看起来不同的操作，其实都能放进这一语言：

- sparse packed 的 CoeffToSlot / SlotToCoeff;
- fully packed 的外层 U -矩阵变换;
- trace;
- unpack / repack 中每层的线性组合;
- 甚至某些 selection / recombination 操作。

因此在实现层，大家真正优化的不是“矩阵本身”，而是：

怎样用尽可能少的 weighted rotations 来表达同一个线性变换。

10.3 BSGS 的基本思想

10.3.1 为什么直接实现 weighted rotations 很贵

若我们朴素实现

$$L(m) = \sum_{v=0}^{n-1} \kappa(v) \rho^v(m),$$

那就需要：

- n 次 rotation;
- n 次对应的 key switching;
- n 次 plaintext multiplication;
- 再加若干次求和。

在 bootstrapping 中， n 可能是：

- 某个 slot 维度 ℓ_i ;
- 或整个 sparse packed 的 slot 数 ℓ ;
- 或某个拆分后子结构的长度。

当 n 较大时，直接实现就会很重。

10.3.2 BSGS 的核心想法

BSGS 的思想来自一个非常经典的分块策略：把一个长索引 v 拆成两个较短索引的组合。

设

$$n = n_1 n_2.$$

则任意 $v \in \{0, \dots, n-1\}$ 都可唯一写成

$$v = a + bn_1, \quad 0 \leq a < n_1, \quad 0 \leq b < n_2.$$

于是 rotation 也可分解为

$$\rho^v = \rho^{a+bn_1} = \rho^a \circ \rho^{bn_1}.$$

这意味着原来“很多个独立 rotation”的求和，可以重写成两层分组：

$$L(m) = \sum_{b=0}^{n_2-1} \left(\sum_{a=0}^{n_1-1} \kappa(a + bn_1) \rho^a(m) \right)^{\rho^{bn_1} \text{调整后}}$$

或者更标准地写成 giant-step 作用在 baby-step 聚合项上的形式。

10.3.3 直观图像

你可以把朴素算法想成：

每个 v 都单独转一次，做一次权重乘法，再加起来。

而 BSGS 想成：

先把“小步平移”预先组织好（baby steps），再用较少的大步平移（giant steps）去搬运这些小块。

直观解释 10.2. 这和普通算法中“先做小块预计算，再把大块复用”是同一个思想。在 FHE 中，最昂贵的是 rotation + key switching，因此任何能让不同项共享 rotation 结构的分组都很有价值。

10.3.4 为什么要叫 baby-step / giant-step

因为这里有两种尺度的索引：

- a ：小范围平移，称为 baby step；
- bn_1 ：大跨度平移，称为 giant step。

这两个尺度一配合，就能把原本 n 个 rotation 的朴素结构重排成更有层次的实现。

10.3.5 一个简单例子

设 $n = 12$, 取

$$n_1 = 3, \quad n_2 = 4.$$

则任意

$$v \in \{0, \dots, 11\}$$

都可写成

$$v = a + 3b, \quad a \in \{0, 1, 2\}, \quad b \in \{0, 1, 2, 3\}.$$

于是原来需要 12 个独立 rotation 的项, 可以组织成:

- 先处理 3 个 baby steps:

$$\rho^0, \rho^1, \rho^2;$$

- 再处理 4 个 giant steps:

$$\rho^0, \rho^3, \rho^6, \rho^9.$$

这就是 BSGS 的雏形。

10.3.6 为什么能减少 automorphism 数量

10.3.6.1 朴素算法

朴素算法中, 每个 v 都要独立做一次 ρ^v , 所以 rotation 数量约为

$$n.$$

10.3.6.2 BSGS 算法

若分解为

$$n = n_1 n_2,$$

则只需要:

- n_1 个 baby-step rotations;
- n_2 个 giant-step rotations。

总数约为

$$n_1 + n_2.$$

若取

$$n_1 \approx n_2 \approx \sqrt{n},$$

则总 rotation 数降到大约

$$2\sqrt{n}.$$

10.3.6.3 形式化比较

命题 10.3. 对于长度为 n 的一维 *weighted rotations* 线性变换, 若采用 *BSGS* 分块

$$n = n_1 n_2,$$

则 *rotation* 数可由朴素的 n 级别降到 $n_1 + n_2$ 级别。当 $n_1 \approx n_2 \approx \sqrt{n}$ 时, 该数量约为 $O(\sqrt{n})$ 。

证明. 朴素算法需要对每个 $v \in \{0, \dots, n-1\}$ 做一次 *rotation*, 共 n 次。BSGS 分块后, 只需准备所有 baby-step rotations ρ^a (共 n_1 次) 与所有 giant-step rotations ρ^{bn_1} (共 n_2 次), 其余通过组合与线性组合完成。故 *rotation* 数为

$$n_1 + n_2.$$

当 $n_1 = n_2 = \sqrt{n}$ 时取到数量级最小, 为

$$2\sqrt{n}.$$

□

注 10.4. 严格实现里, 还要区分:

- 哪些 *rotation* 可以共享;
- bad dimension 下一个 *rotation* 可能对应两支 automorphism;
- hoisting 是否可进一步摊薄 key switching 成本。

因此实际常数会比理论数量级稍复杂, 但核心收益来源就是这个 $n \rightarrow n_1 + n_2$ 的重排。

10.4 one-dimensional BSGS

10.4.1 问题设定

考虑一个沿单一维度的线性变换:

$$L(m) = \sum_{v=0}^{n-1} \kappa(v) \rho^v(m),$$

其中:

- m 是当前密文所承载的 slot 向量;
- ρ^v 是该维度上的 *rotation*;
- $\kappa(v)$ 是对应的 slot-wise 权重。

这就是最典型的一维 *weighted rotations* 问题。

10.4.2 BSGS 分块

取

$$n = n_1 n_2,$$

并把索引写成

$$v = a + bn_1, \quad 0 \leq a < n_1, \quad 0 \leq b < n_2.$$

于是

$$L(m) = \sum_{b=0}^{n_2-1} \sum_{a=0}^{n_1-1} \kappa(a + bn_1) \rho^{a+bn_1}(m).$$

利用 rotation 的群性质,

$$\rho^{a+bn_1} = \rho^{bn_1} \circ \rho^a,$$

于是可重写为

$$L(m) = \sum_{b=0}^{n_2-1} \rho^{bn_1} \left(\sum_{a=0}^{n_1-1} \kappa(a + bn_1) \rho^a(m) \right),$$

或者等价地视为: 先构造 baby-step 内部和, 再施加 giant-step。

10.4.3 实现上的组织方式

一种标准实现组织方式是:

- (1) 预先计算所有 baby-step rotations

$$\rho^0(m), \rho^1(m), \dots, \rho^{n_1-1}(m);$$

- (2) 对每个 b , 把这些 baby-step 项按权重

$$\kappa(a + bn_1)$$

线性组合成一个中间块;

- (3) 再对这些中间块施加 giant-step rotation

$$\rho^{bn_1}.$$

也可反过来先 giant-step 再 baby-step, 具体取决于实现细节与 hoisting 策略, 但基本思想不变。

10.4.4 为什么一维 BSGS 特别适合 rotation 群

一维情况下，所有 rotations 都来自同一个循环群，因此：

- 索引分块最自然；
- giant-step 就是“固定大步长的循环移动”；
- baby-step 就是“小范围局部移位”；
- 所有 rotation 都彼此兼容于同一群结构。

这也是为什么 BSGS 最早通常在单维 cyclic rotation 场景下被引入。

10.4.5 复杂度平衡

若每个 giant-step 都需要重新做一整套 baby-step，收益就会消失。真正的优化点在于：

- baby-step rotations 可以被多个 giant-step 复用；
- giant-step 只搬运“块”，不再关心块内部的所有细节；
- key switching 的某些中间结果也可共享（后面 hoisting 会详细说）。

因此，一维 BSGS 的价值不只是“公式上改写”，而是“共享大量中间结构”。

10.5 multidimensional BSGS

10.5.1 为什么需要多维版本

在 BGV/BFV 的 slot 几何中，slots 不一定总是单环排列。上一章已经说明，slot 索引群可写成

$$C_{\ell_1} \times \cdots \times C_{\ell_t},$$

因此线性变换往往是“沿某一维做变换”，或者“若干维交替作用”。

这时，如果只会一维 BSGS，就不够了。必须推广到：

在多维 slot 坐标系中，分别沿不同维度组织 baby-step / giant-step。

10.5.2 最基本的多维思路

对某个固定维度 i ，考虑线性变换

$$L_i(m) = \sum_{v=0}^{\ell_i-1} \kappa_i(v) \rho_i^v(m).$$

这在形式上与一维完全一样，只是 rotation 现在是第 i 维 rotation。

因此可以直接对该维做 BSGS：

$$v = a + bn_1, \quad 0 \leq a < n_1, \quad 0 \leq b < n_2, \quad \ell_i = n_1 n_2,$$

从而得到该维上的 baby-step / giant-step 分组。

10.5.3 为什么这叫 multidimensional 而不是 many one-dimensional

因为真正的多维结构体现在两点：

- (1) 不同维度的 rotation 成本不同；
- (2) 某些维度是 good dimension，某些是 bad dimension；
- (3) 线性变换可能是按维逐层组合的，而不是完全分离的。

所以 multidimensional BSGS 的目标，不只是“在每一维都做一次一维 BSGS”，而是：

根据 slot 几何与不同维度的实现成本，选择全局最合适的分块方式。

10.5.4 多维分块的一个典型情景

设 slots 写成

$$(e_1, e_2), \quad 0 \leq e_1 < \ell_1, \quad 0 \leq e_2 < \ell_2.$$

若某个线性变换沿第 1 维展开成

$$L_1(m) = \sum_{v=0}^{\ell_1-1} \kappa(v) \rho_1^v(m),$$

则可对第 1 维做 BSGS；若另一层线性变换沿第 2 维展开成

$$L_2(m) = \sum_{w=0}^{\ell_2-1} \lambda(w) \rho_2^w(m),$$

则又可对第 2 维分别做另一套 BSGS。

这就形成了多维交错的优化结构。

10.5.5 为什么多维结构对 bootstrapping 特别重要

在实际的 `CoeffToSlot / SlotToCoeff`、`unpack / repack`、`trace` 等操作中，常常会出现：

- 某些维度只需要少量 rotation；
- 某些维度虽然长，但全是 good dimension；
- 某些维度虽然短，但属于 bad dimension，单次 rotation 就很贵。

因此，“哪一维先做”“哪一维采用更 aggressive 的 BSGS 分块”“哪一维直接朴素实现”都可能影响总复杂度。

10.6 good dimension / bad dimension 对成本的影响

10.6.1 回顾 good / bad dimension

上一章已经定义：

- **good dimension**：某维 rotation 可以由单个 automorphism 干净实现；
- **bad dimension**：某维 rotation 通常需要掩码 + 两支 automorphism 才能拼出理想平移。

这一区别对 BSGS 的影响极大。

10.6.2 在 good dimension 中的成本

若某维是 good dimension，则一次 rotation 大致只对应：

- 一次 automorphism；
- 一次 key switching；
- 再加必要的 mask/权重乘法。

因此一维 BSGS 的复杂度估计大致就是：

$$\text{rotation cost} \sim n_1 + n_2.$$

10.6.3 在 bad dimension 中的成本

若某维是 bad dimension，则一个“理想 rotation”本身就往往要拆成：

$$\rho^v = \mu_v \tau_{j_1(v)} + (1 - \mu_v) \tau_{j_2(v)}.$$

也就是说，一个 rotation 不再对应一次 automorphism，而更像对应“两支 automorphism + 一次掩码拼接”。

因此在 bad dimension 中：

- rotation 数的理论下降仍然存在；
- 但每个 rotation 的单价更高；
- 且 mask 数量、key switching 数量也会增加。

10.6.4 为什么这会影响最优分块

假设有两维：

- 第 1 维长度大，但属于 good dimension；
- 第 2 维长度小，但属于 bad dimension。

直觉上你可能会先优化“更长的维”。但实际上，若 bad dimension 的单次 rotation 成本远高于 good dimension，那么即使长度较短，它也可能成为真正的瓶颈。

因此多维 BSGS 的最优策略不是只看 ℓ_i ，而要看：

$$\text{总成本} \approx (\text{rotation 数}) \times (\text{每次 rotation 代价}).$$

10.6.5 一个实践层面的结论

在有 good / bad dimension 区分的场景里，常见经验是：

- 对 bad dimension 更谨慎地使用 rotation；
- 尽可能把更多工作压到 good dimension 上；
- 若必须在 bad dimension 上做大线性变换，则更需要 BSGS 与 hoisting 的配合。

10.7 hoisting / double-hoisting

10.7.1 为什么只有减少 rotation 数量还不够

到目前为止,BSGS 主要看起来是在减少 rotation 的数量。但在 FHE 中,一次 rotation 的代价本身也很复杂:

$$\text{rotation} = \text{automorphism} + \text{key switching}.$$

而 key switching 里又包含若干重复性的中间步骤,例如:

- 基分解 (decomposition);
- 与切换密钥的乘积;
- 模数重组与归约。

如果你对很多个 rotations 都独立重复这些步骤,那么即使 rotation 数量下降了,代价仍可能很高。

10.7.2 hoisting 的思想

hoisting 的核心思想是:

把许多 rotations 共有的“前半段”预先做一次,然后复用给多个不同的 rotation。

最常见的情况是:多个 automorphism 都作用在同一个原始密文上。这时某些与原密文相关、但与具体 rotation index 无关的预处理,可以先做完,再为不同 rotation 共享。

10.7.3 一个抽象描述

设需要计算

$$\tau_{j_1}(c), \tau_{j_2}(c), \dots, \tau_{j_t}(c)$$

并对它们分别做 key switching。若每次都单独做 decomposition,则代价很大。hoisting 允许我们先对 c 做一次 decomposition,然后把该结果复用于多个 τ_{j_k} 对应的 key switching。

因此,hoisting 并不减少 rotation 的理论个数,但减少了:

- 每个 rotation 的重复预处理;
- 平均 key switching 成本。

10.7.4 double-hoisting 的思想

若在某些算法中，不仅第一层可共享，甚至在更后面的另一层也有共享结构，则可进一步做 double-hoisting。

粗略地说：

- hoisting：共享一次 decomposition；
- double-hoisting：共享两层相关的 decomposition / switching 中间结果。

这通常出现在：

- 两层嵌套的 BSGS；
- 或大线性变换中先后多轮 rotation 且彼此结构相近的场景。

10.7.5 为什么 hoisting 与 BSGS 是天然搭档

BSGS 会把一个大线性变换组织成：

- 少量 baby-step rotations；
- 少量 giant-step rotations；
- 且很多块共享同样的中间结构。

这恰好为 hoisting 创造了理想环境：因为“同一个原始密文被很多 rotation 复用”的结构在 BSGS 中极其明显。

因此实际优化里，经常不是单独说“做 BSGS”或“做 hoisting”，而是：

用 BSGS 组织 rotation 结构，再用 hoisting 摊薄 key switching 成本。

10.8 BSGS 在 CoeffToSlot / SlotToCoeff 中的作用

10.8.1 CoeffToSlot / SlotToCoeff 为什么天然适合 BSGS

无论是 sparse packed 还是 fully packed，CoeffToSlot / SlotToCoeff 最终都可被视为某种大线性变换。

sparse packed. 它们对应于 Vandermonde/FFT 型矩阵。

fully packed. 它们由

$$M, U, M^{-1}$$

这样的块组成，其中 U 是外层 slot 方向的 FFT-like 变换。

这类矩阵有两个共同特征：

- 结构高度规则；
- 可以分解成 weighted rotations 的和。

因此，BSGS 特别适合在这里发挥作用。

10.8.2 在 sparse packed 中的作用

对 sparse packed 而言，CoeffToSlot / SlotToCoeff 本质上是在子环上的求值/插值。因此其矩阵条目形如

$$U_{ik} = y_i^k$$

或某种列置换版本。

实现时，它们可写成

$$L(m) = \sum_{v=0}^{\ell-1} \kappa(v) \rho^v(m).$$

这正是一维 weighted rotations 结构，因此可直接做一维 BSGS，把原本 $O(\ell)$ 级别的 rotations 降到 $O(\sqrt{\ell})$ 级别。

10.8.3 在 fully packed 中的作用

fully packed 情况更复杂，但 BSGS 仍然起核心作用。

对 U 部分： 外层 slot 混合仍然是一个沿 slot 索引方向的线性变换，因此可以做 BSGS。

对 M 、 M^{-1} 部分： 它们虽是逐 slot 基变换，但在实现上也常被写成某种结构化线性变换，因此同样可借助 weighted rotations 与 BSGS 分解。

对 unpack / repack： 每层 even/odd splitting 或其逆过程也可视为线性变换块，因此某些实现中也会对这些块应用 BSGS 思想。

10.8.4 为什么 FFT-like 分解与 BSGS 可以同时存在

这两种优化并不冲突。

- FFT-like 稀疏分解：把一个大密集矩阵拆成若干层稀疏矩阵；

- BSGS: 在实现某一层稀疏矩阵时, 再进一步优化其 rotation 结构。

也就是说:

FFT-like 分解优化的是“矩阵层结构”, BSGS 优化的是“每层内部的 rotation 组织”。

10.8.5 实际意义: 为什么它们决定了 bootstrapping 是否可行

在参数一大时, 若没有 BSGS 与 hoisting:

- CoeffToSlot / SlotToCoeff 的 rotation 数会爆炸;
- key switching 成本会压垮整体复杂度;
- fully packed bootstrapping 尤其难以落地。

因此, 在现代 BGV/BFV bootstrapping 中, BSGS 几乎不是“可选优化”, 而是:

使大规模线性变换真正可实现的基础手段。

10.9 本章小结

本章围绕“如何在 FHE 中高效实现大线性变换”这一问题, 系统介绍了 BSGS 与相关优化。主要结论如下:

- (1) bootstrapping 中的大量代价并不只来自 digit extraction, CoeffToSlot、SlotToCoeff、unpack、repack 这类大线性变换同样是重成本来源。
- (2) 任意沿某一维的线性变换都可写成 weighted rotations 的形式:

$$L(m) = \sum_v \kappa(v) \rho^v(m).$$

- (3) BSGS 的基本思想是把索引

$$v = a + bn_1$$

分成 baby-step 与 giant-step 两层, 从而把原本 n 级别的 rotations 降到 $n_1 + n_2$ 级别。

- (4) 在一维场景下, 取

$$n_1 \approx n_2 \approx \sqrt{n}$$

时, rotation 数可从 $O(n)$ 降到 $O(\sqrt{n})$ 。

- (5) 在多维 slot 几何下, BSGS 需要结合各维长度、good/bad dimension 性质一起设计, 而不是机械地对每一维独立套公式。

- (6) good dimension 与 bad dimension 会显著影响单次 rotation 的实现成本, 因此 BSGS 的优化目标应看总成本, 而不是只看 rotation 数量。
- (7) hoisting / double-hoisting 进一步通过共享 decomposition 和 key switching 中间结果, 摊薄大量 rotations 的平均成本, 是 BSGS 的天然搭档。
- (8) 在 CoeffToSlot / SlotToCoeff 中, BSGS 之所以特别有效, 是因为这些变换天然具有结构化的 weighted rotations 形式; 在 fully packed 情况下, M 、 U 、 M^{-1} 、unpack、repack 等也都可从这一角度被组织和优化。

下一章将继续讨论 稀疏矩阵分解与 *FFT-like* 结构, 从矩阵层面解释:

- 为什么 U 类矩阵能被分解成若干层稀疏矩阵;
- 为什么列置换版本特别重要;
- 蝶形结构如何出现;
- 以及这种分解如何与 BSGS、hoisting 一起组成现代 bootstrapping 的线性变换优化体系。

稀疏矩阵分解与 FFT-like 结构

11.1 为什么要分解 U 矩阵

11.1.1 从“一个大矩阵”到“很多小层”

在前几章中，我们已经反复看到一个核心现象：

- sparse packed 的 $\text{CoeffToSlot} / \text{SlotToCoeff}$ ，本质上是某个 Vandermonde/CRT 型矩阵 U 的作用；
- fully packed 中虽然完整变换是

$$M^{-1}UM$$

或其逆，但其中最核心的“外层 slot 混合”部分仍然集中在 U 上。

因此，若想优化 bootstrapping 中的大线性变换，一个最自然的想法就是：

不要把 U 当成一个整体一次性实现，而是把它拆成很多更简单、更稀疏的小矩阵的乘积。

11.1.2 为什么“直接实现一个大矩阵”很贵

设 U 是一个 $\ell \times \ell$ 的 dense 矩阵。若直接用 weighted rotations 去实现，那么通常会面临：

- 需要较多种不同的 rotation；
- 每个 rotation 后都要做 key switching；
- 每个对角线方向都要配一组权重掩码；
- 许多项之间共享结构不明显。

从矩阵角度看，dense 意味着每一行都依赖很多输入槽；从 FHE 角度看，这意味着：

一次“直接的大矩阵乘法”会展开成大量 rotation + mask + sum。

因此，即使 BSGS 已经能减少 rotation 数量，若矩阵本身仍然太“密”，它仍会是开销大头。

11.1.3 FFT 的经验：大密集矩阵往往隐藏着递归结构

在经典快速 Fourier 变换（FFT）中，长度为 n 的 DFT 矩阵本来是一个 dense 的 Vandermonde 矩阵：

$$(\omega^{ij})_{0 \leq i, j < n}.$$

如果把它当成普通矩阵实现，复杂度约为 $O(n^2)$ 。但 FFT 发现它可以拆成若干层非常稀疏的“蝶形矩阵”，从而把复杂度降到 $O(n \log n)$ 。

这启发我们：若 BGV/BFV 中的 U 也具有某种 roots-of-unity 的递归结构，那么它很可能也能做类似分解。

11.1.4 为什么 BGV/BFV 中的 U 值得期待类似结构

在 sparse packed 中， U 的条目典型形如

$$U_{ik} = y_i^k, \quad y_i = \zeta_{m,i}^c,$$

因此它本质上就是一个 roots-of-unity 型矩阵。在 fully packed 中，外层 U 也有类似形状，例如条目常写作

$$U_{ik} = \zeta_{m,i}^{dk}.$$

因此， U 虽然来自 BGV/BFV 的 CRT 结构，但从矩阵形状上看，它和 FFT/DFT 类矩阵非常相似。这正是“尝试 FFT-like 分解”的基础。

11.1.5 分解 U 的真正目标

分解 U 不是为了写法更漂亮，而是为了实现层收益。真正目标包括：

- (1) 让每一层只涉及很少的非零模式；
- (2) 让每层对应较少的 rotation；
- (3) 让各层更适合 BSGS、hoisting；
- (4) 让整体变换具有“分治 / 递归”结构，便于复杂度分析与实现。

因此，本章的主线可以概括为：

把 U 从“一个大而密的求值矩阵”，改写成“很多层稀疏蝶形矩阵的乘积”。

11.2 列置换版本的意义

11.2.1 为什么要先做列置换

在前面对 BGV/BFV 的 U 矩阵讨论中，我们已经提到过一个容易让人疑惑的现象：

文献有时并不直接对当前定义下的 U 做分解，而是先改用它的一个 列置换版本。

这一步看起来像“只是把列重新排个序”，似乎不影响本质。但实际上，它非常关键。原因是：

某些矩阵虽然本质上与标准 FFT 矩阵等价，但只有在合适的行/列顺序下，递归分块结构才会清晰易见。

11.2.2 列置换在矩阵论中的一般含义

设 $U \in M_\ell(\mathbb{F})$, P 是一个置换矩阵。则

$$UP$$

就是把 U 的列按某种顺序重新排列得到的新矩阵。

从线性映射角度看：

- U 与 UP 表示的是同一个线性变换家族中的两种坐标写法；
- 置换矩阵 P 只改变输入坐标顺序；
- 不会改变这个变换的“本质难度”，但可能改变其显式结构。

11.2.3 为什么列顺序会影响递归分块

设一个矩阵原本是由

$$1, y_i, y_i^2, \dots, y_i^{\ell-1}$$

这些幂次构成的。若列按自然顺序排列，则你看到的是一个标准 Vandermonde 形式。但 FFT 风格的分治通常更喜欢把列索引按“偶数幂 / 奇数幂”或更一般的“分块幂次模式”排序。

例如，把

$$1, y_i, y_i^2, y_i^3, y_i^4, y_i^5, \dots$$

重排成

$$1, y_i^2, y_i^4, \dots ; y_i, y_i^3, y_i^5, \dots$$

后，矩阵就会自然分裂成“偶幂块 + 奇幂块”的结构，这正是 FFT 蝶形的起点。

11.2.4 BGV/BFV 里为什么特别需要列置换

在 BGV/BFV 中， U 的条目虽然是 roots-of-unity 的幂，但它最初的列顺序常常是由“系数位置”或“slot 索引顺序”自然诱导出来的，而不是由递归分治最适合的顺序诱导出来的。

因此若直接看它，可能只看到一个大而密的 Vandermonde 型矩阵；而一旦把列按合适方式重排，原本隐藏的结构会显现为：

- 上下块共享某种相同子矩阵；
- 右半块只是左半块乘一个简单根因子；
- 整个矩阵可写成“两个小问题 + 一个蝶形块”的乘积。

11.2.5 列置换不会改变 bootstrapping 的数学目标

这点必须强调。若 U 的某个列置换版本写成

$$\tilde{U} = UP,$$

那么：

- $\tilde{U}$ 不是“另一个完全不同的变换”；
- 它只是把输入坐标重新编号后的同一个问题；
- 最终若需要回到原始系数顺序，只需再乘上相应的置换逆即可。

因此列置换的意义主要是：

把问题摆成最适合分解的样子。

11.2.6 一个最简单的例子

考虑长度 4 的 Vandermonde 型矩阵：

$$V = \begin{bmatrix} 1 & y_0 & y_0^2 & y_0^3 \\ 1 & y_1 & y_1^2 & y_1^3 \\ 1 & y_2 & y_2^2 & y_2^3 \\ 1 & y_3 & y_3^2 & y_3^3 \end{bmatrix}.$$

若把列重排成“偶幂在前、奇幂在后”：

$$\tilde{V} = \begin{bmatrix} 1 & y_0^2 & y_0 & y_0^3 \\ 1 & y_1^2 & y_1 & y_1^3 \\ 1 & y_2^2 & y_2 & y_2^3 \\ 1 & y_3^2 & y_3 & y_3^3 \end{bmatrix},$$

那么后两列会明显看成“前两列乘一个 y_i ”:

$$(y_i, y_i^3) = y_i \cdot (1, y_i^2).$$

这就把矩阵显式分成了“偶部分”和“奇部分”。

这正是后面蝶形分解的最核心线索。

11.3 蝶形分解思想

11.3.1 为什么叫“蝶形”

在 FFT 文献中, 所谓 butterfly (蝶形) 指的是这样一种二维小块结构:

$$\begin{bmatrix} 1 & \omega \\ 1 & -\omega \end{bmatrix},$$

或者在块矩阵层面写成

$$\begin{bmatrix} I & W \\ I & -W \end{bmatrix}.$$

其形状对应于“把一个问题拆成两个子问题, 再通过加减和少量 twiddle factor (根因子) 组合起来”的模式。

从图上看, 输入两支线在中间交叉后输出两支线, 像蝴蝶翅膀, 因此得名。

11.3.2 从偶奇分裂得到蝶形

考虑一个多项式

$$q(Y) = a_0 + a_1Y + \cdots + a_{\ell-1}Y^{\ell-1}.$$

若按偶奇幂次分组, 则可写成

$$q(Y) = q_{\text{even}}(Y^2) + Y \cdot q_{\text{odd}}(Y^2),$$

其中

$$q_{\text{even}}(Z) = a_0 + a_2Z + a_4Z^2 + \cdots,$$

$$q_{\text{odd}}(Z) = a_1 + a_3Z + a_5Z^2 + \cdots.$$

现在在点 $Y = y$ 上求值, 就得到

$$q(y) = q_{\text{even}}(y^2) + y \cdot q_{\text{odd}}(y^2).$$

若再考虑点 $-y$, 则有

$$q(-y) = q_{\text{even}}(y^2) - y \cdot q_{\text{odd}}(y^2).$$

因此, 一对点 y 与 $-y$ 的值可由同一对子问题

$$q_{\text{even}}(y^2), q_{\text{odd}}(y^2)$$

通过一个 2×2 的蝶形块组合得到。

11.3.3 矩阵形式

把上面两式写成矩阵：

$$\begin{bmatrix} q(y) \\ q(-y) \end{bmatrix} = \begin{bmatrix} 1 & y \\ 1 & -y \end{bmatrix} \begin{bmatrix} q_{\text{even}}(y^2) \\ q_{\text{odd}}(y^2) \end{bmatrix}.$$

这就是最基本的蝶形块。

若对所有成对的点同时做这件事，就会得到块矩阵形式：

$$\begin{bmatrix} I & W \\ I & -W \end{bmatrix},$$

其中 W 是由这些根因子组成的对角矩阵。

11.3.4 为什么这正是 U 分解的核心

BGV/BFV 中的 U 之所以能做 FFT-like 分解，根本原因正是：它对应的求值问题可以沿偶/奇幂次（或更一般的递归分块）不断拆成规模减半的子问题。

每一次拆分都会产生：

- (1) 两个规模减半的子矩阵；
- (2) 一个很稀疏的蝶形组合矩阵。

因此整个大矩阵就可递归写成：

$$\text{大问题} = \text{稀疏蝶形层} \cdot \text{两个小问题}.$$

11.3.5 与 unpack 的 even/odd splitting 的相似性

这里很值得强调一个“跨章节的统一视角”。

- unpack 中，我们对 slot 内部幂次做 even/odd splitting；
- FFT-like 分解中，我们对求值矩阵的列/幂次结构做 even/odd splitting。

二者看起来用途不同：

- unpack 是在拆 slot 内部坐标；
- FFT-like 分解是在拆大矩阵的求值结构；

但背后的数学模式是同一个：

$$f(T) = f_{\text{even}}(T^2) + T f_{\text{odd}}(T^2).$$

这说明 BGV/BFV bootstrapping 中出现的许多“分治结构”其实共享同一代数骨架。

11.4 稀疏矩阵乘积表示

11.4.1 从大矩阵到递归乘积

设 $\tilde{U}$ 是 U 的某个适合分解的列置换版本。蝶形分解告诉我们, $\tilde{U}$ 通常可以写成

$$\tilde{U} = B_1 \begin{bmatrix} U_1 & 0 \\ 0 & U_2 \end{bmatrix},$$

其中:

- B_1 是一个非常稀疏的蝶形矩阵;
- U_1, U_2 是两个规模减半的子问题矩阵。

若 U_1, U_2 本身又有相同结构, 就可继续递归:

$$U_1 = B_2^{(1)} \cdot \text{diag}(U_{1,1}, U_{1,2}),$$

$$U_2 = B_2^{(2)} \cdot \text{diag}(U_{2,1}, U_{2,2}),$$

最终得到

$$\tilde{U} = B_1 B_2 \cdots B_r,$$

其中每个 B_t 都是非常稀疏的矩阵。

11.4.2 为什么说“稀疏”

这里“稀疏”有两层含义:

- (1) **普通矩阵意义下稀疏**: 每一行、每一列只含少量非零块;
- (2) **FHE 实现意义下稀疏**: 每层矩阵只涉及少数几种 rotation 方向, 因此 weighted rotations 实现会显著更便宜。

最典型的蝶形层

$$\begin{bmatrix} I & W \\ I & -W \end{bmatrix}$$

在块矩阵意义下每行只涉及两个块, 因此是高度稀疏的。

11.4.3 一个抽象递归定理

定理 11.1 (FFT-like 递归分解的抽象形式). 设一族矩阵 $\{U_n\}$ 满足:

(1) U_n 是规模为 n 的 *roots-of-unity* 型求值矩阵;

(2) 在适当列置换后, 可写成

$$\tilde{U}_n = B_n \begin{bmatrix} U_{n/2} & 0 \\ 0 & U'_{n/2} \end{bmatrix},$$

其中 B_n 是一个稀疏蝶形矩阵;

(3) 该结构可递归继续到规模 1。

则 $\tilde{U}_n$ 可写成 $\log_2 n$ 层稀疏矩阵的乘积。

证明. 按规模 n 做归纳。当 $n = 1$ 时结论显然成立。若对 $n/2$ 成立, 则

$$U_{n/2} = B_2^{(1)} \cdots B_r^{(1)}, \quad U'_{n/2} = B_2^{(2)} \cdots B_r^{(2)}.$$

代回

$$\tilde{U}_n = B_1 \begin{bmatrix} U_{n/2} & 0 \\ 0 & U'_{n/2} \end{bmatrix},$$

可得 $\tilde{U}_n$ 由第一层蝶形矩阵 B_1 和后续递归层叠而成, 总层数随每次规模减半而增加 1, 故为 $\log_2 n$ 级别。□

11.4.4 为什么这一定理足够适用于 bootstrapping 语境

在 bootstrapping 的实现中, 我们并不总需要把矩阵写成某个完全显式的封闭公式。更重要的是:

- 它能否递归拆成若干层;
- 每层是否稀疏;
- 每层是否适合 weighted rotations + BSGS 实现;
- 这些层之间是否便于插入 hoisting 和 mask 优化。

只要满足这些性质, 就已经足够支撑高效实现。

11.5 为什么这种分解有利于同态实现

11.5.1 从一次“大求和”变成很多次“小求和”

若直接实现一个 dense 的 U ，往往需要：

- 较多种不同 rotation；
- 较密集的对角权重；
- 一次性很大的加总结构。

而稀疏分解把它拆成很多层

$$U = B_1 B_2 \cdots B_r,$$

每一层 B_t 只涉及少量 rotation 模式。因此你不再是在做“一个巨大复杂的矩阵乘法”，而是在做：

很多层规则、局部、低稀疏度的小线性变换。

11.5.2 为什么少量模式特别重要

在 FHE 实现中，真正昂贵的是：

- rotation；
- rotation 后的 key switching；
- 为不同 rotation 模式准备的辅助密钥与预处理。

如果某层矩阵只涉及两三种固定模式，那么：

- 所需 rotation 种类少；
- BSGS 更好组织；
- hoisting 更容易共享中间结果；
- 实现逻辑也更规整。

11.5.3 层数增加为什么不一定是坏事

乍看之下，把一个矩阵拆成 $\log n$ 层似乎增加了步骤数。但关键是：

- 每层远比原始矩阵稀疏；
- 每层的 rotation 数可显著减少；
- 很多层之间可共享结构；
- 对角权重更简单；
- 更适合与 BSGS/hoisting 组合。

因此整体代价通常会下降，而不是上升。

11.5.4 和 FFT 的复杂度直觉一致

这与经典 FFT 的思想完全一致：

- 直接 DFT：一个大 dense 矩阵；
- FFT：分成很多层蝶形矩阵；
- 虽然层数增多，但每层极稀疏，总复杂度下降。

在 FHE 里，虽然复杂度度量从标量乘加变成了 rotation / key switching / plaintext multiplication，但“用递归稀疏结构降低整体代价”的思想并没有改变。

11.5.5 对 bootstrapping 的直接好处

在 CoeffToSlot、SlotToCoeff、以及某些 unpack/repack 的实现中，这种分解会直接带来：

- 更少的 rotation 类型；
- 更少的有效 automorphism 次数；
- 更适合 BSGS 的分组；
- 更高的 hoisting 复用率；
- 更易做层间融合与常数优化。

因此它是现代高效 bootstrapping 线性层设计的核心工具之一。

11.6 与 CKKS decoding matrix 分解的关系

11.6.1 为什么人们总说 “CKKS-like”

在前面的章节中我们已经反复看到：BGV/BFV 中的某些 U 矩阵虽然来自 CRT/slot 分解，但其外形与 CKKS 中的 decoding / encoding 矩阵非常相似：

- 都是 roots-of-unity 的幂构成的矩阵；
- 都具有 Vandermonde/FFT 型结构；
- 都适合递归蝶形分解。

因此，很多 BGV/BFV 工作会说某一步是“CKKS-like”的。这不是说语义相同，而是说：

它们在矩阵结构和实现技巧上高度相似。

11.6.2 CKKS decoding matrix 的经典结构

在 CKKS 中，slot-to-coefficient 与 coefficient-to-slot 通常由某种复根上的求值/插值矩阵给出。这些矩阵在合适的重排后，标准地具有 FFT 蝶形分解，因此可以递归拆成若干层稀疏矩阵。

这给了后续 BGV/BFV 工作一个强烈启发：如果自己的 U 在合适重排下与 CKKS 的某个标准矩阵同型，那么就能直接复用 CKKS 里的分解套路。

11.6.3 BGV/BFV 与 CKKS 在这一步上的共同点

二者在矩阵分解层面的共同点包括：

- (1) 都是 roots-of-unity 型求值矩阵；
- (2) 都能通过列重排显式暴露偶/奇分裂；
- (3) 都能写成“两个子问题 + 一个蝶形块”的递归形式；
- (4) 都适合再叠加 BSGS 与 hoisting。

因此，从“怎么实现线性层”的角度看，BGV/BFV 确实能大量借鉴 CKKS 的经验。

11.6.4 但它们并不完全相同

尽管结构上相似，BGV/BFV 与 CKKS 在本质上仍有重要差别：

- CKKS 的 slots 来源于 canonical embedding / 复数根值；
- BGV/BFV 的 slots 来源于 CRT / slot algebra 分解；
- CKKS 中每个 slot 更像一个标量（复数）；
- BGV/BFV fully packed 中每个 slot 可能是一般 E -元素，因此还要考虑 M 、 M^{-1} 、unpack、repack 等额外结构。

因此可以说：

BGV/BFV 在矩阵层像 CKKS，在明文语义层却并不等同于 CKKS。

11.6.5 为什么这种“像但不等于”恰好最有价值

如果两者完全不同，就无法共享实现技巧；如果两者完全相同，那就没有必要单独研究 BGV/BFV 的特点。恰恰是这种“矩阵层高度相似、语义层明显不同”的关系，使得：

- 线性层优化可借鉴 CKKS；
- 但整体 bootstrapping 流程仍必须按 BGV/BFV 自己的 digit extraction / unpack / repack 需求来设计。

11.7 BGV/BFV 中“像 FFT 但不完全一样”的地方

11.7.1 相似之处：都在做分治的 roots-of-unity 求值

从矩阵形式看，BGV/BFV 的 U 与经典 FFT/DFT 最相似的地方在于：

- 都由 roots-of-unity 的幂构成；
- 都有偶/奇幂次递归；
- 都可拆成蝶形网络；
- 都能产生 $\log n$ 层的稀疏矩阵乘积。

因此说“FFT-like”完全合理。

11.7.2 不同之处一：roots of unity 的来源不同

在经典 FFT 中，根通常是复数域中的单位根。在 CKKS 中，roots-of-unity 也主要在复数嵌入意义下解释。而在 BGV/BFV 中，这些根来自：

- 模 p 的 Frobenius 轨道；
- 模 p^e 的 Hensel 提升；
- 或对应的 Galois ring / slot algebra 结构。

也就是说，表面上都是“根的幂”，但底层代数环境不同。

11.7.3 不同之处二：fully packed 时还多了一层内部基结构

经典 FFT 中，一个“频率槽”基本就是一个标量。BGV/BFV sparse packed 也差不多如此。但 fully packed 中，一个 slot 是

$$\sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j$$

这样的 E_i -元素。

因此对 BGV/BFV fully packed 而言：

- U 只负责外层 slot 混合；
- M 、 M^{-1} 负责局部基与统一基之间的切换；
- unpack / repack 则进一步处理 slot 内部坐标。

这就使得整个流程比经典 FFT 多了一整层“slot 内部结构”。

11.7.4 不同之处三：rotation 实现受 good/bad dimension 影响

在经典 FFT 里，“索引重排”只是索引重排。而在 BGV/BFV 中，一个 rotation 是否便宜，取决于：

- 它沿哪一维发生；
- 那一维是 good 还是 bad；
- 一个 rotation 是否对应一个 automorphism 还是两支 automorphism + mask。

因此即使矩阵看起来和 FFT 一样，其实际实现代价模型却更复杂。

11.7.5 不同之处四：FHE 中层数与稀疏度的权衡不同

FFT 在经典数值分析中主要关心标量算术复杂度。而在 FHE 中，除了“非零个数”以外，还必须关心：

- 需要多少次 automorphism；
- 需要多少次 key switching；
- 是否可 hoist；
- 对角权重是否可共享；
- 乘法层数是否增加。

因此“最像 FFT 的分解”未必就是“FHE 中最优的分解”；有时还要与 BSGS、hoisting、mask 结构一起平衡。

11.7.6 一个统一理解

可以把 BGV/BFV 里的 FFT-like 结构总结成：

它们在矩阵递归结构上像 FFT，在实现代价模型上却比 FFT 更复杂。

这也是为什么 bootstrapping 文献中常既借鉴 FFT 语言，又必须加入 automorphism、key switching、good/bad dimension、BSGS、hoisting 等额外层。

11.8 本章小结

本章从矩阵结构角度解释了为什么 BGV/BFV 中的大线性变换可以进一步做稀疏分解与 FFT-like 优化。主要结论如下：

- (1) 分解 U 矩阵的根本目的，不是形式上的好看，而是为了把一个大而密的线性变换改写成很多层规则、稀疏、适合 FHE 实现的小线性变换。
- (2) 列置换版本的意义在于：它能把原本隐藏的偶/奇分裂结构显式暴露出来，使矩阵更像标准 FFT 矩阵，从而便于递归分块。
- (3) 蝶形分解的核心来自恒等式

$$q(Y) = q_{\text{even}}(Y^2) + Yq_{\text{odd}}(Y^2),$$

它使得根值求值问题可以递归拆成两个规模减半的子问题和一个稀疏蝶形块。

(4) 因此，适当重排后的 U 可写成若干层稀疏矩阵的乘积：

$$\tilde{U} = B_1 B_2 \cdots B_r,$$

其中每层 B_t 都非常稀疏。

- (5) 这种分解之所以有利于同态实现，是因为每层只涉及少量 rotation 模式，更适合与 BSGS、hoisting 结合，从而降低整体 rotation / key switching 成本。
- (6) BGV/BFV 的 U 在矩阵结构上与 CKKS decoding matrix 非常相似，因此大量线性层优化思路可以借鉴 CKKS；但两者的明文语义和 fully packed 额外结构并不相同。
- (7) 所以 BGV/BFV 中的这些结构可以称为“FFT-like”：它们在递归矩阵分解上像 FFT，但在 FHE 实现代价模型上又比经典 FFT 更复杂。

下一章将继续讨论 *fully packed* 与 *sparse packed* 的优化路线，系统总结：

- 为什么 sparse packed 更简单；
- 为什么 fully packed 更贵；
- thin/full bootstrap 该如何权衡；
- 子环优化、trace、coefficient selection 的角色；
- 以及 partial CoeffToSlot 这类“只打开一部分”的新思路。

fully packed 与 sparse packed 的优化路线

12.1 sparse packed 为什么更简单

12.1.1 从对象类型上看：slot 是否是标量

在前面的章节中，我们已经建立了一个最核心的区分：

- **sparse packed**：每个 slot 中装的是一个标量

$$m_i \in \mathbb{Z}_{p^e};$$

- **fully packed**：每个 slot 中装的是一个一般的 slot algebra 元素

$$m_i \in E_i, \quad \dim_{\mathbb{Z}_{p^e}} E_i = d.$$

这一区别一旦放进 bootstrapping 流程里，就会立刻导致复杂度分叉。对 sparse packed 而言，CoeffToSlot 之后得到的已经是标量槽，因此后续 digit extraction 可以直接逐槽进行。而 fully packed 则必须先把 E -slot 拆成标量槽，这会引入 unpack / repack 及其全部额外成本。

因此，从最根本的输入对象类型看，sparse packed 更简单的原因只有一句话：

它一开始就把后续非线性步骤最想看到的对象——标量槽——直接给出来了。

12.1.2 从线性变换结构上看：是否需要 M 与 M^{-1}

在 sparse packed 中，由于每个 slot 本来就是标量，因此：

- 不存在“局部幂基 vs 统一幂基”的差异；
- 不需要在 slot 内部先统一坐标再做外层混合；
- CoeffToSlot / SlotToCoeff 通常退化成单个 U 矩阵及其逆。

因此，sparse packed 的线性层结构可概括为：

一个子环上的 Vandermonde / CRT 变换.

而 fully packed 则必须处理：

$$M, \quad U, \quad M^{-1},$$

甚至与 unpack / repack 交织。因此 sparse packed 在线性变换层面天然更“薄”，更接近标准 FFT/CRT 场景。

12.1.3 从子环视角看：问题维数更小

上一章已经说明，sparse packed 明文天然落在某个子环

$$R'_{p^e} \cong \mathbb{Z}_{p^e}[Y]/(Y^{N'} + 1), \quad Y = X^c.$$

在最常见的稀疏打包情形下，该子环维数往往就是 slot 数 ℓ 。因此：

- 大环 R_{p^e} 的维数是 N ；
- sparse 子环 R'_{p^e} 的维数是 $N' = \ell$ 或同数量级；
- 所以 sparse packed 的很多核心变换只需在更小子环上完成。

这意味着：

sparse packed 不仅 slot 更简单，而且整个线性代数问题本身也更小。

12.1.4 从 bootstrapping 流程上看：中间环节更少

对 sparse packed 而言，一条典型的 bootstrapping 流程更接近：

$$\text{同态线性解密} \rightarrow \text{CoeffToSlot} \rightarrow \text{digit extraction} \rightarrow \text{SlotToCoeff}.$$

而 fully packed 则通常是：

$$\text{同态线性解密} \rightarrow \text{CoeffToSlot} \rightarrow \text{unpack} \rightarrow \text{digit extraction} \rightarrow \text{repack} \rightarrow \text{SlotToCoeff}.$$

多出来的 unpack / repack 不是常数级小修小补，而是整块大成本。因此，sparse packed 的“简单”不是来自某个局部技巧，而是来自整条流程更短。

12.1.5 从实现层面看：更容易吃满现有优化

BSGS、hoisting、FFT-like 分解这些优化，本来就最适合处理：

- 结构简单的 roots-of-unity 型线性层；
- 没有额外内部基切换的场景；
- 直接作用于标量槽的情况。

因此 sparse packed 往往更容易把这些优化吃满，而 fully packed 则必须在这些优化之外，再额外付出内部结构重组的成本。

12.1.6 一个总结性判断

可以把 sparse packed 的优势总结为三层：

- (1) **对象更简单**：slot 是标量；
- (2) **空间更小**：问题天然落在子环里；
- (3) **流程更短**：无需 unpack / repack。

因此，当研究者想先把 bootstrapping 做出来、做快、做稳时，sparse packed 往往是最自然的起点。

12.2 fully packed 为什么更贵

12.2.1 最根本的原因：吞吐量不是免费的

fully packed 的吸引力在于它使用了整个明文环的全部自由度：

$$N = \ell d.$$

也就是说，相比 sparse packed，它在一个密文里塞进了更多信息。

但这个额外吞吐量不会凭空得到。它带来的代价本质上是：

你不是在刷新“ ℓ 个标量”，而是在刷新“ ℓ 个 d -维小环元素”。

因此，fully packed 更贵并不是偶然，而是高吞吐量本身要求你同时维护更复杂的代数结构。

12.2.2 第一层代价：slot 内部基结构

在 fully packed 中，每个 slot 写成

$$m_i = \sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j.$$

这立刻带来两个问题：

- 不同 slot 用的是不同局部幂基；
- 若想做统一外层变换，必须先统一成标准幂基。

于是你必须引入：

$$M, \quad U, \quad M^{-1}.$$

这说明 fully packed 的 CoeffToSlot / SlotToCoeff 从一开始就比 sparse packed 多一层难度。

12.2.3 第二层代价：digit extraction 不能直接作用

digit extraction 想面对的是：

$$u_{i,j} = m_{i,j} + p^e r_{i,j}$$

这样的标量关系。但 fully packed 的 slot 值首先是 E_i -元素，不是标量。因此必须先：

$$E\text{-slot} \xrightarrow{\text{unpack}} \text{scalar-slot 流} \xrightarrow{\text{digit extraction}} \text{修复后的 scalar-slot 流} \xrightarrow{\text{repack}} E\text{-slot}.$$

也就是说，digit extraction 前后都要额外做一次“大规模结构重组”。

12.2.4 第三层代价：automorphism / key switching 爆炸

fully packed 中最贵的附加代价，往往不是某个单点公式，而是 automorphism 的数量和层次显著增加。

原因包括：

- M 、 M^{-1} 往往都要实现为大线性变换；
- unpack / repack 本质上是一棵递归的线性分裂/重组树；
- 每层都需要 automorphism + key switching；
- 若某些维度属于 bad dimension，则一个理想 rotation 甚至需要两支 automorphism + mask 拼接。

因此 fully packed 常见的成本放大路径是：

更多结构 $\Rightarrow$ 更多 rotation $\Rightarrow$ 更多 key switching $\Rightarrow$ 更重的 bootstrapping.

12.2.5 第四层代价：层数与中间密文数量

fully packed 通常还会额外增加：

- 中间密文的数量；
- 需要维护的分支数；
- 需要暂存的 sparse 流数；
- 临时结果融合的复杂度。

例如，若内部维数 d 较大，则 unpack 后会产生 d 条或同数量级的标量流。这会对：

- 内存；
- 调度；
- 并行实现；
- 后续 repack；
- 以及整体工程复杂度

都施加显著压力。

12.2.6 第五层代价：优化空间更碎片化

对于 sparse packed，很多优化都可以统一地围绕一个 U 矩阵来做。而 fully packed 的优化则要分散到：

- M ；
- U ；
- M^{-1} ；
- unpack；
- repack；
- 中间 digit extraction 接口；

- 各层之间是否可融合。

这意味着：

fully packed 的优化不仅更难，而且更难做到“一个技巧通杀”。

12.2.7 总结：fully packed 的成本本质

可以把 fully packed 为什么更贵压缩成一句话：

它不是简单地“多装了一点数据”，而是把 bootstrapping 从“刷新标量槽”升级成了“刷新带内部代数结构的槽”，因此需要在整个流程中不断维护、拆开、恢复这份内部结构。

12.3 何时值得做 full bootstrap

12.3.1 问题不是“能不能做”，而是“值不值”

从理论上讲，只要参数允许、辅助密钥齐全、线性层和 digit extraction 都能实现，就可以做 fully packed bootstrap。真正的问题通常不是“能否完成”，而是：

为了保住 fully packed 吞吐量，付出的额外代价是否值得。

因此 full bootstrap 的价值判断，本质上是一个吞吐量与复杂度的 trade-off。

12.3.2 当后续计算强依赖高吞吐 SIMD 时

若应用场景后续还有大量逐槽并行运算，尤其是：

- 深层神经网络中的大批量线性层；
- 多轮多项式或查找表近似；
- 需要长期保持高并行度的批量推理；
- 数据天然“宽向量”结构，且每一层都能吃满 slot。

那么 fully packed 往往值得做，因为：

- 一次 bootstrapping 虽贵；
- 但刷新后仍保持最大并行度；
- 后面很多层计算都能摊薄这次 bootstrapping 成本。

换句话说，若刷新之后还有大量“值得满槽利用”的工作，那么 full bootstrap 就更合理。

12.3.3 当 ciphertext 数量本身是瓶颈时

若不做 full bootstrap，而改用 thin/sparse 路线，往往意味着：

- 每个密文承载的信息减少；
- 同一批数据需要用更多密文表示；
- 后续加法、乘法、旋转、通信、存储都可能随密文数量增加。

因此，当系统的真正瓶颈在“密文数量太多”而非“单次 bootstrap 太贵”时，full bootstrap 反而可能更优。

12.3.4 当参数规模允许有效 amortization 时

若：

- slot 数很大；
- d 不是过于夸张；
- BSGS、hoisting、FFT-like 分解、融合实现都已做好；
- bootstrapping 的固定成本能被很多 payload 分摊；

那么 fully packed 的“每消息平均成本”可能会显著下降。此时，即使绝对时间更大，平均到每个标量消息上的代价仍可能是有竞争力的。

12.3.5 当应用需要保持表示一致性时

某些系统中，整个计算图或协议设计默认始终使用 fully packed 的表示约定。若中途切到 sparse/thin 表示，再切回来，可能会引入：

- 额外的数据重排；
- 编码接口复杂度；
- 多种表示混杂的工程维护成本；
- 后续层难以无缝衔接。

这种情况下，尽管 full bootstrap 本身更贵，但它能维持系统表示的一致性，从整体软件架构上看也可能值得。

12.3.6 一个经验性判断

若满足下列多数条件，则 full bootstrap 往往是有意义的：

- (1) 后续还有很多层高并行计算；
- (2) 每个密文的满槽利用率长期较高；
- (3) 密文数量本身是系统瓶颈；
- (4) 参数与优化足以摊薄 fully packed 的额外代价；
- (5) 工程上希望全程维持一种统一表示。

12.4 何时 thin / sparse 更合适

12.4.1 当你最在乎的是“刷新便宜”而不是“刷新后最满”

thin / sparse 路线最大的优势是：它们接受“刷新后不保持 fully packed 最大吞吐量”这一点，从而避开 fully packed 中最昂贵的结构维护。

因此当目标是：

- 尽量便宜地刷新一次；
- 尽量降低 bootstrapping 延迟；
- 尽量缩小中间状态与辅助密钥复杂度；

那么 thin / sparse 往往更有吸引力。

12.4.2 当后续计算不需要满槽并行时

若 bootstrapping 之后的剩余计算并不多，或者其本身并不能很好利用 fully packed 结构，例如：

- 只需再做少量线性/非线性处理；
- 后续层本身宽度较小；
- 数据本来就稀疏；
- 算法逻辑不要求长期维持满并行度；

那么为 full bootstrap 额外付出的成本往往摊不回来。此时 thin / sparse 的“简单、快、轻”通常更实用。

12.4.3 当 d 太大时

fully packed 最大的额外成本往往随着 d 增长而放大，因为：

- M 、 M^{-1} 更复杂；
- unpack / repack 层数约为 $\log_2 d$ ；
- 需要拆出来的标量流更多；
- 中间密文数量增加；
- 实现与调度复杂度也上升。

因此当 d 很大时，即使理论上可以 fully packed，实践上也可能更适合 thin / sparse 路线。

12.4.4 当想优先验证新 bootstrapping 原型时

在研究与工程原型阶段，thin / sparse 路线常常是优先选择，因为：

- 数学与实现更简单；
- 更容易独立验证 digit extraction 的正确性；
- 更容易把线性层优化吃透；
- 更适合逐步增加复杂度。

因此很多工作先在 sparse packed 场景把 CoeffToSlot / SlotToCoeff 或 digit extraction 做到很强，再逐步向 fully packed 推进。

12.4.5 当系统更关心“峰值时延”而非“平均吞吐”时

full bootstrap 往往能提高长期平均吞吐，但单次 bootstrap 的时延通常更高。若应用是交互式或时延敏感型，可能更关心：

- 单次刷新尽快完成；
- 中间内存尽量低；
- 系统峰值负载更平滑。

在这种情况下，thin / sparse 经常更合适。

12.4.6 一个经验性判断

若满足下列多数条件，则 thin / sparse 更合适：

- (1) 后续计算深度不大，或难以吃满 fully packed 吞吐；
- (2) d 较大，fully packed 结构重组过贵；
- (3) 更在意单次 bootstrapping 时延；
- (4) 想降低实现复杂度与中间状态数量；
- (5) 研究/工程阶段更希望先验证核心非线性而非全部表示维护。

12.5 子环优化

12.5.1 为什么子环是 sparse 路线的天然舞台

上一章已经说明，sparse packed 明文天然落在子环

$$R'_{p^e} \cong \mathbb{Z}_{p^e}[Y]/(Y^{N'} + 1), \quad Y = X^c.$$

这意味着：

- 你不必总在大环 R_{p^e} 上做所有事；
- 某些变换与 bootstrapping 子步骤可直接在更小的 R'_{p^e} 上完成；
- 这会同时减小维数、矩阵规模和 rotation 范围。

因此，子环并不是一个“代数上的巧合”，而是 sparse/thin 优化路线的基本平台。

12.5.2 子环优化带来的直接收益

12.5.2.1 维数缩小

若大环维数为 N ，子环维数为

$$N' = \frac{N}{c},$$

则许多线性变换和辅助操作都只需在 N' -维空间中完成，而不是在 N -维空间中完成。

12.5.2.2 矩阵规模缩小

例如 sparse packed 中的 U 矩阵规模通常只与 ℓ 或 N' 相关，而不是整个 N 。这意味着：

- FFT-like 分解更浅；
- BSGS 的基础规模更小；
- 需要的 rotation 种类更少。

12.5.2.3 辅助密钥压力下降

若只需要支持子环上的部分 automorphism 或特定 trace 结构，则所需的 rotation key / switching key 也可能减少。

12.5.3 为什么子环优化与 sparse packed 高度绑定

在 fully packed 中，你希望用满整个环的自由度，因此很难一直停留在一个更小子环里。但 sparse packed 本来就只占用了大环中的一部分结构，因此它自然适合“直接在那部分结构上做文章”。

这也是为什么 thin / sparse bootstrap 的很多工作，本质上都在问：

能否在更小子环里完成尽可能多的刷新逻辑，而不是每一步都升回大环。

12.5.4 子环优化并不等于“信息丢失”

这里容易有一个误解：“既然只在子环里工作，是不是把原消息的一部分丢掉了？”

答案取决于你的 packing 约定：

- 若你一开始就只把消息编码在子环对应的 sparse 结构里，那么在子环里工作不会丢失信息；
- 真正丢失的是“原本本可以装在 fully packed 中但你没有使用的自由度”。

因此子环优化不是“把已编码的信息丢弃”，而是“从一开始就只使用对当前任务必要的那部分代数空间”。

12.5.5 与本文主题的直接关系

对于与你的论文相关的 fully packed / sparse packed 对比来说，子环优化就是两条路线分叉的最核心代数表现：

- sparse packed: 主动利用

$$R'_{p^e}$$

来换取实现简洁；

- fully packed: 坚持停留在整个

$$R_{p^e}$$

中，以换取最大吞吐。

所以，子环优化不是一个边缘技巧，而是整个 sparse/thin 设计哲学的数学核心。

12.6 trace / coefficient selection 的作用

12.6.1 为什么会出现 trace

在子环优化路线中，经常需要做一件事：

把大环中的信息“压到”一个更小的子环，或者把若干 slot/internal 结构汇总到标量层。

trace 正是完成这件事的一个非常自然的代数工具。

12.6.2 抽象定义

若 E/F 是一个有限代数扩张或更一般的有限自由模扩张，则 trace 映射定义为：

$$\mathrm{Tr}_{E/F}(x) = x + \sigma(x) + \sigma^2(x) + \cdots$$

其中 σ 是适当的生成自同构（在有限域/有限环 setting 中可对应 Frobenius 或某个 Galois 群生成元）。

在 bootstrapping 语境里，它的作用通常是：

沿 slot 内部或某个扩张方向，把多个共轭分量线性汇总成更小层次的对象。

12.6.3 在 BGV/BFV 中它为什么重要

在 BGV/BFV 的 sparse/thin 路线中，trace 常出现在两类位置：

- (1) **把大环压到子环**：当你只希望保留某个子环上的有效信息时，trace 是一个自然的“压缩”映射；
- (2) **把 E -slot 再压到 $\mathbb{Z}_{p^e}$** ：在某些情形下，仅仅做了 CoeffToSlot 还不足以让 slot 变成真正的标量，这时可能要再做逐槽 trace。

12.6.4 coefficient selection 是什么

除了 trace 之外，还有一类常见操作可概括为 coefficient selection。它的目标不是“把所有信息求和”，而是：

从一个更大的表示中选出我们真正关心的那一部分系数或子分量。

例如：

- 只保留子环中允许出现的稀疏幂次；
- 只保留某些 digit/block；
- 只保留某个 outer polynomial 的系数部分。

从线性代数看，这常对应于：

- 某种掩码；
- 某个稀疏投影矩阵；
- 或 trace 与置换的组合。

12.6.5 为什么 trace / selection 对 thin 路线特别重要

thin / sparse 路线的关键目标是：

只刷新“需要保留的那部分结构”，而不是维护整个 fully packed 结构。

因此：

- trace 负责把冗余方向压缩掉；
- coefficient selection 负责把无关部分筛掉；
- 二者一起使 bootstrapping 的工作量与“真正使用的自由度”对齐。

12.6.6 一个结构性理解

可以把 trace / coefficient selection 视为 thin 路线中的“信息压缩层”：

大环 / fully packed 结构 $\rightarrow$ 更小子环 / 更少自由度 $\rightarrow$ 更便宜的 bootstrapping 核心步骤.

这也是为什么很多工作在介绍 thin bootstrap 时，总会在 CoeffToSlot 前后或 digit extraction 前后嵌入 trace 类操作。

12.7 partial CoeffToSlot 的概念与用途

12.7.1 为什么会想到“只做一半 CoeffToSlot”

完整 CoeffToSlot 的目标是把 coefficient view 中的 noisy plaintext 完整打开成标准 slot 表示：

$$u(X) \mapsto (u_1, \dots, u_\ell).$$

但这会立刻引出两个问题：

- 完整打开本身很贵；
- 后续的核心步骤有时并不真的需要“最标准、最完整”的 slot 表示。

于是一个自然的问题是：

能否只把 coefficient 表示打开到“足够用”的程度，而不是完全打开到底？

这就是 partial CoeffToSlot 背后的基本思想。

12.7.2 partial CoeffToSlot 的概念

所谓 partial CoeffToSlot，可以粗略理解为：

不是把 noisy plaintext 一次性完全变成最终标准 slot 向量，而是只把它变成某种中间、部分展开、但已经足够供下一步非线性使用的表示。

因此它既不是“做错了一半”，也不是“近似地做 CoeffToSlot”，而是：

故意只做必要的那一部分。

12.7.3 为什么这样做可能更便宜

完整 CoeffToSlot 的成本往往包含：

- 完整的 Vandermonde/FFT 型线性变换；
- 完整的 basis unification / basis restore；
- 可能还要为 fully packed 结构服务。

而若后续步骤只需要：

- 部分 slots；

- 部分 blocks;
- 某种半展开表示;
- 或某个更适合 blind rotation / digit extraction 的中间结构;

那么完整 CoeffToSlot 就做过头了。partial CoeffToSlot 通过停在中间层, 避免支付多余成本。

12.7.4 它的用途在哪里

partial CoeffToSlot 的典型用途包括:

- (1) **与下一步核心非线性融合**: 若后续 digit extraction / blind rotation / selection 只依赖部分 slot-like 结构, 就没必要先做完整转换;
- (2) **减少完整线性层代价**: 省去一部分 FFT-like 层、rotation、key switching;
- (3) **作为 thin/sparse 路线中的中间接口**: 既不维持 fully packed 全结构, 也不必一次性压缩到底。

12.7.5 与本文主题的关系

对于本报告所围绕的 fully packed 与 sparse packed 优化路线来说, partial CoeffToSlot 的意义在于:

- 它体现了一种“不要把表示转换做满”的哲学;
- 它通常更偏向 thin / sparse 思路, 而不是传统 fully packed 完整打开思路;
- 它说明 bootstrapping 的瓶颈并不总在 digit extraction 本身, 有时完整 CoeffToSlot 已经做得太重了。

因此, partial CoeffToSlot 可看成:

在 full 与 thin 之间寻找更细粒度折中方案的一种代表性思路。

12.7.6 一个统一理解

如果说:

- full bootstrap 追求“完整恢复 fully packed 结构”;
- thin/sparse bootstrap 追求“只保留必要自由度”;

那么 partial CoeffToSlot 体现的就是:

在表示转换层面, 尽量只打开到下一步真正需要的层次。

这使它成为近年 bootstrapping 优化中很自然、也很有前景的一条思路。

12.8 本章小结

本章围绕 fully packed 与 sparse packed 两条优化路线，系统分析了它们的取舍逻辑与代数基础。主要结论如下：

- (1) **sparse packed 更简单**，因为它从一开始就把后续非线性想要处理的对象——标量槽——直接给出来了；同时它天然落在子环中，线性变换规模更小，流程更短。
- (2) **fully packed 更贵**，因为它不仅要刷新更多信息，还必须在整个流程中维护、统一、拆开、恢复每个 slot 的内部 d -维代数结构，这会带来 M/M^{-1} 、unpack/repack、更多 automorphism 与 key switching。
- (3) **full bootstrap 值得做的场景**包括：后续仍有大量高吞吐 SIMD 计算、密文数量本身是瓶颈、参数足以摊薄 fully packed 的固定成本、以及系统希望全程保持统一表示。
- (4) **thin / sparse 更合适的场景**包括：更在意单次刷新便宜、后续并不需要满槽吞吐、 d 太大导致 fully packed 结构重组过贵、以及原型阶段更希望先把核心刷新逻辑做简单做稳。
- (5) **子环优化**是 sparse/thin 路线的数学核心：它把 bootstrapping 从整个大环 R_{p^e} 压到更小的 R'_{p^e} ，从而减小维数、线性层规模和辅助密钥压力。
- (6) **trace / coefficient selection** 是 thin 路线中的信息压缩工具：前者负责沿某些扩张方向线性汇总，后者负责只保留真正有用的系数/子结构。
- (7) **partial CoeffToSlot** 体现了一种新的折中思路：不再把 coefficient 表示完整打开成最终标准 slot，而只打开到后续步骤真正需要的层次，以减少多余的线性层成本。

下一章将进入 复杂度与成本来源，对整个 BGV/BFV bootstrapping 流程做统一的代价拆解，分析：

- 乘法深度；
- automorphism 数量；
- plaintext-ciphertext multiplication 数量；
- key switching 成本；
- fully packed 与 sparse packed 的复杂度差别；
- 以及 BSGS、FFT-like 分解、unpack/repack、digit extraction 分别占了多少“账”。

复杂度与成本来源

13.1 评估指标

在前面的章节中，我们已经从代数结构、流程结构与线性变换实现三个角度理解了 BGV/BFV bootstrapping。到了这一章，我们要把这些抽象结构统一翻译成更“工程”和“实现”的问题：

到底什么在决定一次 bootstrapping 的成本？

与经典算法复杂度分析只关心“加法次数、乘法次数”不同，FHE 中的复杂度必须同时看多个维度。原因是：

- 有些操作对时间昂贵；
- 有些操作对模数层数昂贵；
- 有些操作对辅助密钥与内存昂贵；
- 有些操作虽然是线性的，但因为要做 automorphism / key switching，反而很重。

因此，分析 bootstrapping 复杂度时，至少要看以下四类指标。

13.1.1 乘法深度

13.1.1.1 定义

在 FHE 里，最重要的“深度”通常不是加法深度，而是乘法深度，即需要串行堆叠多少层非平凡乘法。

更具体地说，若某个电路中存在一条从输入到输出的路径，沿途经过了 L 层“不可并行压缩”的乘法，那么这个电路的乘法深度就是 L 。

13.1.1.2 为什么乘法深度特别关键

原因在于：乘法深度直接决定了需要预留多少模数预算。在 leveled FHE 中，每增加一层乘法，噪声和模数消耗通常都会显著上升。因此 bootstrapping 的一个核心任务就是：

- 自己先消耗一部分深度；
- 但最终把密文刷新到一个“重新可用”的状态。

这意味着，bootstrapping 本身的乘法深度越低，就越容易在给定参数下实现。

13.1.1.3 哪些步骤主要贡献乘法深度

在完整 bootstrapping 流程中：

- CoeffToSlot / SlotToCoeff 多数情况下主要是线性层，对乘法深度贡献较小或为常数级；
- unpack / repack 多为线性组合与 automorphism，对乘法深度贡献也不算主导；
- 真正主导乘法深度的，通常是 digit extraction 及其内部的非线性电路。

因此，讨论“bootstrapping 深不深”时，本质上常常是在讨论：

digit extraction 的非线性电路到底有多深。

13.1.1.4 一个重要提醒

虽然 automorphism / key switching 不一定显著增加“乘法深度”，但它们非常消耗时间与资源。所以“深度低”并不自动意味着“实现便宜”。

13.1.2 automorphism 数量

13.1.2.1 为什么要单独数 automorphism

在普通线性代数里，一个矩阵乘法只是矩阵乘法。但在 FHE 中，大多数大线性变换都被实现为：

$$\text{weighted rotations} = \sum_v \kappa(v) \rho^v.$$

而每个 ρ^v 底层通常都依赖某个 automorphism τ_j 。因此，automorphism 的数量往往是时间成本的最直接来源之一。

13.1.2.2 一个 automorphism 背后包含什么

一个 automorphism 的真实代价通常包括：

- (1) 对密文系数做 $a(X) \mapsto a(X^j)$;
- (2) 将密文从新秘密钥 $\tau_j(s)$ 切回标准秘密钥 s ;
- (3) 这一步又通常需要一次 key switching。

因此“做一次 rotation”在现实中常常意味着“做一次 automorphism + 一次 key switching”。

13.1.2.3 为什么 automorphism 数量比矩阵非零数更重要

一个大矩阵若写成 weighted rotations，即使总非零很多，只要能共享较少种 rotation 模式，实际代价可能仍然可控。反之，若需要很多彼此不同的 automorphism，即使每个都只配很简单的 mask，整体仍然会很重。

因此，在线性层复杂度评估中，研究者常常先问：

需要多少种不同的 rotation / automorphism?

而不是先问矩阵总共有多少非零项。

13.1.3 plaintext-ciphertext multiplication 数量

13.1.3.1 为什么明文乘法也要单独记

在 FHE 里，密文与明文的乘法（plaintext-ciphertext multiplication）虽然比密文与密文乘法便宜，但并不免费。在 bootstrapping 中，这类操作大量出现在：

- weighted rotations 中的对角掩码 $\kappa(v)$;
- unpack / repack 的偶奇选择掩码;
- trace / coefficient selection;
- 各种辅助线性组合。

因此，一个线性变换即使 rotation 数量已经通过 BSGS 压低，若仍需要大量明文乘法，整体成本依然可观。

13.1.3.2 为什么它们特别适合和线性层一起统计

因为在很多实现中，线性层的基本原子操作就是：

$$\text{rotation} + \text{plaintext multiplication} + \text{sum}.$$

因此数 plaintext-ciphertext multiplication 的数量，实际上是在量化：

这层线性变换到底需要多少“按位置加权”的操作。

13.1.3.3 与 digit extraction 中明文乘法的区别

在 digit extraction 中，也可能出现一些明文乘法，但那里更多是非线性电路的一部分。而在线性层中，plaintext-ciphertext multiplication 的意义更偏向“结构性加权”。因此，在总复杂度统计里，把二者放在同一个桶里往往会掩盖问题来源，最好分开理解。

13.1.4 key switching 代价

13.1.4.1 为什么 key switching 是独立的大头

在许多 FHE 实现中，key switching 常常是最重的基本操作之一。它不仅出现在乘法后的 relinearization 中，更密集地出现在：

- rotation / automorphism 之后；
- 某些层间辅助切换中；
- 多轮线性层优化的中间步骤里。

在 bootstrapping 中，因为线性层本身就要大量用 rotation，所以 key switching 往往成为线性层成本的主要组成部分。

13.1.4.2 为什么不能只数“多少次 key switching”

严格来说，只看次数还不够，因为一次 key switching 的代价还取决于：

- 当前密文模数大小；
- decomposition base 的选择；
- 是否使用 hoisting；
- 是否能共享某些中间 decomposition；
- 当前密文维数与参数规模。

因此,“key switching 代价”更准确地说是:

key switching 次数 $\times$ 单次 key switching 单价.

13.1.4.3 为什么 hoisting 特别重要

正是因为 key switching 很贵, 所以 hoisting / double-hoisting 变得极其重要。它们做的不是减少逻辑上必须的 rotation, 而是:

把很多 rotations 共有的 decomposition 和切换前处理摊平。

因此在复杂度表里, 常常会同时看到:

- rotation 数量;
- key switching 数量;
- hoisting 后的有效 key switching 成本。

13.2 哪些步骤最贵

13.2.1 不能只问“哪一步最慢”

bootstrapping 的成本来源不是单一的, 因此“哪一步最贵”也没有永远统一的答案。它依赖于:

- 是 BGV 还是 BFV;
- 是 sparse packed 还是 fully packed;
- digit extraction 电路的具体设计;
- 线性层是否做了 FFT-like 分解、BSGS、hoisting;
- 当前参数规模、slot 数、 d 、模数链长度。

不过, 在大多数现代 packed bootstrapping 设计中, 重成本通常集中在以下几块。

13.2.2 第一类大头: CoeffToSlot / SlotToCoeff

这两步常常是“大线性变换”最直接的来源。其成本主要包括:

- 大量 rotation;
- 每个 rotation 后的 key switching;
- 对角权重的 plaintext multiplication;
- 若未优化好, 矩阵稠密度导致的高常数因子。

对 sparse packed 而言, 这通常对应于 U 或 U^{-1} 的实现。对 fully packed 而言, 成本还会进一步叠加 M 与 M^{-1} 的基变换部分。

13.2.3 第二类大头: digit extraction

从乘法深度和真正的非线性复杂度角度, digit extraction 往往是最核心的大头。其成本主要包括:

- 逐位或逐块提取的非线性电路;
- carry / correction;
- 中间值范围控制;
- 多轮组合与重建。

因此, 若你问“哪一步最决定 bootstrapping 的理论可实现深度”, 答案通常是 digit extraction。

13.2.4 第三类大头: unpack / repack

在 fully packed 中, 这两步经常成为额外的重成本来源。它们虽然主要是线性的, 但会带来:

- 递归层数 $\log_2 d$;
- 较多 automorphism;
- 中间密文数量膨胀;
- 与其它步骤难以彻底融合的结构开销。

因此, 在 fully packed 系统里, 线性层大头往往不止 CoeffToSlot / SlotToCoeff, 还包括 unpack / repack。

13.2.5 不同语境下的“最贵”不同

可以把“最贵步骤”分成三种语境：

- (1) 从乘法深度看：digit extraction 最贵；
- (2) 从 rotation / key switching 看：CoeffToSlot、SlotToCoeff、unpack、repack 常最贵；
- (3) 从 fully packed 额外负担看：unpack / repack 是最具结构惩罚性的部分。

13.2.6 所以成本分析应分层做

这也是为什么严谨的 bootstrapping 分析，不会只给一个“总时间”数字，而是尽量拆成：

- 线性层；
- 非线性层；
- fully packed 附加层；
- key switching 摊销层。

否则很难真正看清优化该往哪里发力。

13.3 fully packed 与 sparse packed 的复杂度对比

13.3.1 先给一个总判断

从总体上看：

fully packed 一般比 sparse packed 更贵

但这句话必须细化理解。它的意思不是“fully packed 绝对不值得做”，而是说：

为了保住更高吞吐量，fully packed 必须额外支付一整套结构维护成本。

13.3.2 线性层规模的差别

sparse packed. 问题天然落在子环 R'_{p^e} 中。因此 CoeffToSlot / SlotToCoeff 主要是规模约为 ℓ 的线性变换，常可直接看成某个 U 与 U^{-1} 。

fully packed. 线性层不仅包括 U ，还包括：

- 局部幂基与统一幂基之间的 M ；
- 恢复局部幂基的 M^{-1} ；
- unpack / repack 的递归线性层；
- 某些情形下还要额外结合 trace 或 recombination。

因此 fully packed 的线性层复杂度不是 sparse packed 的一个常数倍扩展，而是结构上更复杂的一大块。

13.3.3 rotation / key switching 数量的差别

sparse packed. 通常主要承担：

- U 层的 rotations；
- digit extraction 前后必要的组织；
- 较少的附加结构性 rotations。

fully packed. 除了上面的外层 U 部分，还要额外承担：

- M 、 M^{-1} 的 rotations；
- unpack / repack 多层递归的 rotations；
- 若某些维度是 bad dimension，则每个理想 rotation 的单价也会更高。

因此在 rotation 数量与有效 key switching 成本上，fully packed 往往显著更大。

13.3.4 中间密文数量的差别

sparse packed. 中间态通常仍维持为一条或少量几条密文流。数据结构相对扁平。

fully packed. 在 unpack 之后，可能会临时生成 d 条或同数量级的 scalar-slot 流，再在 repack 时回收。这对：

- 内存；
- 调度；
- 并行；

- 中间缓存;
- 工程实现

都会带来明显压力。

13.3.5 平均每消息成本不一定谁更高

这是一个必须强调的细节。虽然 fully packed 的总 bootstrapping 成本几乎总更高，但它承载的 payload 也更大。因此若按“每个消息槽的平均成本”衡量，情况就不一定单调。

也就是说：

- sparse packed：总成本低，但每个密文装的信息少；
- fully packed：总成本高，但每次刷新更多消息。

因此真实比较要看：

$$\text{总成本} \quad \text{vs.} \quad \frac{\text{总成本}}{\text{每次刷新消息数}}.$$

13.3.6 一个结构性总结

可以用下面这个表述概括二者的区别：

sparse packed 更像“先把问题做小再刷新”，fully packed 更像“维持最大吞吐地完整刷新”。

前者节省的是绝对复杂度，后者优化的是长期吞吐与信息密度。

13.4 BSGS 带来的下降

13.4.1 从朴素实现到 BSGS

设某一维线性变换写成

$$L(m) = \sum_{v=0}^{n-1} \kappa(v) \rho^v(m).$$

若朴素实现，则 rotation 数大约是 n 。而 BSGS 分块

$$n = n_1 n_2$$

后，可把 rotation 数降到大约

$$n_1 + n_2.$$

当

$$n_1 \approx n_2 \approx \sqrt{n}$$

时，复杂度量级变成

$$O(\sqrt{n}).$$

13.4.2 为什么这是 bootstrapping 里最实在的收益之一

在 bootstrapping 的许多线性层里， n 可能是：

- 某个 slot 维度 ℓ_i ；
- 整个 sparse packed slot 数 ℓ ；
- 某层递归结构的局部规模。

因此，BSGS 直接带来的不是一个抽象“渐近更好”，而是：

automorphism 数量显著下降，进而带动 key switching 成本下降。

13.4.3 不是所有成本都按 $\sqrt{n}$ 降

要注意，BSGS 主要压的是 rotation 结构。它并不会自动让下列成本同样按 $\sqrt{n}$ 降：

- digit extraction 的乘法深度；
- 某些与 slot 内部结构无关的固定成本；
- bad dimension 下单次 rotation 的额外 mask 成本；
- 未被 hoisting 摊掉的其它 decomposition 成本。

因此 BSGS 的正确理解应该是：

它主要优化“线性层中的 rotation 组织”，而不是整个 bootstrapping 的所有部分。

13.4.4 配合 hoisting 后的实际下降

若再叠加 hoisting，则 BSGS 不仅降低 rotation 数量，还能让更多 rotations 共享 decomposition 结果。于是总收益通常体现在：

- rotation 种类下降；
- key switching 的平均单价下降；
- 明文权重应用的组织更规整；

- 临时结果复用更多。

因此在实际实现中，研究者更关心的是：

BSGS + hoisting 后的有效 key switching 总成本.

13.4.5 对 full 与 sparse 路线的意义不同

sparse packed. 由于核心线性层更干净，BSGS 带来的收益通常更接近理论值。

fully packed. 虽然 U 部分同样能被 BSGS 有效优化，但 M 、 M^{-1} 、unpack、repack 等额外结构会稀释“总流程层面”的收益。

因此，BSGS 对 fully packed 很重要，但它无法单独解决 fully packed 的全部复杂度问题。

13.5 unpack / repack 的额外成本

13.5.1 为什么这是 fully packed 的“额外账”

在 sparse packed 路线中，digit extraction 前后不需要做 slot 内部结构重组。因此 unpack / repack 这一整块成本根本不存在。

而在 fully packed 中，它们是纯额外开销：不是 digit extraction 本身的逻辑必需，而是为了让 digit extraction 可作用于标量槽，不得不支付的结构转换代价。

13.5.2 递归层数带来的成本

若内部维数为 d ，则 unpack / repack 的递归层数大约是

$$\log_2 d.$$

每一层都要做：

- automorphism;
- mask;
- 线性组合;
- 往往还伴随 key switching。

因此其成本不会随着 d 线性增长，而是具有“层数 $\times$ 分支数”的递归结构。

13.5.3 中间态爆炸的成本

unpack 的一个现实代价是中间密文数量可能膨胀。若最终要拆成 d 条 scalar-slot 流, 那么中途会经历多层分支展开。这会增加:

- 峰值内存;
- 调度复杂度;
- 中间结果管理成本;
- 后续 repack 的同步负担。

因此 unpack / repack 的复杂度不能只按“总算术操作数”看, 还要看中间态管理。

13.5.4 为什么它们常成为 fully packed 的主惩罚项

在许多 fully packed 方案中, 哪怕你已经把 U 部分做得很优化、BSGS 和 hoisting 用得很好, unpack / repack 仍会留下明显的固定结构成本。

因此, 在全流程比较中, fully packed 相比 sparse packed 的额外成本, 往往很大一部分就体现在这里。

13.6 digit extraction 的成本占比

13.6.1 为什么要单独看占比

digit extraction 本身是 bootstrapping 的核心非线性步骤。但它在总成本中到底占多大比例, 并不是固定不变的。这取决于:

- 线性层是否已被高度优化;
- fully packed 的结构开销是否显著;
- digit extraction 是逐位、逐块还是其它电路;
- p, e 与 block 大小如何选择。

因此, 在做复杂度评估时, 常常需要单独讨论:

digit extraction 在总成本中占多大头。

13.6.2 从深度看：它通常是第一大头

若按乘法深度来衡量，则 digit extraction 几乎总是最主要的部分。原因是：

- CoeffToSlot / SlotToCoeff 多为线性层；
- unpack / repack 也主要是线性结构变换；
- digit extraction 则要真正评估非线性函数。

因此若你关心“参数是否足以支撑这次 bootstrap”，digit extraction 往往是最关键的深度决定项。

13.6.3 从运行时间看：它未必总是第一大头

若线性层非常大、rotation 很多、key switching 很贵，那么在总运行时间中，digit extraction 不一定永远占最大比例。尤其是在 fully packed 方案中：

- unpack / repack 可能非常重；
- $M/U/M^{-1}$ 线性层可能消耗大量 rotation；
- digit extraction 虽深，但未必占绝对多数时间。

因此在实践中常见两种情况：

- (1) 理论深度视角：digit extraction 最重；
- (2) 实际时间视角：线性层与 key switching 也可能与之相当甚至超过。

13.6.4 为什么优化 digit extraction 仍然最关键

即便它不是总时间中唯一最大的部分，digit extraction 仍然极重要，因为：

- 它决定可实现的乘法深度；
- 它决定 bootstrapping 的核心正确性；
- 其它步骤大多是线性层，可借助 BSGS/FFT-like/hoisting 持续优化；
- digit extraction 的非线性本质更难被“通用技巧”彻底消解。

因此，很多 bootstrapping 论文真正的创新点，最终还是会落到“怎样更便宜地完成这一非线性 residue 恢复”上。

13.7 参数选择对复杂度的影响

13.7.1 参数不是只影响安全性，也直接影响 bootstrapping 成本

在 FHE 中，人们常先想到参数与安全性相关，例如 N, Q 决定 RLWE 安全强度。但对于 bootstrapping 来说，参数还深刻影响实现代价：

- slot 数；
- slot 内部维数；
- 线性层规模；
- digit extraction 深度；
- rotation / key switching 单价；
- fully packed 是否现实。

因此参数选择不是“先确定安全参数再算运行时间”那么简单，而是一个与算法设计耦合的整体问题。

13.7.2 $d = \text{ord}_m(p)$ 的影响

d 是最关键的结构参数之一。它决定：

- 每个 slot 的内部维数；
- slot 数
$$\ell = \frac{N}{d};$$
- fully packed 时是否需要复杂的 unpack / repack；
- sparse packed 子环步长 c 的大小；
- M 、 M^{-1} 的复杂度。

因此：

- d 小：slot 数多，内部结构浅，fully packed 更容易；
- d 大：slot 数少，但内部结构复杂，fully packed 成本迅速上升。

13.7.3 ℓ 的影响

slot 数 ℓ 决定了 many linear layers 的“外层规模”。例如：

- sparse packed 的 U 矩阵规模通常与 ℓ 有关；
- BSGS 的基础长度 n 常常就是 ℓ 或其某维因子；
- more slots 通常意味着更高吞吐，但也意味着更大的外层变换矩阵。

因此 ℓ 增大往往同时带来：

- 更高 payload；
- 更大的 CoeffToSlot / SlotToCoeff 线性层。

13.7.4 p^e 的影响

明文模数

$$p^e$$

直接决定 digit extraction 的难度。

若 e 较大：需要恢复更多低位 digits 或更多 blocks，因此 digit extraction 电路通常更深或更大。

若 p 较大：单个 digit/block 的范围变大，识别与 correction 可能更复杂。

因此 p^e 不仅决定消息空间大小，也直接决定核心非线性步骤的代价。

13.7.5 Q 与 bootstrapping 模数链的影响

更大的 Q 意味着：

- 有更多空间容纳 bootstrapping 电路；
- 但每次运算、尤其 key switching 的单价变高；
- decomposition 长度可能增加；
- 内存与辅助密钥体积也增大。

因此“给更大的 Q ”并不是免费解法，而是拿单次运算成本去换可实现深度。

13.7.6 good / bad dimension 相关参数的影响

slot 索引群如何分解成多维循环结构，也会影响复杂度。同样的 ℓ ，若分解成：

- 更多 good dimensions；
- 更少 bad dimensions；
- 更适合 BSGS 的维长度组合；

则整体 rotation 与 key switching 成本会明显降低。

这说明，参数选择不只是数值规模问题，也包括“slot 几何的好坏”。

13.7.7 一个统一视角

可以把参数对复杂度的影响分成三层：

- (1) **代数结构层**： d, ℓ, c, m', N' 决定 slot 结构与子环结构；
- (2) **非线性层**： p, e 决定 digit extraction 难度；
- (3) **实现层**： Q 、slot geometry、good/bad dimension 决定 rotation / key switching 单价与优化空间。

因此真正好的参数，不只是“安全”或“可实现”，而是：

能在代数结构、非线性深度与实现代价之间给出平衡。

13.8 本章小结

本章从复杂度角度统一分析了 BGV/BFV bootstrapping 的成本来源。主要结论如下：

- (1) 评估 bootstrapping 复杂度不能只看一个指标，至少要同时关注：
 - 乘法深度；
 - automorphism 数量；
 - plaintext-ciphertext multiplication 数量；
 - key switching 代价。
- (2) 从“哪一步最贵”来看，要分层讨论：
 - digit extraction 通常主导乘法深度；

- $\text{CoeffToSlot} / \text{SlotToCoeff}$ 、 $\text{unpack} / \text{repack}$ 往往主导 $\text{rotation} / \text{key switching}$ 成本；
 - fully packed 的额外结构维护会显著抬高总成本。
- (3) fully packed 与 sparse packed 的复杂度差异，不只是常数倍区别，而是结构上是否需要维护 slot 内部代数结构的根本区别。
- (4) BSGS 的核心收益是把一维长度为 n 的 $\text{weighted rotations}$ 从 $O(n)$ 级 rotation 组织成 $O(\sqrt{n})$ 级别；再配合 hoisting ，可进一步降低有效 key switching 成本。
- (5) $\text{unpack} / \text{repack}$ 是 fully packed 的“额外账”，其递归结构、分支膨胀与 automorphism 数量往往构成 fully packed 相对 sparse 路线最显著的附加成本。
- (6) digit extraction 的成本占比在不同实现中可能不同，但它几乎总是理论深度上的第一大头，也是 bootstrapping 的核心非线性难点。
- (7) 参数选择对复杂度影响深刻：
- d, ℓ 决定 slot 几何；
 - p^e 决定 digit extraction 难度；
 - Q 决定可实现深度与单次运算单价；
 - $\text{good/bad dimension}$ 决定 rotation 的结构成本。

下一章将进入 BGV 、 BFV 、 CKKS 三者的 bootstrapping 对照，把我们前面建立的全部框架放到三个主流方案的横向比较中，重点解释：

- 三者共同的“同态解密”骨架；
- BGV 的 residue reduction 、 BFV 的 scale-and-round 、 CKKS 的 EvalMod 之间的根本差别；
- $\text{CoeffToSlot} / \text{SlotToCoeff}$ 在三者中的相同点与不同点；
- 以及为什么 CKKS 的命名传统特别容易让人混淆。

伪代码附录：从原始数据到 Bootstrapping 再回到原始数据

A.1 统一记号与对象约定

为避免伪代码中来回切换记号，本附录统一采用以下约定。

A.1.1 基础参数

- $m = 2N$: power-of-two cyclotomic 指数;
- $R_q := \mathbb{Z}_q[X]/(X^N + 1)$;
- $t := p^e$: 明文模数;
- $d = \text{ord}_m(p)$;
- $\ell = N/d$: slot 数;
- $Q_0 \rightarrow Q_1 \rightarrow \cdots \rightarrow Q_L$: 模数链, Q_0 最大, Q_L 最小;
- Q_{boot} : bootstrapping 阶段使用的提升模数。

A.1.2 打包模式

本附录允许两种 packing 模式:

- **sparse packed**: 输入数据为

$$\mathbf{m} = (m_0, \dots, m_{\ell-1}), \quad m_i \in \mathbb{Z}_t;$$

- **fully packed**: 输入数据为

$$\mathbf{M} = (m_{i,j})_{0 \leq i < \ell, 0 \leq j < d}, \quad m_{i,j} \in \mathbb{Z}_t,$$

其中第 i 个 slot 表示为

$$m_i = \sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j \in E_i.$$

A.1.3 方案标志

伪代码中使用一个方案标志

$$\text{scheme} \in \{\text{BGV}, \text{BFV}\}.$$

其中:

- BGV 的核心 noisy plaintext 关系写作

$$u = m + tr;$$

- BFV 的核心 noisy plaintext 关系写作

$$u = \Delta m + e, \quad \Delta \approx Q/t.$$

A.1.4 辅助对象

我们将使用下列辅助对象:

- **pk**: 公钥;
- **sk**: 秘密钥;
- **rlk**: 重线性化密钥;
- **rotk**[j]: 对应 automorphism τ_j 的 rotation / key switching 密钥;
- **btk**: bootstrapping key (用于同态线性解密);
- **EncMeta**: 编码与 slot 几何元数据;
- **LinMeta**: CoeffToSlot、SlotToCoeff、unpack、repack 所需的线性变换元数据;
- **DigitMeta**: digit extraction 或 BFV scale-and-round 所需的电路/块信息。

A.1.5 密文类型

我们统一把密文写成

$$\mathbf{ct} = (c_0, c_1) \in R_Q^2$$

或在乘法中间态临时出现的高阶分量形式

$$(c_0, c_1, \dots, c_k).$$

经过重线性化后, 最终仍压回标准二元密文。

A.2 参数生成与预处理

A.2.1 算法 A.1: SetupBGVBFVBootstrap

输入

- 安全参数 λ ;
- 目标方案 $\text{scheme} \in \{\text{BGV}, \text{BFV}\}$;
- 明文素数 p 与指数 e , 即 $t = p^e$;
- 目标计算深度 DepthTarget ;
- 打包模式 $\text{PackingMode} \in \{\text{Sparse}, \text{Full}\}$;
- 期望支持的 rotation 集合 $\mathcal{R} \subseteq \mathbb{Z}_m^\times$;
- 是否启用 full bootstrap / thin bootstrap 的模式标记 BootMode 。

输出

- 公共参数 Params ;
- 密钥 (pk, sk) ;
- 评估密钥 $\text{EvalKeys} = (\text{rlk}, \text{rotk}, \text{btk})$;
- 编码元数据 EncMeta ;
- 线性层元数据 LinMeta ;
- 核心非线性元数据 DigitMeta 。

步骤

1. 选择 power-of-two cyclotomic 维数 N , 令

$$m := 2N, \quad R_q := \mathbb{Z}_q[X]/(X^N + 1).$$

要求 (N, Q_0) 满足安全参数 λ 。

2. 计算

$$d := \text{ord}_m(p), \quad \ell := N/d.$$

3. 选择普通计算模数链

$$Q_0 \rightarrow Q_1 \rightarrow \cdots \rightarrow Q_L$$

使其足以支持 `DepthTarget` 的 leveled 计算。

4. 选择 bootstrapping 模数 Q_{boot} ，要求其足以支持：

- 同态线性解密；
- `CoeffToSlot` / `SlotToCoeff`；
- (若 `PackingMode=Full`) `unpack` / `repack`；
- BGV 的 digit extraction，或 BFV 的 scale-and-round。

5. 因式分解

$$X^N + 1 = F_1(X) \cdots F_\ell(X) \quad \text{in } \mathbb{Z}_t[X],$$

并构造 CRT 分量

$$E_i := \mathbb{Z}_t[X] / (F_i(X)).$$

6. 构造 slot 索引集 $S \subset \mathbb{Z}_m^\times / \langle p \rangle$ 的一组代表元，并计算 slot 几何分解：

$$\mathbb{Z}_m^\times / \langle p \rangle \cong C_{\ell_1} \times \cdots \times C_{\ell_r}.$$

记录生成元 $(g_1, \dots, g_r)$ 与各维长度 $(\ell_1, \dots, \ell_r)$ 。

7. 若 `PackingMode=Sparse`，构造 sparse packed 子环

$$R'_t \cong \mathbb{Z}_t[Y] / (Y^{N'} + 1), \quad Y = X^c,$$

其中 $c = d$ 或 $d/2$ （按具体情形决定）。

8. 预计算编码/解码所需数据：

- sparse packed 的 CRT 基 / Vandermonde 型矩阵 U 及其逆；
- fully packed 的局部幂基、统一幂基、 M 、 M^{-1} 、 U ；
- 必要时的 trace / selection / 子环投影矩阵。

9. 为 BSGS 预计算线性层分块信息：

- 各线性变换对应的 weighted rotations；
- 每一维的 BSGS 分块 (n_1, n_2) ；
- good dimension / bad dimension 标记；
- 需要的掩码与对角权重。

10. 为 digit extraction (BGV) 或 scale-and-round (BFV) 选择具体策略并预计算元数据:

- base- p 或 base- p^k ;
- 块大小 $B = p^k$;
- 所需低位块数 $h = \lceil e/k \rceil$;
- 低位提取电路 / correction 电路参数;
- BFV 下所需的 Δ 、rounding 信息。

11. 采样秘密钥 $\mathbf{sk} = s \in R_{Q_0}$ 。

12. 生成公钥 $\mathbf{pk}$ (标准 RLWE 方式)。

13. 生成重线性化密钥 $\mathbf{rlk}$ 。

14. 对每个所需 rotation / automorphism 索引 $j \in \mathcal{R}$, 生成

$$\mathbf{rotk}[j].$$

15. 生成 bootstrapping key $\mathbf{btk}$, 使得可同态评估

$$c_0 + c_1 s \quad \text{或更一般的} \quad \sum_i c_i s^i.$$

16. 输出:

$$\mathbf{Params}, (\mathbf{pk}, \mathbf{sk}), \mathbf{EvalKeys}, \mathbf{EncMeta}, \mathbf{LinMeta}, \mathbf{DigitMeta}.$$

A.3 编码与解码

A.3.1 算法 A.2: EncodeData

输入

- 打包模式 $\mathbf{PackingMode} \in \{\mathbf{Sparse}, \mathbf{Full}\}$;
- 原始数据:
 - sparse packed: $\mathbf{m} = (m_0, \dots, m_{\ell-1}) \in \mathbb{Z}_t^\ell$;
 - fully packed: $\mathbf{M} = (m_{i,j})_{0 \leq i < \ell, 0 \leq j < d} \in \mathbb{Z}_t^{\ell \times d}$;
- 编码元数据 $\mathbf{EncMeta}$ 。

输出

- 明文多项式 $m(X) \in R_t$ 。

步骤

1. 若 PackingMode=Sparse, 则:

- 1.1. 把输入标量槽视为 sparse 子环上的 slot 数据:

$$(m_0, \dots, m_{\ell-1}).$$

- 1.2. 用预计算的 sparse SlotToCoeff 变换 (等价于子环上的 CRT 逆映射) 求出

$$q(Y) \in R'_t.$$

- 1.3. 再通过 $Y = X^c$ 代换得到大环中的明文

$$m(X) \in R_t.$$

2. 若 PackingMode=Full, 则:

- 2.1. 对每个 slot i , 构造局部 slot 元素

$$\mu_i := \sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j \in E_i.$$

- 2.2. 若实现按统一幂基组织, 则先把 μ_i 映到标准拷贝 E 的统一幂基表示。

- 2.3. 对向量

$$(\mu_0, \dots, \mu_{\ell-1})$$

应用 fully packed 的 SlotToCoeff 线性变换, 即

$$M^{-1}UM$$

或其等价实现。

- 2.4. 得到明文多项式

$$m(X) \in R_t.$$

3. 输出 $m(X)$ 。

A.3.2 算法 A.3: DecodeData

输入

- 明文多项式 $m(X) \in R_t$;
- 打包模式 PackingMode $\in \{\text{Sparse}, \text{Full}\}$;
- 编码元数据 EncMeta。

输出

- 若 PackingMode=Sparse, 输出

$$(m_0, \dots, m_{\ell-1}) \in \mathbb{Z}_t^\ell;$$

- 若 PackingMode=Full, 输出

$$(m_{i,j})_{0 \leq i < \ell, 0 \leq j < d} \in \mathbb{Z}_t^{\ell \times d}.$$

步骤

1. 对 $m(X)$ 应用 plaintext 版 CoeffToSlot。
2. 若 PackingMode=Sparse, 则直接读取每个 slot 的标量值

$$m_i \in \mathbb{Z}_t.$$

3. 若 PackingMode=Full, 则:

- 3.1. 得到每个 slot 的 E_i -元素

$$\mu_i \in E_i;$$

- 3.2. 把 μ_i 展开到局部幂基

$$\mu_i = \sum_{j=0}^{d-1} m_{i,j} \zeta_{m,i}^j;$$

- 3.3. 读出系数 $m_{i,j} \in \mathbb{Z}_t$ 。

4. 输出原始数据数组。

A.4 加密与解密

A.4.1 算法 A.4: Encrypt

输入

- 方案标志 $\text{scheme} \in \{\text{BGV}, \text{BFV}\}$;
- 公钥 $\text{pk} = (a, b)$;
- 明文多项式 $m(X) \in R_t$;
- 当前工作模数 Q 。

输出

- 密文 $\mathbf{ct} = (c_0, c_1) \in R_Q^2$ 。

步骤

1. 采样小噪声 / 小随机元

$$u, e_0, e_1 \in R_Q.$$

2. 若 $\text{scheme}=\text{BGV}$, 令

$$c_0 := b \cdot u + te_0 + m, \quad c_1 := a \cdot u + te_1.$$

3. 若 $\text{scheme}=\text{BFV}$, 设

$$\Delta := \left\lfloor \frac{Q}{t} \right\rfloor,$$

并令

$$c_0 := b \cdot u + e_0 + \Delta m, \quad c_1 := a \cdot u + e_1.$$

4. 输出

$$\mathbf{ct} = (c_0, c_1).$$

A.4.2 算法 A.5: Decrypt

输入

- 方案标志 $\text{scheme} \in \{\text{BGV}, \text{BFV}\}$;
- 秘密钥 $\mathbf{sk} = s$;
- 密文 $\mathbf{ct} = (c_0, c_1) \in R_Q^2$;
- 明文模数 $t = p^e$ 。

输出

- 明文多项式 $m(X) \in R_t$ 。

步骤

1. 计算 noisy plaintext

$$u := c_0 + c_1 s \in R_Q.$$

2. 若 `scheme=BGV`, 输出

$$m := u \bmod t.$$

3. 若 `scheme=BFV`, 令

$$m := \left\lfloor \frac{t}{Q} u \right\rfloor \bmod t.$$

4. 输出 $m(X) \in R_t$ 。

A.5 基础同态运算与 key switching

A.5.1 算法 A.6: Add

输入

- 两个同模数密文 $\mathbf{ct}^{(1)}, \mathbf{ct}^{(2)} \in R_Q^2$ 。

输出

- 密文 $\mathbf{ct}^{(\text{sum})} \in R_Q^2$ 。

步骤

$$(c_0, c_1) := (c_0^{(1)} + c_0^{(2)}, c_1^{(1)} + c_1^{(2)}).$$

A.5.2 算法 A.7: MulAndRelinearize

输入

- 两个密文 $\mathbf{ct}^{(1)} = (a_0, a_1)$ 、 $\mathbf{ct}^{(2)} = (b_0, b_1) \in R_Q^2$;
- 重线性化密钥 `rlk`。

输出

- 重线性化后的密文 $\mathbf{ct}^{(\text{mul})} \in R_Q^2$ 。

步骤

1. 先做 RLWE 密文乘法, 得到三元中间态:

$$d_0 := a_0 b_0, \quad d_1 := a_0 b_1 + a_1 b_0, \quad d_2 := a_1 b_1.$$

2. 调用 $\text{KeySwitchToDegreeOne}(d_2, \text{rlk})$, 把 $d_2 s^2$ 项折回一阶密钥表示。

3. 得到标准二元密文

$$\mathbf{ct}^{(\text{mul})} = (c_0, c_1).$$

4. 输出 $\mathbf{ct}^{(\text{mul})}$ 。

A.5.3 算法 A.8: KeySwitch

输入

- 待切换分量 $d \in R_Q$;
- key switching 密钥 $\mathbf{ksk}$;
- gadget base w 与分解长度 L 。

输出

- 两个环元 $(k_0, k_1) \in R_Q^2$, 满足它们在新秘密钥下等价承载 d 的旧密钥贡献。

步骤

1. 将 d 做 gadget decomposition:

$$d = \sum_{\ell=0}^{L-1} d^{(\ell)} w^\ell.$$

2. 初始化

$$k_0 := 0, \quad k_1 := 0.$$

3. 对每个 $\ell = 0, \dots, L-1$, 累加

$$(k_0, k_1) := (k_0, k_1) + d^{(\ell)} \cdot \mathbf{ksk}[\ell].$$

4. 输出 (k_0, k_1) 。

A.5.4 算法 A.9: Rotate

输入

- 密文 $\mathbf{ct} \in R_Q^2$;
- automorphism 索引 $j \in \mathbb{Z}_m^\times$;
- rotation 密钥 $\mathbf{rotk}[j]$ 。

输出

- 旋转后的密文 $\mathbf{ct}' \in R_Q^2$ 。

步骤

1. 对密文每个分量施加 automorphism:

$$(\tilde{c}_0, \tilde{c}_1) := (\tau_j(c_0), \tau_j(c_1)).$$

2. 注意此时 $(\tilde{c}_0, \tilde{c}_1)$ 是在新秘密钥 $\tau_j(s)$ 下有效的密文。
3. 使用 $\mathbf{rotk}[j]$ 做 key switching, 把秘密钥切回 s 。
4. 输出新的标准密文 $\mathbf{ct}'$ 。

A.6 大线性变换: weighted rotations、BSGS 与 hoisting

A.6.1 算法 A.10: EvalLinearTransform_BSGS

输入

- 密文 $\mathbf{ct}$;
- 某一维上的线性变换权重 $\{\kappa(v)\}_{v=0}^{n-1}$;
- 该维 rotation 算子 ρ^v ;
- BSGS 分块 $n = n_1 n_2$;
- 对应 rotation 密钥集合 $\mathbf{rotk}$;
- 是否启用 hoisting 的标记 `UseHoisting`。

输出

- 线性变换后密文 $\mathbf{ct}_{\text{out}}$ 。

步骤

1. 把索引写成

$$v = a + bn_1, \quad 0 \leq a < n_1, \quad 0 \leq b < n_2.$$

2. 若启用 hoisting, 则先对 **ct** 做可共享的 decomposition 预处理。
3. 预计算所有 baby-step rotations:

$$\mathbf{B}[a] := \rho^a(\mathbf{ct}), \quad a = 0, \dots, n_1 - 1.$$

4. 对每个 giant-step 索引 b , 构造中间块

$$\mathbf{T}[b] := \sum_{a=0}^{n_1-1} \kappa(a + bn_1) \cdot \mathbf{B}[a].$$

5. 对每个 b , 再施加 giant-step rotation:

$$\mathbf{G}[b] := \rho^{bn_1}(\mathbf{T}[b]).$$

6. 求和:

$$\mathbf{ct}_{\text{out}} := \sum_{b=0}^{n_2-1} \mathbf{G}[b].$$

7. 输出 $\mathbf{ct}_{\text{out}}$ 。

A.7 同态计算主循环

A.7.1 算法 A.11: EvalCircuitWithBootstrap

输入

- 方案标志 $\text{scheme} \in \{\text{BGV}, \text{BFV}\}$;
- 初始密文 $\mathbf{ct}_0$;
- 待计算电路 / 程序 $\mathcal{C}$;
- 评估密钥 EvalKeys ;
- 编码与线性层元数据 $(\text{EncMeta}, \text{LinMeta})$;
- digit extraction / rounding 元数据 DigitMeta ;
- 噪声预算阈值 NoiseThreshold 。

输出

- 最终密文 $\mathbf{ct}_{\text{final}}$ 。

步骤

1. 令

$$\mathbf{ct} \leftarrow \mathbf{ct}_0.$$

2. 按拓扑顺序遍历电路 $\mathcal{C}$ 的每个门 / 操作。

3. 若当前门为加法，则调用 **Add**。

4. 若当前门为乘法，则：

4.1. 调用 **MulAndRelinearize**；

4.2. 若实现包含模数切换，则执行 **ModSwitch**。

5. 若当前门为 rotation / automorphism，则调用 **Rotate**。

6. 若当前门为一般线性变换（如 **CoeffToSlot**、**SlotToCoeff**、**trace**、**selection** 等），则调用 **EvalLinearTransform_BSGS** 或其多维扩展。

7. 每一步后估计当前噪声预算。若

$$\text{NoiseBudget}(\mathbf{ct}) < \text{NoiseThreshold},$$

则执行

$$\mathbf{ct} \leftarrow \text{Bootstrap}(\mathbf{ct}).$$

8. 所有门执行结束后，输出

$$\mathbf{ct}_{\text{final}} := \mathbf{ct}.$$

A.8 Bootstrapping 的核心伪代码

A.8.1 算法 A.12: Bootstrap

输入

- 方案标志 $\text{scheme} \in \{\text{BGV}, \text{BFV}\}$ ；
- 当前密文 $\mathbf{ct}$ ；
- 公共参数 **Params**；

- 评估密钥 $\text{EvalKeys} = (\text{rlk}, \text{rotk}, \text{btk})$;
- 编码元数据 EncMeta ;
- 线性层元数据 LinMeta ;
- digit extraction / rounding 元数据 DigitMeta ;
- 打包模式 $\text{PackingMode} \in \{\text{Sparse}, \text{Full}\}$ 。

输出

- 刷新后的密文 ct_{fresh} 。

步骤

1. **ModulusRaise**: 把当前密文从最低有效层提升到 bootstrapping 模数:

$$\text{ct}^\uparrow \leftarrow \text{ModulusRaise}(\text{ct}, Q_{\text{boot}}).$$

2. **Homomorphic Linear Decryption**: 利用 btk 同态计算 noisy plaintext:

$$\text{ct}_u \leftarrow \text{LinearDecryptEval}(\text{ct}^\uparrow, \text{btk}).$$

其中:

- BGV 语义下 ct_u 加密

$$u = m + tr;$$

- BFV 语义下 ct_u 加密

$$u = \Delta m + e.$$

3. **CoeffToSlot**:

$$\text{ct}_{\text{slot}} \leftarrow \text{EvalLinearTransform_BSGS}(\text{ct}_u, \text{LinMeta.CTS}).$$

4. 如为 fully packed, 则 unpack:

$$\mathcal{S} \leftarrow \begin{cases} \{\text{ct}_{\text{slot}}\}, & \text{PackingMode}=\text{Sparse}, \\ \text{Unpack}(\text{ct}_{\text{slot}}, \text{LinMeta}), & \text{PackingMode}=\text{Full}. \end{cases}$$

5. 核心非线性修复:

5.1. 若 $\text{scheme}=\text{BGV}$, 对每个流 $s \in \mathcal{S}$ 调用

$\text{DigitExtract_ModT}(s, \text{DigitMeta}),$

得到已清理流集合 $\mathcal{S}_{\text{clean}}$ 。

5.2. 若 $\text{scheme}=\text{BFV}$, 对每个流 $s \in \mathcal{S}$ 调用

$\text{ScaleAndRound}(s, \text{DigitMeta}),$

得到 $\mathcal{S}_{\text{clean}}$ 。

6. 如为 **fully packed**, 则 **repack**:

$$\mathbf{ct}_{\text{slot}}^{\text{clean}} \leftarrow \begin{cases} \mathcal{S}_{\text{clean}}[0], & \text{PackingMode}=\text{Sparse}, \\ \text{Repack}(\mathcal{S}_{\text{clean}}, \text{LinMeta}), & \text{PackingMode}=\text{Full}. \end{cases}$$

7. **SlotToCoeff**:

$$\mathbf{ct}_{\text{poly}} \leftarrow \text{EvalLinearTransform_BSGS}(\mathbf{ct}_{\text{slot}}^{\text{clean}}, \text{LinMeta.STC}).$$

8. 收尾归一化: 必要时做:

- key switching;
- relinearization;
- modulus switching 回到正常计算模数层。

记最终结果为

$$\mathbf{ct}_{\text{fresh}}.$$

9. 输出 $\mathbf{ct}_{\text{fresh}}$ 。

A.9 unpack / repack 的伪代码

A.9.1 算法 A.13: Unpack

输入

- fully packed slot 密文 $\mathbf{ct}_{\text{slot}}$;
- 内部维数 $d = 2^r$;
- 线性层元数据 LinMeta , 其中包含:
 - 对应的内部 automorphism;
 - 每一层奇支归一化所需明文因子;
 - 统一基/局部基切换信息。

输出

- 一组 d 条 sparse packed 标量流

$$\mathcal{S} = \{\mathbf{s}^{(0)}, \dots, \mathbf{s}^{(d-1)}\}.$$

递归步骤

1. 若当前内部宽度 $w = 1$, 则直接返回当前流。
2. 取能实现

$$\zeta \mapsto -\zeta$$

的内部 automorphism, 记为 $\sigma_{w/2}$ 。

3. 令

$$\mathbf{ct}_{\text{flip}} := \sigma_{w/2}(\mathbf{ct}_{\text{slot}}).$$

4. 构造偶支:

$$\mathbf{ct}_{\text{even}} := \frac{1}{2}(\mathbf{ct}_{\text{slot}} + \mathbf{ct}_{\text{flip}}).$$

5. 构造奇支:

$$\mathbf{ct}_{\text{odd}} := \nu_w \cdot \frac{1}{2}(\mathbf{ct}_{\text{slot}} - \mathbf{ct}_{\text{flip}}),$$

其中 ν_w 是本层预计算的归一化明文因子, 用于把奇次项重新归到同一标准形。

6. 递归调用:

$$\mathcal{S}_{\text{even}} := \text{Unpack}(\mathbf{ct}_{\text{even}}, w/2),$$

$$\mathcal{S}_{\text{odd}} := \text{Unpack}(\mathbf{ct}_{\text{odd}}, w/2).$$

7. 返回

$$\mathcal{S} := \mathcal{S}_{\text{even}} \parallel \mathcal{S}_{\text{odd}}.$$

A.9.2 算法 A.14: Repack

输入

- 一组 d 条 sparse packed 标量流

$$\mathcal{S} = \{\mathbf{s}^{(0)}, \dots, \mathbf{s}^{(d-1)}\};$$

- 线性层元数据 `LinMeta`。

输出

- fully packed slot 密文 $\mathbf{ct}_{\text{slot}}$ 。

递归步骤

1. 若当前只有 1 条流，则直接返回该流。
2. 把当前流集合均分成两半：

$$\mathcal{S}_{\text{even}}, \mathcal{S}_{\text{odd}}.$$

3. 递归重构：

$$\mathbf{ct}_{\text{even}} := \text{Repack}(\mathcal{S}_{\text{even}}), \quad \mathbf{ct}_{\text{odd}} := \text{Repack}(\mathcal{S}_{\text{odd}}).$$

4. 用与 unpack 对偶的线性组合把两支合回一层更宽的内部结构：

$$\mathbf{ct}_{\text{slot}} := \mathbf{ct}_{\text{even}} + \nu_w^{-1} \cdot \mathbf{ct}_{\text{odd}},$$

并应用相应的逆向内部 automorphism / 组合规则。

5. 返回 $\mathbf{ct}_{\text{slot}}$ 。

注 A.1. 上述 Unpack / Repack 伪代码刻意保留了归一化因子 ν_w 的抽象形式。具体实现中，它对应奇支重新编号所需的某个预计算明文乘子，其具体表达依赖于所采用的幂基与 automorphism 约定。

A.10 BGV 的 digit extraction 与 BFV 的 rounding 伪代码

A.10.1 算法 A.15: DigitExtract_ModT (BGV)

输入

- 一个 scalar-slot 密文流 $\mathbf{s}$ ，其每个 slot 满足

$$u = m + tr, \quad t = p^e;$$

- DigitMeta，其中包含：
 - 块基数 $B = p^k$;
 - 块数 $h = \lceil e/k \rceil$;
 - 低块提取子电路 LowBlock_B ;
 - 必要的 correction / carry 元数据。

输出

- 一个同模数密文流 $\mathbf{s}_{\text{clean}}$ ，其 slot 中加密的是

$$m = u \bmod t.$$

步骤

1. 初始化

$$\mathbf{z}^{(0)} := \mathbf{s}, \quad \mathbf{acc} := 0.$$

2. 对 $r = 0, 1, \dots, h - 1$ 重复:

2.1. 调用低块提取子电路:

$$\mathbf{d}^{(r)} := \text{LowBlock}_B(\mathbf{z}^{(r)}),$$

其中每个 slot 现在承载当前最低 base- B block。

2.2. 把该块累加到结果:

$$\mathbf{acc} := \mathbf{acc} + B^r \cdot \mathbf{d}^{(r)}.$$

2.3. 从当前值中剥离最低块并右移一块:

$$\mathbf{z}^{(r+1)} := \frac{\mathbf{z}^{(r)} - \mathbf{d}^{(r)}}{B}.$$

2.4. 若采用 correction / carry cancellation, 则在本轮或相邻轮插入相应修正步骤。

3. 最终输出

$$\mathbf{s}_{\text{clean}} := \mathbf{acc}.$$

A.10.2 算法 A.16: ScaleAndRound (BFV)

输入

- 一个 scalar-slot 密文流 $\mathbf{s}$, 其每个 slot 近似承载

$$u = \Delta m + e;$$

- DigitMeta, 其中包含:
 - Δ 或 (t, Q) 的缩放信息;
 - 舍入/区间修正电路;
 - 必要的 correction 元数据。

输出

- 一个密文流 $\mathbf{s}_{\text{clean}}$, 其 slot 中加密的是 m 。

步骤

1. 先把当前值缩放回明文尺度：

$$\mathbf{y} := \frac{t}{Q} \cdot \mathbf{s} \quad \text{或等价的实现形式.}$$

2. 调用 rounding / correction 子电路，把 $\mathbf{y}$ 的每个 slot 舍入到最近的整数 residue。
3. 把结果约到 $\mathbb{Z}_t$ 中，得到

$$\mathbf{s}_{\text{clean}}.$$

4. 输出 $\mathbf{s}_{\text{clean}}$ 。

A.11 从数据到数据的端到端伪代码

A.11.1 算法 A.17: EndToEnd_BGVBFV_BootstrapWorkflow

输入

- 原始数据：
 - sparse packed: $(m_0, \dots, m_{\ell-1})$;
 - fully packed: $(m_{i,j})_{0 \leq i < \ell, 0 \leq j < d}$;
- 方案标志 $\text{scheme} \in \{\text{BGV}, \text{BFV}\}$;
- 待同态求值的程序 / 电路 $\mathcal{C}$;
- 安全参数、明文模数、bootstrapping 模式等 setup 输入。

输出

- 最终明文结果（解密并解码后的原始数据格式）。

步骤

1. 调用 SetupBGVBFVBootstrap，得到：

$$\text{Params}, (\text{pk}, \text{sk}), \text{EvalKeys}, \text{EncMeta}, \text{LinMeta}, \text{DigitMeta}.$$

2. 调用 EncodeData，把原始数据编码成明文多项式

$$m(X) \in R_t.$$

3. 调用 `Encrypt`，得到初始密文

$$\mathbf{ct}_0.$$

4. 调用 `EvalCircuitWithBootstrap`，在需要时自动执行 bootstrapping，得到最终密文

$$\mathbf{ct}_{\text{final}}.$$

5. 调用 `Decrypt`，得到最终明文多项式

$$m_{\text{final}}(X) \in R_t.$$

6. 调用 `DecodeData`，把

$$m_{\text{final}}(X)$$

还原回原始数据格式：

- sparse packed: $(y_0, \dots, y_{\ell-1})$;
- fully packed: $(y_{i,j})_{0 \leq i < \ell, 0 \leq j < d}$.

7. 输出最终数据。

A.12 附录总结：如何读这套伪代码

本附录把 BGV/BFV bootstrapping 的端到端流程统一写成了：

参数生成 $\rightarrow$ 编码 $\rightarrow$ 加密 $\rightarrow$ 同态计算 $\rightarrow$ 必要时 bootstrapping $\rightarrow$ 解密 $\rightarrow$ 解码

其中最值得注意的是三条主线：

(1) **表示主线：**

$$\text{data} \leftrightarrow m(X) \leftrightarrow \text{slot view}$$

由 `EncodeData`、`DecodeData`、`CoeffToSlot`、`SlotToCoeff` 负责；

(2) **密文主线：**

$$\mathbf{ct} \xrightarrow{\text{RLWE 运算}} \mathbf{ct}'$$

由 `Encrypt`、`Add`、`Mul`、`Relinearize`、`Rotate`、`KeySwitch` 等负责；

(3) **刷新主线：**

$$\mathbf{ct} \rightarrow \text{Enc}(u) \rightarrow \text{Enc}(m)$$

由 `Bootstrap`、`Unpack/Repack`、`DigitExtract_ModT` 或 `ScaleAndRound` 负责。

如果在汇报中需要最简洁的一页版算法总图，可直接保留算法 A.17，并把 A.12(Bootstrap) 作为其唯一展开子程序；若需要技术细节，则再把 A.10、A.13、A.14、A.15、A.16 展开。